CONLANGING WITH INKHAVEN

Developing a Constructed Language

A Beginner's Guide to Conlanging with Inkhaven

Vladimir Ulogov

2026 · examples assume Inkhaven 3.18.0 or newer

Contents

Who this book is for 1
How this book is organised 2
What you will need 2
Part I — Foundations 3
1 — What is a constructed language? 4
Why make one? 5
The parts of a language 5
A priori and a posteriori 6
Naturalism: should it feel real? 7
2 — Meet Inkhaven 8
Two ways to talk to Inkhaven 8
Where your language lives 9
The role of artificial intelligence 10
3 — Getting set up 12
1. Install Inkhaven 12
2. Create a project 13
3. Create your language 13
4. How you edit a language 14
5. Connect an AI provider (optional) 15
Part II — The Sounds of Your Language 17
4 — Phonemes: the sounds 18
Consonants and vowels 19
A note on the IPA 19
Building your inventory 20
Grouping sounds into classes 21
Checking your work 22
5 — Syllables and word shapes 23
What a syllable is 23
Describing word shapes with templates 24
Generating words 25
Inspecting a single word 25
6 — Phonotactics: the rules of combination 27
Constraints 28
Putting it together 29
7 — Allophony: sounds that shift 31

Underlying and surface forms 32
Writing a rule 32
Adding allophony to Eldar 33
8 — Stress, spelling, and tone 35
Stress 35
Romanization: writing it down 36
Tone (optional) 37
Part III — Words 40
9 — Building a vocabulary 41
Adding a word 41
Recording more than the basics 42
Finding words again 43
Importing many words at once 43
Using your words while you write 44
10 — A healthy lexicon 45
Auditing for problems 45
Generating words with AI 46
Finding undefined words in your writing 47
The Oracle watches as you write 48
What is your lexicon still missing? 48
Part IV — Grammar 50
11 — Morphology: words that change 51
Morphemes and affixes 52
Declaring your affixes 52
Paradigms: the forms of a word 54
Generating the forms 55
Interlinear glossing 55
Beyond prefixes and suffixes 56
Agreement 57
12 — Building new words 59
Declaring a derivation rule 60
Coining derived words 60
13 — Typology: the shape of your grammar 62
The questionnaire 63
Three choices worth understanding 63
Answering a feature 64
Putting words into a sentence 65
Saying no, and asking: negation and questions 66

Building bigger sentences: relative clauses and coordination 67
14 — Idioms and metaphors 70
Idioms 71
Conceptual metaphors 71
Reviewing what you have 72
Part V — A History for Your Language 74
15 — Sound change and proto-languages 75
Sound change is just allophony over time 76
Declaring a descent 76
Evolving a word 77
Evolving the whole vocabulary 77
16 — Language families 79
Cognates: the same word, evolved 80
The family tree 81
Looking backward: reconstruction 81
Is your history believable? 82
Keeping a register of hypotheses 82
Part VI — A Language in a World 84
17 — Dialects and registers 85
The one idea: a variety is a delta 86
Declaring varieties 87
Two ways a variety differs 88
Rendering speech in a variety 88
The dialectology table 89
Letting the AI propose a dialect 89
18 — Languages in contact 91
Borrowing: a word crosses over 92
Declaring how a language borrows 92
Borrowing a word 93
When a whole region converges 94
Reading the convergence 94
AI help for contact 95
19 — Speech communities and ecology 97
Linking languages to your world 98
Which variety, exactly 98
The ecology view 99
Speaking as a character 99
Part VII — A Writing System 101

20 — Designing glyphs 102
What a glyph file looks like 103
What makes a good font glyph 103
Drawing a glyph with AI 104
21 — Building the font 106
Binding a glyph 106
The font lives in your language 107
Compiling the font 108
22 — Complex scripts and typing 110
Composed blocks 110
Layout-time arrangement 111
Typing the script 112
Part VIII — Producing the Books 114
23 — Producing the books 115
A quick word about output formats 116
The dictionary 116
The reference grammar 116
The study guide (AI) 117
The learner's tutorial (AI) 117
Beyond the books 118
Creative text: names, verse, and ritual 118
Part IX — Putting the Language to Work 120
24 — Translating with your language 121
Translating into, and out of 122
A memory that learns your voice 122
Measuring the engine against your own texts 123
Taking the memory with you 123
Part X — Sharing Your Language 125
25 — Sharing your language 126
The landscape: who uses what 127
Exporting your lexicon 129
Importing a lexicon 130
A round-trip workflow 131
Part XI — A Complete Walkthrough 133
26 — A complete walkthrough 134
1. Project and language 135
2. Phonology 135
3. A vocabulary 136

4. Grammar 136
5. A writing system 137
6. The books 138
You have built a language 138
Part XII — Scripting Your Language 139
27 — Building your conlang with Bund 140
Two ways to say the same thing 141
What Bund is 142
Why a script can do more than a file 142
A whole language in one breath 143
Three kinds of word, and a safety gate 144
Building artefacts the native way 144
Running a script 145
Part XIII — Analysing Your Language 146
28 — The Inner Linguist 147
Opening it 148
The views 148
The grounded chat 149
Sessions 149
Slash-commands in the chat 149
29 — Measuring, testing, and tracing 153
The shape of the sound system 154
Is the inventory natural? 154
Which contrasts carry the weight? 155
Does the grammar obey the universals? 155
Tracing a change before you make it 156
A corpus of your own texts 157
30 — Syntax: parsing and structure 159
Reading a word backwards: the parser 160
Who did what to whom: argument linking 160
The shape of a sentence: the X-bar tree 161
Moving a piece: fronting and traces 162
Who can refer to whom: binding 162
At your fingertips 163
31 — The Oracle: judging well-formedness 164
Is this a possible word? 165
Is this a possible clause? 165
The Oracle watching your prose 166

At your fingertips ... 166
32 — Interlinear glossed text ... 168
Glossing a sentence ... 168
Keeping your texts ... 169
Curating the translation ... 170
Out to the page ... 170
Appendix A — Command reference ... 172
Setup ... 173
Phonology ... 173
Lexicon ... 173
Grammar ... 174
Diachronics ... 176
Analysis ... 177
Morphology and syntax ... 178
The Linguistic companion ... 179
Sociolinguistics and contact ... 179
Translation ... 180
Writing systems ... 181
Output ... 181
Worldbuilding links ... 182
Appendix B — Configuration block reference ... 183
Meta chapter — the overview block ... 183
Phonology chapter ... 184
Grammar chapter ... 186
Dictionary chapter ... 188
SPE rule notation ... 189
Appendix C — Bund API reference ... 190
The safety gate ... 191
Inspectors — sounds and words ... 191
Inspectors — grammar and sentences ... 192
Inspectors — analysis and history ... 192
Inspectors — creative text ... 192
Inspectors — sociolinguistics ... 193
Inspectors — translation ... 193
Data constructor ... 194
Mutators ... 194
AI-backed words ... 194
File output ... 195

Appendix D — Glossary of terms 196

Before You Begin

This is a book about making a language — one that has never existed, with sounds you choose, words you coin, a grammar you design, and even its own alphabet. The craft is called **conlanging**, and the language you build is a **constructed language**, or **conlang**. People make conlangs for novels and games, for films, for the pure pleasure of the puzzle, or simply to understand how human language works by building one from the inside.

You will do all of it inside **Inkhaven** — a writing tool with a built-in conlang workshop. By the last page you will have taken a language from an empty project to three finished, printable books: a dictionary, a reference grammar, and a beginner's textbook for your own invented tongue.

Who this book is for

This guide assumes **no prior knowledge**. You do not need to be a linguist: every technical term is defined the first time it appears, in a marked box like the one below. You do not need to be an Inkhaven expert: we start from installing it and creating your first project. If you can type commands into a terminal and edit a text file, you have everything you need.

> TERM **Linguistics**
>
> The scientific study of human language — its sounds, its words, its grammar, and how these change over time. You will meet small, friendly pieces of it throughout this book. You do not need a background in it; that is what the term boxes are for.

How this book is organised

The book follows the natural order in which a language is built, one layer at a time. Each layer rests on the one before:

1. **Foundations** — what a conlang is, what Inkhaven is, and getting set up.
2. **The sounds** — choosing the building blocks of pronunciation and the shapes words can take.
3. **Words** — coining a vocabulary and keeping it consistent.
4. **Grammar** — how words change and combine to make meaning.
5. **History** — giving your language a past, and spawning related languages.
6. **A writing system** — designing an alphabet and compiling it into a real font.
7. **The books** — turning all of it into printed documents.
8. **A complete walkthrough** — building one small language end to end.

At the back you will find three references you will return to often: a list of every command, a list of every configuration block, and a glossary of every linguistic term used in the book.

> **HOW TO READ THE EXAMPLES**
>
> Lines you type into a terminal are shown in a monospace box and begin with the program name, like `inkhaven language init Eldar`. Configuration you write into your language is shown as `hjson` blocks. Throughout, we build one running example language called **Eldar**; in the final chapter we build a second, **Avesha**, from nothing to finished books, so you can watch the whole process at once.

What you will need

Three things, all free, all covered in Chapter 3: a copy of **Inkhaven**; a terminal to run it in; and — for the parts that use artificial intelligence, which are always optional — an **API key** for an AI provider. We will explain each when we reach it. You will also want **Typst** (a free typesetting program) if you wish to turn your finished books into PDFs, but that too can wait.

Turn the page, and let us begin with the most basic question of all: what, exactly, is a constructed language?

PART I

Foundations

CHAPTER 1

1

What is a constructed language?

A **constructed language** is a language that someone designed on purpose, rather than one that grew up naturally among a community of speakers over centuries. English, Mandarin, and Swahili are **natural languages** — nobody invented them; they evolved. Quenya (spoken by the Elves in Tolkien's books), Klingon (from **Star Trek**), and Dothraki (from **Game of Thrones**) are constructed languages — each was built, word by word and rule by rule, by a person.

TERM **Constructed language (conlang)**

A language deliberately created by one or more people, with an invented vocabulary and grammar, instead of arising naturally. The activity of making one is called **conlanging**, and a person who does it is a **conlanger**.

Why make one?

People build conlangs for many reasons, and yours is as good as any:

1. **Storytelling.** A believable invented language makes a fictional world feel real. A few words of a character's native tongue carry more weight than pages of description.
2. **Worldbuilding.** Place names, character names, and inscriptions all flow from a language. A consistent one makes a map or a history hang together.
3. **Art and play.** A language is an enormous, beautiful puzzle. Many people build them purely for the joy of the craft.
4. **Understanding.** Building a language teaches you, from the inside, how real languages solve the problem of turning thoughts into sound.

The parts of a language

Every human language, natural or constructed, has the same broad layers. This book has one part for each, and they build on one another in order:

TERM **Phonology**

The sound system of a language: which sounds it uses and how they may be combined. This is the foundation — you choose it first, because everything else is spoken (or written) using these sounds. **Part II.**

TERM **Lexicon**

The vocabulary — the full set of words. A single word in the lexicon is a **lexeme**. Building the lexicon means coining words and recording what they mean. **Part III.**

TERM Grammar

The rules for how words change their shape and combine into phrases and sentences. It has two halves: **morphology** (how a single word changes form, for example to mark "more than one") and **syntax** (how words are ordered into sentences). **Part IV.**

TERM Diachronics

The history of a language — how its sounds and words change over time, and how one language splits into a family of related daughter languages. Optional, but it is what gives a language depth and realism. **Part V.**

TERM Sociolinguistics

How a language varies across its speakers and lives among its neighbours: its **dialects** and **registers**, the words it **borrows** from other tongues, and the speech of the particular people and places of your world. **Part VI.**

TERM Writing system (orthography)

How the language is written down: an alphabet, a syllabary, or a set of symbols, and the rules for using them. You can borrow the Latin alphabet you are reading now, or invent a script of your own. **Part VII.**

A priori and a posteriori

Conlangers use two Latin phrases you may meet elsewhere. An **a priori** language is built from scratch, owing nothing to any existing language — its words and grammar are wholly invented. An **a posteriori** language is based on real languages, like the way Esperanto draws its vocabulary from European tongues. This book builds an **a priori** language, because it is the clearest way to see every part of the process. Nothing stops you from leaning on a favourite real language for inspiration as you go.

Naturalism: should it feel real?

A common goal is a **naturalistic** conlang — one that could plausibly be a real human language, with the kinds of patterns and irregularities that natural languages have. Inkhaven is built to help here: it knows about the typical shapes of sound systems, the common ways languages mark grammar, and the natural ways sounds change over time, and it will gently steer you toward choices that feel real. But naturalism is a choice, not a rule. If you want a perfectly regular, clockwork language, you can build that too.

> **YOU DO NOT HAVE TO DO EVERYTHING**
>
> A language does not need a thousand words, a deep history, and a hand-drawn alphabet to be useful or satisfying. A few dozen well-chosen words and a simple grammar are plenty for naming places and characters in a story. Read the parts you need; the book is built so each pillar stands on its own once you have the foundations.

> **WHAT YOU LEARNED**
>
> - A **conlang** is a language built on purpose; the craft is **conlanging**.
> - Every language has layers: **phonology** (sounds), **lexicon** (words), **grammar** (rules), optional **diachronics** (history), and a **writing system**.
> - We build them in that order, each resting on the last.
> - **Naturalism** — making the language feel like a real one — is a goal you can choose or ignore.

CHAPTER 2

2

Meet Inkhaven

Inkhaven is a writing tool — a program for authors who write books, with a text editor, a place to keep notes and characters and research, and tools for turning a manuscript into a finished, formatted book. Built into it is a complete **conlang workshop**: the set of features, called the **ConLang Suite**, that this book is about. You do not need to know anything about the rest of Inkhaven to use it.

Two ways to talk to Inkhaven

Inkhaven has a full-screen interface you can run in a terminal — the **TUI**, for "text user interface" — where you write and edit. But almost everything in this book is

done through **commands**: short lines you type at a terminal prompt, each beginning with the word `inkhaven`. This is deliberate. Commands are precise, they are easy to show in a book, and they are easy to repeat.

> TERM **Command line**
>
> A way of using a program by typing instructions, rather than clicking buttons. You type a line, press Enter, and the program does one thing and prints the result. The conlang tools all live under one command word: `inkhaven language …`.

Every conlang command has the same shape:

```
inkhaven language <action> <language-name> [options]
```

For example, `inkhaven language add-word Eldar makil --type noun --translation sword` runs the **add-word** action on the language named **Eldar**, with three pieces of extra information. Throughout the book, the word after `language` is the action, the next word is usually your language's name, and the parts that begin with `--` are **options** that fine-tune what happens.

> GETTING HELP ON ANY COMMAND
>
> Add `--help` to any command to see exactly what it does and what options it takes — for example `inkhaven language add-word --help`. This works for every command in the book and is the fastest way to check a detail.

Where your language lives

Inside an Inkhaven project, your work is organised into **books** — not printed books, but containers for related material. Books can nest, and a book inside another is a **sub-book**. There is a special built-in one called the **Language** book, and each language you create becomes a sub-book inside it. When you create a language, Inkhaven sets up five **chapters** inside its sub-book, each holding one kind of information:

TERM **The Language book**

Inkhaven's home for constructed languages. Creating a language with `inkhaven language init` adds a sub-book under it, pre-divided into five chapters: **Meta**, **Dictionary**, **Grammar**, **Phonology**, and **Sample texts**.

You will spend most of your time putting information into these chapters — sounds into **Phonology**, words into **Dictionary**, grammar rules into **Grammar**. The book is always the master copy of your language; the tools read from it and write back to it. We will see exactly how in the next chapter.

The role of artificial intelligence

Some of Inkhaven's conlang tools can call on an **AI** (a large language model, the kind of program behind chat assistants) to help — to invent a batch of words on a theme, to draft a letter-shape from a description, or to write a study guide. These features are powerful, but they are always **optional** and always **advisory**: the AI proposes, and nothing is saved to your language until you approve it. Wherever a command uses AI, the book says so plainly, and you can skip it.

TERM **AI provider and API key**

An **AI provider** is a company that runs a language model you can connect to (examples include DeepSeek, OpenAI, and others). An **API key** is a secret password that lets your copy of Inkhaven use that provider. You set one up once; Chapter 3 shows how. If you choose not to use AI at all, every essential part of building a language still works without it.

WHAT NEEDS AI, AND WHAT DOES NOT

Does not need AI: choosing sounds, generating word-shapes, building the lexicon by hand, all grammar and morphology, sound change and family trees, importing and compiling fonts, and producing the dictionary and grammar reference. **Uses AI (optional):** generating themed vocabulary, drafting glyph artwork from a description, reconstructing a proto-form, the grammar **study guide**, and the learner's **tutorial**.

WHAT YOU LEARNED

- Inkhaven is a writing tool with a built-in **ConLang Suite**.
- You drive the conlang tools with **commands** of the form `inkhaven language <action> <name> [options]`; `--help` explains any of them.
- Each language lives in a **Language** sub-book with five chapters: Meta, Dictionary, Grammar, Phonology, Sample texts.
- **AI** features are optional and advisory — nothing is saved without your approval, and everything essential works without AI.

CHAPTER 3

3

Getting set up

This chapter takes you from nothing to an empty language, ready to fill in. Five short steps: install Inkhaven, create a project, create your language, learn how to edit its chapters, and (optionally) connect an AI provider.

1. Install Inkhaven

Inkhaven is a single program. If you have the Rust toolchain installed, the simplest route is:

```
cargo install inkhaven
```

This downloads and builds the latest release and puts an `inkhaven` command on your system. Pre-built downloads are also available from the project's releases page. To check it worked, run:

```
inkhaven --version
```

You should see a version number of `1.3.19` or newer (this book's examples assume that release — the syntax, interchange, and `compose` features in later chapters arrived in it). If the command is not found, make sure the install location is on your system's `PATH`; the install step prints where it placed the program.

2. Create a project

Everything you do lives inside a **project** — a folder Inkhaven manages. Create one with `init`, giving it a path:

```
inkhaven init ~/eldar-project
```

This makes the folder, sets up Inkhaven's internal storage, and prints a short summary. From now on, run conlang commands from inside that folder:

```
cd ~/eldar-project
```

> **ONE PROJECT, MANY LANGUAGES**
>
> A single project can hold as many languages as you like — useful when you build a family of related tongues later (Part V). Each is a separate language under the same project.

3. Create your language

Now create the language itself. We will call ours **Eldar**:

```
inkhaven language init Eldar
```

This adds an **Eldar** sub-book under the Language book and creates its five chapters: **Meta**, **Dictionary**, **Grammar**, **Phonology**, and **Sample texts**. The language

exists now, but it is empty — no sounds, no words, no rules. Filling those in is the rest of the book.

You can list your languages at any time:

```
inkhaven language list
```

4. How you edit a language

Here is the one mechanic that everything else depends on, so read it carefully.

Most of what defines a language — its sounds, its grammar rules, its history — is written as small blocks of structured text in a format called **HJSON**, and placed into the right chapter of the language book. HJSON is a gentle, human-friendly way to write structured data: things in `{ … }` are collections of named fields, and things in `[ … ]` are lists.

> **TERM HJSON**
>
> A relaxed, human-readable data format (a friendlier cousin of JSON). You write fields as `name: value`, group them with curly braces `{ }`, and make lists with square brackets `[ ]`. Inkhaven reads these blocks to reconstruct your language.

On disk, each chapter of your language is a folder of small text files. To add a phonology block, for example, you create a text file — name it anything ending in `.typ`, say `inventory.typ` — in the Phonology chapter's folder and paste the block in. The chapters live under your project at:

```
books/language/<your-language>/04-phonology/
books/language/<your-language>/03-grammar/
books/language/<your-language>/05-sample-texts/
```

(The exact numbers may differ; the names are what matter.) After you add or change a file by hand, tell Inkhaven to notice it:

```
inkhaven reindex --adopt
```

This **adopts** any new files into the language so the tools can read them. You run it once after editing files; the commands that write changes for you (like `add-word`) do not need it.

> **THE ONE HJSON PITFALL TO REMEMBER**
>
> In HJSON, a value that is not in quotes runs to the end of the line. So always put quotes around short word-like values such as `kind: "consonant"` or `position: "suffix"`. If you forget, you will get a clear error pointing at the line. When in doubt, quote it.

We will write our first real block — the sound inventory — in the very next chapter, so this will quickly become familiar.

5. Connect an AI provider (optional)

Skip this section entirely if you do not want to use the AI features; everything essential works without it. To enable them, you tell Inkhaven which provider to use and give it your API key. You obtain a key by signing up with a provider (this book's live examples use **DeepSeek**); the provider gives you a long secret string. You make it available to Inkhaven through an **environment variable** — a named value your terminal holds. For DeepSeek, for instance:

```
export DEEPSEEK_API_KEY="your-secret-key-here"
```

An `export` like this lasts only for the **current** terminal session — open a new window and you would set it again. To make it permanent, add the same line to your shell's start-up file (`~/.zshrc`, `~/.bashrc`, or the equivalent). Then, on any command that uses AI, you select the provider with `--provider deepseek`. Other providers work the same way with their own key names. That is all the setup AI needs; we will use it sparingly and always say when.

WHAT YOU LEARNED

- Install with `cargo install inkhaven`; check with `inkhaven --version`.
- Create a project with `inkhaven init <path>`, then `cd` into it.
- Create a language with `inkhaven language init <name>` — it gets five chapters.
- Define a language by placing **HJSON** blocks into chapter folders, then running `inkhaven reindex --adopt`. Always quote short values.
- AI features are optional: set an API key and pass `--provider` when you want them.

PART II

The Sounds of Your Language

CHAPTER 4

4

Phonemes: the sounds

A language begins with its sounds. Before a single word can exist, you must decide what your language is **made of** — the small set of distinct sounds that its words are built from. These are its **phonemes**.

TERM **Phoneme**

A single distinct sound that can change the meaning of a word in a given language. In English, /p/ and /b/ are separate phonemes, because **pat** and **bat** mean different things. The slashes are the usual way to write a phoneme, to distinguish it from a letter. A language typically has somewhere between a dozen and a few dozen phonemes.

Consonants and vowels

Phonemes come in two broad families. **Vowels** are the open, sung sounds your voice glides through — the /a/ in **father**, the /i/ in **machine**. **Consonants** are the sounds made by closing or narrowing the mouth somewhere — /p/, /t/, /k/, /s/, /m/. Every word is a weave of the two.

TERM **Vowel and consonant**

A **vowel** is a sound made with open, unobstructed airflow (a, e, i, o, u and their many cousins). A **consonant** is a sound made by obstructing the airflow with the lips, tongue, or throat (p, t, k, s, m, n, l, r, and so on). Most syllables are one or more consonants around a vowel.

A note on the IPA

You will see phonemes written with the letters you know, but also with a few unusual symbols like /ʃ/ (the **sh** sound) or /ŋ/ (the **ng** at the end of **sing**). These come from the **International Phonetic Alphabet**, a worldwide system where one symbol always means exactly one sound. You do not need to learn it; for an a-priori language you may simply use ordinary letters for your sounds. But the IPA is handy when a sound has no obvious letter, and Inkhaven happily accepts either.

> **TERM International Phonetic Alphabet (IPA)**
>
> A standard set of symbols in which each symbol represents one and only one speech sound, used by linguists worldwide. Useful for sounds that ordinary spelling writes ambiguously (English **th** is two different sounds; IPA writes them /θ/ and /ð/). Inkhaven lets you give each phoneme an IPA symbol and a plain-letter spelling.

Building your inventory

Your full set of phonemes is your **inventory**. You declare it as a `phonemes` list inside a phonology block, placed in the **Phonology** chapter (Chapter 3 explained how to add a block). Each phoneme gives three things: its `ipa` symbol (the sound), an optional `romanize` spelling (how you will write it with ordinary letters), and its `kind` — `"consonant"` or `"vowel"`.

The `romanize` field earns its keep when the sound has no plain letter. In the inventory below the eighth consonant is a **tap** (IPA `ɾ`, the quick flicked "r" of Spanish **pero**); writing `romanize: "r"` lets you keep typing an everyday "r" while the engine knows the real sound. When the IPA symbol **is** an ordinary letter, the two simply match.

Here is a small starter inventory for Eldar — eight consonants and three vowels:

```
{
  phonemes: [
    { ipa: "p", romanize: "p", kind: "consonant" }
    { ipa: "t", romanize: "t", kind: "consonant" }
    { ipa: "k", romanize: "k", kind: "consonant" }
    { ipa: "s", romanize: "s", kind: "consonant" }
    { ipa: "m", romanize: "m", kind: "consonant" }
    { ipa: "n", romanize: "n", kind: "consonant" }
    { ipa: "l", romanize: "l", kind: "consonant" }
    { ipa: "ɾ", romanize: "r", kind: "consonant" }
    { ipa: "a", romanize: "a", kind: "vowel" }
    { ipa: "i", romanize: "i", kind: "vowel" }
    { ipa: "u", romanize: "u", kind: "vowel" }
  ]
}
```

Save this as a file in your language's `04-phonology` folder, then run `inkhaven reindex --adopt`. Eldar now has a sound system.

> **HOW BIG SHOULD THE INVENTORY BE?**
>
> Real languages range from about eleven phonemes to over a hundred, but most sit between twenty and forty. A small, balanced set like the eleven above is an excellent place to start — it is easy to pronounce, easy to spell, and gives plenty of room for words. You can always add more later. A rough rule of thumb for a natural feel: more consonants than vowels, and at least three vowels.

Grouping sounds into classes

Many rules you will write later apply not to one sound but to a whole **family** of sounds — "all consonants", "all vowels", "the stop consonants". To name such a family, you declare a **class**: a label and the list of phonemes in it. The two most useful are a class of all consonants and a class of all vowels, usually called `C` and `V`:

```
classes: {
  C: ["p", "t", "k", "s", "m", "n", "l", "ɾ"]
  V: ["a", "i", "u"]
}
```

(A class lists phonemes by their `ipa` symbol, which is why the tap appears as `ɾ` here, not as its `r` spelling.)

Add this `classes` field inside the same phonology block, beside `phonemes`. We will use `C` and `V` in the next chapter to describe the shapes words can take.

> **TERM Phoneme class**
>
> A named group of phonemes that behave alike for some rule — for example all consonants, all vowels, or all **front** vowels (those made with the tongue forward in the mouth, like **i** and **e**). Classes let you write a rule once for a whole family instead of repeating it for each sound.

Checking your work

You can confirm Inkhaven has read your inventory with the **stats** command, which prints a profile of the language:

```
inkhaven language stats Eldar
```

Early on it will simply report your inventory size — eight consonants and three vowels. As you add words later, the same command grows into a rich picture of how your sounds are actually used.

> **WHAT YOU LEARNED**
>
> - A **phoneme** is a meaning-distinguishing sound; the full set is your **inventory**.
> - Phonemes are **consonants** or **vowels**; you may write them with ordinary letters or **IPA** symbols.
> - Declare the inventory as a `phonemes` list in the Phonology chapter; give each an `ipa`, a `romanize` spelling, and a `kind`.
> - Group sounds into **classes** (like `C` and `V`) to write rules over whole families.
> - `inkhaven language stats` confirms what Inkhaven read.

CHAPTER 5

5

Syllables and word shapes

You have sounds. Now you decide how they fit together into **syllables**, and from those, into possible words. This is where a language starts to have a characteristic feel — the difference between a flowing tongue of open syllables (like `ta-ki-na`) and a crunchy one full of consonant clusters (like `strkth`).

What a syllable is

A **syllable** is one beat of speech, built around a vowel. The word **banana** has three: **ba-na-na**. Each syllable has up to three parts.

TERM **Syllable**

A unit of pronunciation containing one vowel sound, optionally wrapped in consonants. It has three parts: the **onset** (consonants before the vowel), the **nucleus** (the vowel itself, the core), and the **coda** (consonants after the vowel). In **cat**, the onset is /k/, the nucleus is /a/, and the coda is /t/.

TERM **Onset, nucleus, coda**

The three slots in a syllable. The **nucleus** is the required centre (a vowel). The **onset** is the consonant(s) before it, and the **coda** the consonant(s) after it; both are usually optional. A syllable with no coda (like **ta**) is called **open**; one with a coda (like **tan**) is **closed**.

Describing word shapes with templates

How do you tell Inkhaven what a possible Eldar word looks like? With a **template**: a small pattern written with your class names, where `C` means "a consonant", `V` means "a vowel", and parentheses mean "optional". The pattern `C V (C)` reads: a consonant, then a vowel, then optionally another consonant — that is, the syllables **ta** or **tan**, but not **atra**.

TERM **Syllable template**

A pattern describing the allowed shape of a syllable, written with phoneme class names. `C V` is consonant-plus-vowel; `C V C` adds a final consonant; `(C)` marks an optional slot. A language usually allows a handful of templates.

You declare templates in the phonology block, under a `templates` field. Each template can carry a `weight` — a number saying how common that shape should be when Inkhaven generates words (higher means more frequent). Here we let Eldar have open `CV` syllables and closed `CVC` ones, with open ones twice as common:

```
templates: {
  root: [
    { pattern: "C V",   weight: 2.0 }
```

```
    { pattern: "C V C", weight: 1.0 }
  ]
}
```

The name `root` groups these as the shapes used for word **roots** (the core of a word). Add this field beside `phonemes` and `classes`, and reindex.

Generating words

With an inventory and templates, Inkhaven can invent word-shapes for you that obey your rules. This is the fastest way to discover the **sound** of your language and to find candidate words to give meanings to later.

```
inkhaven language generate-word Eldar --role root --count 10
```

This prints ten random roots that fit your templates — perhaps **taki**, **mun**, **sala**, **kira**. None of them mean anything yet; they are raw material. When one sounds right, you will turn it into a real word in Part III. `--count` takes 1 to 100 000; if not a single candidate passes your constraints the command says so (`0 of 10 candidates passed the constraints — …`) and points you at `language doctor`. Add `--json` to get the batch as a list for a script.

> **LISTEN TO WHAT COMES OUT**
>
> Generating a batch of words is the best way to **audition** your phonology. If the results feel too harsh, too samey, or hard to say, adjust your inventory or template weights and generate again. This loop — tweak, generate, listen — is the heart of designing a sound system. It costs nothing and uses no AI.

Inspecting a single word

To see how Inkhaven breaks a word into syllables, use the **syllabify** inspector:

```
inkhaven language syllabify Eldar --word takina
```

It prints the syllable division — `ta.ki.na` — showing the onsets, nuclei, and codas it found. This is useful for checking that your templates do what you expect, and it is the same machinery that later figures out where stress falls.

TERM **Word root**

The core form of a word, carrying its basic meaning, before any endings are added. **build** is a root; **builder** and **rebuilding** are formed from it. In Part IV you will add endings to roots; here you are just shaping the roots themselves.

WHAT YOU LEARNED

- A **syllable** is a beat of speech: an optional **onset**, a vowel **nucleus**, and an optional **coda**.
- **Templates** like `C V (C)` describe allowed syllable shapes; `weight` controls how often each appears.
- `generate-word` invents word-shapes that obey your rules — raw material for real words.
- `syllabify --word` shows how a word divides, for checking your templates.
- Designing phonology is a loop: tweak, generate, listen, repeat.

CHAPTER 6

6

Phonotactics: the rules of combination

Templates say what shape a syllable has. **Phonotactics** says which actual combinations of sounds are allowed inside that shape. Every language forbids certain sequences: English allows **spr-** at the start of a word (**spring**) but never **tlp-**; Japanese allows almost no consonant clusters at all. These rules are a big part of why languages sound the way they do.

> **TERM Phonotactics**
>
> The rules governing which sequences of sounds are permitted in a language — which consonants may cluster, which sounds may end a syllable, and so on. They are constraints layered on top of your syllable templates.

Constraints

You express these rules as a list of **constraints** in the phonology block, under a `constraints` field. Each constraint has a `kind`, and some take extra details. The most common kinds:

> **TERM Constraint**
>
> A single phonotactic rule that rejects certain words. Constraints are checked whenever Inkhaven generates or validates a word; a word that breaks any constraint is not a legal word of the language.

Limit cluster length

A **cluster** is two or more consonants in a row. To cap how long clusters may be, use `max_cluster_size`:

```
{ kind: "max_cluster_size", value: 2 }
```

This forbids three-consonant pileups while still allowing pairs like **tr** or **sk**. Setting the value to `1` forbids all clusters, giving a smooth, open-syllabled language.

Forbid doubled consonants

A **geminate** is the same consonant written twice, like the **tt** in Italian **gatto**. If you do not want them:

```
{ kind: "no_geminate" }
```

> **TERM Geminate**
>
> A doubled or lengthened consonant, as in Italian **notte** (“night”). Some languages use them meaningfully; many forbid them. `no_geminate` rules them out.

Restrict what may end a syllable

Often a language allows many consonants at the start of a syllable but only a few at the end. Use `forbid_in_coda` (or its mirror `forbid_in_onset`) with a class name. This is **syllable-aware** — Inkhaven works out the coda for you:

```
{ kind: "forbid_in_coda", classes: ["Stop"] }
```

(This assumes you have declared a class named `Stop`; you can name a class for any group of sounds you like, just as you named `C` and `V`.) With this rule a word like **tak** — which ends in the stop **k** — is rejected, while **tan**, ending in the nasal **n**, is allowed: the stop may open a syllable but never close one.

Follow the sonority rule

Real consonant clusters are not random: they tend to rise in **sonority** (roughly, loudness or openness) toward the vowel and fall away after it. That is why **pla-** feels natural and **lpa-** does not. Turn on this natural tendency with:

```
{ kind: "sonority_sequencing" }
```

With it on, an onset **pl-** passes — the stop **p** (low sonority) rises to the liquid **l** (higher) and on to the vowel — but **lp-** is rejected, because it would **fall** from **l** down to **p** before the vowel. The rule lets your generated words sound pronounceable without you listing every legal cluster by hand.

> TERM **Sonority**
>
> How open and resonant a sound is. Vowels are most sonorous; then glides (w, y), liquids (l, r), nasals (m, n), and finally the quiet stops (p, t, k). The **sonority sequencing principle** says syllables tend to rise in sonority up to the vowel and fall afterward — a strong ingredient of a natural sound.

Putting it together

A complete phonotactics section for Eldar might read:

```
constraints: [
  { kind: "max_cluster_size", value: 2 }
  { kind: "no_geminate" }
  { kind: "sonority_sequencing" }
]
```

Add it to the phonology block, reindex, and generate a fresh batch of words with `generate-word`. You will notice the output is now cleaner — no triple clusters, no doubled letters, and clusters that "lean" the natural way.

CONSTRAINTS ARE A SOUND DESIGNER'S DIALS

There is no single correct set. Tight constraints (short clusters, restricted codas) give a smooth, melodic language; loose ones give a rugged, clattering one. Generate words after each change and let your ear decide. The constraints are also the gatekeeper later: when you coin or import words, anything that breaks them is flagged automatically (Chapter 10).

WHAT YOU LEARNED

- **Phonotactics** are the rules for which sound sequences are allowed.
- You write them as a list of `constraints`, each with a `kind`.
- Useful kinds: `max_cluster_size`, `no_geminate`, `forbid_in_coda` / `forbid_in_onset`, and `sonority_sequencing`.
- Tighter constraints sound smoother; looser ones sound rougher — tune by ear using `generate-word`.

CHAPTER 7

7

Allophony: sounds that shift

Here is a subtle, lovely fact about real languages: a single phoneme is often pronounced differently depending on its neighbours, without speakers even noticing. The English /t/ in **top** is a sharp, puffed sound; the /t/ in **stop** is softer; the /t/ in **butter** (in American speech) is a quick tap. Same phoneme, three pronunciations. These context-dependent variants are **allophones**, and the phenomenon is **allophony**. Adding a little of it is one of the easiest ways to make a conlang feel alive.

TERM **Allophone and allophony**

An **allophone** is one of the several ways a single phoneme is actually pronounced, chosen automatically by its surroundings. **Allophony** is the system of such variations. Speakers treat the allophones as "the same sound", but the surface pronunciation differs. Example: a /k/ that becomes a **ch**-sound before /i/.

Underlying and surface forms

To talk about allophony we need two ideas. The **underlying form** of a word is how it is built from phonemes — its blueprint. The **surface form** is how it is actually pronounced after the allophony rules have applied. A rule turns one into the other.

TERM **Underlying form and surface form**

The **underlying form** is the abstract phoneme sequence a word is made of. The **surface form** is the real pronunciation after allophony rules run. If /k/ becomes **ch** before /i/, then the word with underlying /kira/ has the surface form **chira** — but it is still "the /k/ word" underneath.

Writing a rule

Allophony rules use a compact notation borrowed from linguistics, sometimes called **SPE notation**. A rule has four parts:

```
WHAT  >  BECOMES  /  LEFT _ RIGHT
```

Read it as: "**WHAT** becomes **BECOMES** when it sits between **LEFT** and **RIGHT**". The underscore `_` marks the spot where the changing sound sits. So:

```
k > tʃ / _ i
```

means "/k/ becomes /tʃ/ (the **ch** sound) when an /i/ follows it" — the left side of the context is empty, the right side is /i/. A few more building blocks:

1. A `#` stands for a **word boundary** — the start or end of a word. `p > f / _ #` means "/p/ becomes /f/ at the end of a word".
2. An empty side means "no condition there". `_ i` cares only about what follows.
3. A context token may be a **class name** (like `V` for any vowel) rather than a single sound: `k > h / V _ V` means "/k/ becomes /h/ between two vowels".
4. The target and the result may each be a **sequence** of sounds, written with spaces between them: `a i > e / _ #` turns a final /ai/ into /e/, and `t > t s / _ i` splits one sound into two. Written without the space, `ai` still works when every reading is the same two phonemes; when it could be read two ways, `language doctor` tells you and asks for the spaces.

> TERM **SPE notation**
>
> A standard shorthand for sound rules, named after a famous 1968 linguistics book (**The Sound Pattern of English**). The form is `target > result / left _ right`, where `_` is the target's position, `#` is a word edge, a class name matches any sound in that class, and a space-separated run of sounds on either side of `>` is a sequence. Inkhaven uses it for both allophony and, later, historical sound change.

Adding allophony to Eldar

You declare rules under an `allophony` field in the phonology block. Each rule has an optional `name` (for your own reference) and the `rule` itself:

```
allophony: [
  { name: "palatalization", rule: "k > tʃ / _ i" }
  { name: "lenition",       rule: "t > s / _ i" }
]
```

Add these, reindex, and check the effect with the **ipa** inspector, which shows a word's **surface** form:

```
inkhaven language ipa Eldar --word kira
```

The underlying **kira** comes back pronounced **tʃira** — the /k/ has palatalized before the /i/. From now on, every word Inkhaven generates or inflects will have these rules applied automatically, so the variation is consistent everywhere.

ALLOPHONY RULES APPLY IN ORDER

The rules run top to bottom, and an earlier rule can feed a later one. Order them thoughtfully, and use the `ipa` inspector to confirm a tricky word comes out the way you intend. A little allophony goes a long way — two or three well-chosen rules are plenty for a natural feel.

WHY THIS MATTERS LATER

Allophony is not just decoration. The same rules fire when you add grammatical endings to words (Part IV) — so a suffix can trigger a sound change at the join, exactly as in real languages. And the **historical** sound changes of Part V use this very same notation. Learn it once; use it three times.

WHAT YOU LEARNED

- An **allophone** is a context-dependent pronunciation of a phoneme; the system is **allophony**.
- The **underlying form** is the blueprint; the **surface form** is the real pronunciation after rules apply.
- Rules use **SPE notation**: `target > result / left _ right`, with `_` the target, `#` a word edge, and class names matching families.
- Declare them under `allophony`; check results with `ipa --word`.
- The same notation reappears for grammar boundaries and for historical change.

CHAPTER 8

8

Stress, spelling, and tone

Three finishing touches complete your sound system: where the emphasis falls in a word (**stress**), how you write the language with ordinary letters (**romanization**), and — for some languages — meaningful pitch (**tone**). The first two you will almost certainly want; the third is optional.

Stress

In most languages, one syllable of a word is said a little louder or longer than the others. That is **stress**. English stress is unpredictable (**REcord** the noun versus

reCORD the verb), but many languages place it by a simple rule — always the first syllable, or always the next-to-last.

> TERM **Stress**
>
> The relative emphasis given to one syllable of a word, by loudness, length, or pitch. A **fixed** stress rule places it predictably; common rules are **initial** (first syllable), **final** (last), **penultimate** (second-to-last), and **antepenultimate** (third-to-last).

You set a stress rule with the `stress` field in the phonology block. The value is one of `initial`, `final`, `penultimate`, `antepenultimate`, or `latin`. That last one is **weight-sensitive**: it stresses the second-to-last syllable when that syllable is **heavy** (it has a long vowel or ends in a consonant) and otherwise moves the stress one syllable earlier — the rule classical Latin used. (See **syllable weight** in the glossary.) Eldar will simply stress the second-to-last syllable:

```
stress: "penultimate"
```

Check it with the **stress** inspector, which marks the stressed syllable:

```
inkhaven language stress Eldar --word takina
```

It returns something like `ta.'ki.na` — the mark `'` sits before the stressed syllable.

With stress known, you can **scan** a line of verse — mark every syllable strong or weak and name its metre:

```
inkhaven language scan Eldar --text "takina ómor takina"
```

Each word is syllabified and stressed (by the rule above, by a per-word `stress` field in the lexicon for irregular words, or by an acute mark you type over a vowel), and the line is matched against the metrical feet — iamb, trochee, dactyl, anapest — to report its metre, e.g. **trochaic trimeter**. A monosyllable is left flexible, free to fall on a strong or weak beat as the metre needs.

Romanization: writing it down

Your phonemes may include symbols like /ʃ/ that you do not want to type constantly. **Romanization** is the system for writing the language with ordinary

Latin letters — the **romanize** spellings you already gave each phoneme are the simple version of this. For anything trickier, you can define a named **romanization scheme**.

> **TERM Romanization**
>
> A way of writing a language's sounds using the Latin alphabet (the letters a–z). The simplest romanization gives each phoneme a spelling; a fuller scheme can spell one sound several ways depending on context, the way Italian writes /k/ as **c** before **a** but **ch** before **e**.

A scheme maps phonemes to letters, and can include **contextual** rules — a spelling that depends on the surrounding sounds. Suppose you want /k/ and /s/ both written `c`, with `c` read as /s/ before a front vowel (as in Italian or English **city**):

```
romanizations: [
  { name: "default"
    mappings: [ { ipa: "k", roman: "c" }, { ipa: "s", roman:
"c" } ]
    contextual: [ { roman: "c", ipa: "s", before: "FrontV" } ] }
]
default_romanization: "default"
```

You can then convert text either way with the **romanize** inspector:

```
inkhaven language romanize Eldar --text cace
inkhaven language romanize Eldar --text /kase/ --reverse
```

The first reads spelled text into sounds; the second (`--reverse`) writes sounds back out as spelling — note the slashes around `/kase/`, the usual way of marking that the input is a string of sounds (IPA) rather than ordinary spelling. For most languages the simple per-phoneme `romanize` fields are all you need, and you can skip schemes entirely.

Tone (optional)

Some languages — Mandarin, Yoruba, Thai — use **pitch** to distinguish words: the same syllable said with a rising versus a falling pitch means different things. That is **tone**. If your language has no tone, skip this section; most conlangs do.

TERM **Tone**

The use of pitch (the rise and fall of the voice) to distinguish word meanings. In a **tonal** language, **ma** on a high pitch and **ma** on a falling pitch are different words. Tones can also interact: a **tone sandhi** rule changes one tone next to another.

To add tone, declare a `tone` block with the tones you use and any **sandhi** rules (written in the same SPE notation as allophony):

```
tone: {
  kind: "contour"
  tones: ["1", "2", "3", "4"]
  sandhi: [ { rule: "3 > 2 / _ 3" } ]
}
```

The `kind` says what sort of tones these are. A **contour** tone glides during the syllable (rising, falling, dipping) — the Mandarin style, where the numbers name shapes; the alternative, `register`, is for tones held level at a fixed pitch (high, mid, low). This example gives four contour tones (numbered) and one **sandhi** rule — "a third tone becomes a second tone before another third tone", the famous Mandarin pattern. Apply it to a sequence of tone numbers with:

```
inkhaven language tone Eldar --tones "3 3 3"
# → 2 2 3   (each 3 before another 3 lowers to 2; the last one is
unchanged)
```

YOUR PHONOLOGY IS DONE

With sounds, syllable shapes, phonotactics, allophony, stress, and (optionally) romanization and tone, your sound system is complete. Everything from here on — words, grammar, history, writing — is spoken with these sounds and obeys these rules automatically. This is the foundation the whole language stands on.

WHAT YOU LEARNED

- **Stress** is the emphasised syllable; set a fixed rule with the `stress` field (`initial`, `final`, `penultimate`, …).
- **Romanization** writes the language in Latin letters; per-phoneme `romanize` spellings suffice for most languages, with `romanizations` schemes for harder cases.
- **Tone** (optional) uses pitch to distinguish words; declare a `tone` block with tones and `sandhi` rules.
- Inspectors `stress`, `romanize`, and `tone` let you check each at a word.

PART III

Words

CHAPTER 9

9

Building a vocabulary

Your language can now make word-shapes; it is time to give them meanings. The collection of words and their meanings is the **lexicon**, and each entry pairs a word with what it means, what part of speech it is, and any extra notes you care to keep. This chapter is about adding words by hand; the next adds two more powerful ways to grow the lexicon.

Adding a word

The basic command is **add-word**. It needs the language, the word itself, its **part of speech**, and its meaning (the **translation** into your everyday language):

```
inkhaven language add-word Eldar makil --type noun --translation
sword
```

This stores **makil** in Eldar's **Dictionary** chapter as a noun meaning "sword". No reindex is needed — commands that write changes for you handle that themselves. Add a few more:

```
inkhaven language add-word Eldar kira --type noun --translation
bird
inkhaven language add-word Eldar nami --type verb --translation
see
inkhaven language add-word Eldar mira --type adjective --
translation bright
```

> TERM **Part of speech**
>
> The grammatical category of a word — what **kind** of word it is. The common ones are **noun** (a thing: sword, bird), **verb** (an action: see, run), **adjective** (a description: bright, cold), and **adverb**. The category matters later, when grammar rules apply only to words of a certain kind.

> TERM **Translation (gloss)**
>
> The meaning of a word given in your working language — for **makil**, the English "sword". A short meaning like this is also called a **gloss**. It is how you and Inkhaven know what the word means.

Recording more than the basics

A word often carries more than a bare meaning. Is it formal or vulgar? What subject does it belong to — warfare, cooking, religion? Does it belong to an older era of the language? You can record all of this, and search by it later. A dictionary entry, stored as a small HJSON block, can hold these extra fields:

```
{ word: "makil", type: "noun", translation: "sword",
  register: "formal", domain: ["weapon"], era: "third_age" }
```

TERM **Register**

The level of formality or social setting a word belongs to — **formal**, **neutral**, **vulgar**, **sacred**, **archaic**. The same idea in your everyday language separates "purchase" (formal) from "buy" (neutral). Marking register lets a language have words that fit a king's court and others that fit a tavern.

TERM **Domain**

The subject area a word belongs to — **weapon**, **kinship**, **weather**, **magic**. Domains help you build vocabulary by theme and find related words quickly.

Finding words again

As the lexicon grows you will want to search it. The **query** command filters by any of the rich fields — part of speech, register, domain, era, or a substring of the word or its meaning:

```
inkhaven language query Eldar --pos noun
inkhaven language query Eldar --domain weapon
inkhaven language query Eldar --text bright
```

Each prints the matching entries. With `--json` you get machine-readable output, handy for scripts.

Importing many words at once

If you already have a word list — perhaps in a spreadsheet — you can bring it in all at once. Export it as a CSV file (a simple table of word, part of speech, translation, …) and import:

```
inkhaven language add-word Eldar --import words.csv
```

Every row becomes an entry. This is the fast path when you are moving an existing list into Inkhaven: the whole file is written as one batch and the search index is refreshed once at the end, so a few hundred rows land in seconds, already in alphabet order.

REMOVING A WORD

Made a mistake, or retired a word? `inkhaven language remove-word Eldar makil` deletes it from the dictionary. Like `add-word`, it finds the entry by name and needs no reindex.

Using your words while you write

The reason to build a lexicon is, of course, to **use** it. When you write prose in Inkhaven's editor, you can drop an invented word straight in: type a colon, your language's name or code, and another colon — `:eldar:` — and a picker appears listing your words; choose one and it is inserted. Inkhaven also lights up your invented words where they appear in a manuscript, and can translate whole paragraphs into or out of the language. These editor features are beyond the scope of this command-focused book, but it is worth knowing the lexicon you are building is meant to be lived in.

WHAT YOU LEARNED

- The **lexicon** is your vocabulary; each entry pairs a word with a meaning.
- `add-word <lang> <word> --type <pos> --translation <meaning>` adds one.
- Record **part of speech**, **register**, **domain**, and **era**; search them with `query`.
- Import a CSV with `add-word --import`; remove with `remove-word`.
- In the editor, `:lang:` inserts a word from the lexicon.

CHAPTER 10

10

A healthy lexicon

A growing vocabulary can drift into trouble: two words that sound identical, two words that mean the same thing, a word that quietly breaks your own sound rules. This chapter shows how Inkhaven keeps the lexicon honest — and how it can invent whole batches of new words for you, on a theme, without ever creating a duplicate.

Auditing for problems

The **audit** command is your lexicon's health check. It reads every entry and reports three kinds of trouble:

```
inkhaven language audit Eldar
```

1. **Phonotactic violations** — a word whose sounds break the constraints you set in Chapter 6 (perhaps imported from an old list before you tightened the rules).
2. **Homophones** — two different words that come out pronounced the same after allophony. Sometimes you want these; often they are accidents.
3. **Duplicate meanings** — two words with the same translation, which may be an oversight.

> TERM **Homophone**
>
> A word that sounds the same as another but means something different — like English **to**, **too**, and **two**. A few are natural; many by accident usually signal a mistake. The audit catches them even when two words only **become** identical after your allophony rules apply.

The audit only reports; it changes nothing. Use it whenever the lexicon grows, or before you publish your dictionary, to catch surprises.

Generating words with AI

Building a few hundred words by hand is slow. Inkhaven can generate a batch for you on a theme — but with an important guarantee about **where the words come from**. The **forms** (the actual sound-shapes) are made by Inkhaven's own generator, so they always obey your phonotactics. The AI only supplies the **meanings**. This split — forms from the engine, meanings from the AI — is what keeps generated words consistent with your language.

```
inkhaven language generate-lexicon Eldar --topic seafaring --
count 20
```

This proposes twenty seafaring words, each a legal Eldar form with an AI-chosen meaning. By default it only **prints** the proposals; add `--yes` to actually add them to the dictionary.

> TERM **The dedup gate**
>
> A set of automatic checks every proposed word passes before it is accepted, so generation never creates a duplicate. It rejects a word that breaks the phonotactics, that sounds identical to an existing word, that repeats a meaning already in the lexicon, or — with the optional `--semantic` check — that is too close in meaning to an existing word (a near-synonym). "Forms obey the language; meanings come from the AI; nothing duplicates."

Catching near-synonyms

Two words can have different translations yet mean almost the same thing — "stone" and "rock", say. To reject such near-synonyms too, add `--semantic`:

```
inkhaven language generate-lexicon Eldar --topic seafaring --
count 20 --semantic --yes
```

This compares the meaning of each proposal against your existing words using a language-model technique and drops the ones that are too close. It is slower but keeps a tight, non-redundant vocabulary.

> AI HERE IS ADVISORY, LIKE EVERYWHERE
>
> Without `--yes`, generation shows you the proposals (and the reasons any were rejected) and changes nothing. You stay in control: review the list, then commit. Meanings come back in your working language. If you would rather not use AI at all, build the lexicon by hand with `add-word` — the audit and the phonotactic checks work just the same.

Finding undefined words in your writing

If you have been writing prose that **uses** the language, Inkhaven can scan your manuscript for words that look like your language but are not yet in the dictionary — names and terms you coined on the fly and never recorded:

```
inkhaven language scan-manuscript Eldar
```

It lists candidate undefined words, so you can add the ones worth keeping. This closes the loop between writing and the lexicon.

The Oracle watches as you write

`scan-manuscript` finds the **well-formed** words you have not defined yet. Its mirror image runs automatically. Whenever you save a paragraph, the **Oracle** looks at the conlang words in it — words that segment fully into one of your languages' inventories — and quietly flags any that break that language's own phonotactics. The finding appears in the **Output pane** (`Ctrl+B` then `Tab`), marked `ϟ`:

```
ϟ [Eldar] `ktal` breaks phonotactics — contains an illegal onset
cluster
```

It is the same judgement as `inkhaven language check`, pointed at your prose instead of a single word — a proofreader for the shape of your invented words. It is purely advisory: it never changes a syllable of your writing, and it only speaks up in paragraphs that already contain a word of the language, so prose in your working language is left alone. A well-formed word you simply have not defined is `scan-manuscript`'s business, not the Oracle's, so it stays silent on those.

The same guardian runs across your **whole** manuscript in the review pass (`Ctrl+B J`), alongside the other readers — so you can sweep every chapter for malformed conlang words at once, not only the paragraph in front of you.

> **TURNING IT OFF**
>
> The guardian is on by default. To silence it, set `oracle.on_save: false` in `inkhaven.hjson` (or `oracle.enabled: false` to disable the Oracle entirely). Projects with no languages never trigger it.

What is your lexicon still missing?

A young lexicon always has holes. Inkhaven can tell you **which** basic concepts you have not coined yet, by comparing your dictionary against a reference list of concepts:

```
inkhaven language gaps Eldar
```

By default it checks against the **Swadesh-100** — the classic list of the hundred most fundamental words every language has (I, water, sun, eat, big…), here translated

into your working language. It reports your coverage and lists what is missing, most-core words first:

> **TERM Swadesh list**
>
> A short list of universal, culture-independent concepts (body parts, basic verbs, natural features, pronouns) compiled by the linguist Morris Swadesh. Because every language has words for them, it is the standard yardstick for "have I covered the basics?" and for comparing related languages.

The missing list is shaped to hand straight back to the generator — coin exactly the gaps with `generate-lexicon`. You can also point `--scope` at your own concept list (an HJSON file of a topic like seafaring or cookery) to check coverage of any domain you care about.

> **WHAT YOU LEARNED**
>
> - `audit` reports phonotactic violations, **homophones**, and duplicate meanings — it only reports, never changes.
> - `generate-lexicon --topic … --count …` invents themed words: **forms** from the engine (always legal), **meanings** from the AI.
> - The **dedup gate** blocks illegal, homophone, and duplicate-meaning words; `--semantic` also blocks near-synonyms.
> - Nothing is added without `--yes`; AI is optional.
> - `scan-manuscript` finds language-like words in your prose that are not yet defined; the **Oracle** watches every save and flags conlang words that break your phonotactics (`⌥` in the Output pane) — advisory, never editing.
> - `gaps` compares your lexicon against the **Swadesh-100** (or your own concept list) and reports what is still missing.

PART IV

Grammar

CHAPTER 11

11

Morphology: words that change

Words are not frozen. **Cat** becomes **cats**; **walk** becomes **walked**. A word changes its shape to express grammar — number, tense, who is doing what. **Morphology** is the study of those changes, and it is the first half of grammar. This chapter shows how to build the pieces words are made of and how to assemble them.

> TERM **Morphology**
>
> The part of grammar dealing with the internal structure of words — how they are built from smaller meaningful pieces, and how they change shape to express grammar. (The other half of grammar, **syntax**, deals with word order, covered in Chapter 13.)

Morphemes and affixes

The smallest meaningful pieces of a word are **morphemes**. **Cats** has two: **cat** (the thing) and **-s** (meaning "more than one"). A morpheme like **-s** that attaches to a word is an **affix**. Affixes that go on the front are **prefixes** (**re-** in **rebuild**); on the end, **suffixes** (**-s**, **-ed**); inside, **infixes**.

> TERM **Morpheme**
>
> The smallest unit of a word that carries meaning. **Unhappiness** has three: **un-** (not), **happy**, and **-ness** (the quality of). A **root** is the central morpheme; **affixes** attach to it.

> TERM **Affix**
>
> A morpheme attached to a root to modify its meaning or grammar. A **prefix** attaches to the front (**re-do**), a **suffix** to the end (**cat-s**), an **infix** in the middle. Inkhaven supports prefixes, suffixes, infixes, and circumfixes — and beyond gluing pieces on, the stem-changing processes covered later in this chapter (ablaut and reduplication).

Declaring your affixes

You list a language's affixes in a **morphology block** in the **Grammar** chapter. Each **morpheme** gives an `id` (a short name you choose), a `gloss` (a label for what it means, conventionally in small capitals), a `form` (the actual sounds it adds), and a `position` (`"prefix"` or `"suffix"`):

```
{
  morphemes: [
    { id: "pl",  gloss: "PL",  form: "i",  position: "suffix",
category: "number", value: "plural" }
    { id: "dat", gloss: "DAT", form: "ti", position: "suffix",
category: "case",   value: "dative" }
    { id: "def", gloss: "DEF", form: "na", position: "prefix",
category: "definiteness" }
  ]
}
```

Here `pl` is a plural suffix **-i**, `dat` a dative suffix **-ti**, and `def` a definite prefix **na-**. The glosses `PL`, `DAT`, `DEF` are the standard linguistic abbreviations for "plural", "dative", and "definite".

> TERM **Category and value**
>
> Two optional tags that say what **kind** of grammar a morpheme expresses. The **category** is the grammatical dimension — `number`, `case`, `tense`, `gender` — and the **value** is the specific point on it — `plural`, `dative`, `past`. They are not used to build words, but the reference grammar (Chapter 20) reads them to **organise** its affix list by category, which makes a grammar far easier to read. Tag your affixes as you go.

Ordering stacked affixes

When two or more affixes pile onto the same side of a root, which comes first? In many languages the order is fixed — a case ending might always sit closer to the root than a number ending. You control this with an optional `precedence` number on each morpheme:

```
{ id: "pl",  gloss: "PL",  form: "i",  position: "suffix",
precedence: 2 }
{ id: "dat", gloss: "DAT", form: "ti", position: "suffix",
precedence: 1 }
```

TERM **Affix ordering (precedence)**

A number saying how close an affix sits to the root when several stack: `0` (the default) means any position — the order you listed them in is kept; `1` means immediately next to the root; `2` the next slot out; and so on. A lower non-zero number is closer to the root. Above, the case suffix (precedence 1) always hugs the root and the number suffix (precedence 2) sits outside it, no matter which order a paradigm lists them.

TERM **Gloss (grammatical)**

A short label for a grammatical piece, written in small capitals by convention: PL (plural), SG (singular), DAT (dative case), PST (past tense), and so on. Glosses let you write what a word **means** grammatically, piece by piece.

TERM **Inflection**

Changing a word's form to express grammar **without** making it a new word — **cat** / **cats**, **walk** / **walked**. The set of all inflected forms of a word is its **paradigm**. (Contrast with **derivation**, Chapter 12, which makes a genuinely new word.)

Paradigms: the forms of a word

A **paradigm** is the full table of a word's inflected forms — for a noun, perhaps singular and plural, in each grammatical case. You describe one as a list of **cells**, each saying which features it expresses (number, case) and which morphemes build it:

```
paradigms: [ { name: "noun", cells: [
  { features: { number: "sg", case: "nom" }, morphemes: [] }
  { features: { number: "pl", case: "dat" }, morphemes: ["dat",
"pl"] }
] } ]
```

Add this beside `morphemes` in the Grammar chapter. The first cell is the bare root (no morphemes); the second stacks the dative and plural suffixes.

> TERM **Paradigm**
>
> The organised set of all inflected forms of a word — for example a noun in singular and plural across every case. Each form is one **cell** of the paradigm, defined by its grammatical **features** and the morphemes that build it.

Generating the forms

Now Inkhaven can build the whole paradigm for any root, applying the morphemes **and** your allophony rules at the joins:

```
inkhaven language paradigm Eldar --root kata --template noun --
gloss stone
```

For the root **kata** ("stone") this prints each cell's surface form and its gloss — for example **katati** for "stone-DAT". And here is the elegant part: if your allophony rule says /t/ softens to /s/ before /i/, then **kata** plus the dative **-ti** comes out as **katasi**, because the rule fires at the boundary, exactly as it would in a real language. The sound machinery from Part II is working for you automatically.

Interlinear glossing

When you write a sentence in the language, you will often want to show, word by word, what each piece means — a **interlinear gloss**, the two-line format linguists use. A dictionary entry can declare which paradigm it inflects by (`paradigm: "noun"`), and then Inkhaven can gloss running text:

```
inkhaven language gloss Eldar --text "kata katai katat"
```

It prints each word above its Leipzig-style gloss — `kata` → `stone`, `katat` → `stone-DAT` — and it recognises inflected **and** allophony-altered forms, because it works out each entry's paradigm forward and matches what it finds.

TERM **Interlinear gloss (Leipzig glossing)**

A way of showing a sentence in an unfamiliar language with a word-by-word breakdown lined up underneath, using grammatical glosses (PL, DAT, …). The name comes from the widely used **Leipzig Glossing Rules**. It is how grammars and textbooks make a foreign sentence transparent.

Beyond prefixes and suffixes

Not every language marks grammar by gluing pieces to the ends of words. Inkhaven supports the other common strategies too; you choose them on a morpheme with `position` or `process`.

TERM **Infix**

An affix inserted **inside** the root rather than at an edge. Tagalog forms an actor from **sulat** (“write”) as **s-um-ulat** — the **-um-** sits after the first consonant. Declare `position: "infix"`; the `anchor` is `before_first_vowel` (the default) or `after_first_vowel`.

TERM **Circumfix**

A single affix in two pieces that wrap around the root — German makes a past participle with **ge-…-t** (**ge-sag-t**). Declare `position: "circumfix"` and write the `form` with a `_` marking where the stem goes: `ge_t` → **ge** + stem 1. **t**.

TERM **Ablaut**

Marking grammar by **changing a sound inside** the root instead of adding anything — English **sing** / **sang** / **sung**. Declare `process: "ablaut"` and give the change as an SPE `rules` list (Chapter 7), e.g. `i > a`. The vowel swaps in place.

> **TERM Reduplication**
>
> Marking grammar by **repeating** part (or all) of the root — Malay **buku** "book" → **buku-buku** "books". Declare `process: "reduplication"` with a `reduplicate` mode: `full` (the whole stem doubled), `initial_cv` (the first consonant + vowel copied to the front), `initial_syllable`, or `final_syllable`.

All four are written as ordinary morphemes and used in paradigm cells exactly like prefixes and suffixes — and allophony still applies across the new seams. Here is each one as a declaration and the form it produces, so you can see the process at work:

- **Infix** — `{ gloss: "AG", form: "um", position: "infix" }` turns the root **tanik** into **t-um-anik**.
- **Circumfix** — `{ gloss: "PTCP", position: "circumfix", form: "ge_t" }` wraps the root **sag** into **ge-sag-t** (the `_` is where the stem lands).
- **Ablaut** — `{ gloss: "PAST", process: "ablaut", rules: ["i > a"] }` rewrites the vowel of **kit** in place, giving **kat**.
- **Reduplication** — `{ gloss: "PL", process: "reduplication", reduplicate: "full" }` doubles the stem, so **kata** becomes **kata-kata**; with `reduplicate: "initial_cv"` only the first consonant-plus-vowel copies to the front, so **tanik** becomes **ta-tanik**.

Agreement

Often one word must echo the grammar of another: an adjective takes its noun's number and case, a verb takes its subject's person and number. This matching is called **agreement** (or **concord**), and it is what makes "these tall trees" work where "this tall tree" also does.

TERM Agreement (concord)

The requirement that a **dependent** word copy certain grammatical features from the **head** it modifies — an adjective agreeing with its noun, a verb with its subject. The shared features (number, case, gender, person) must match.

You declare agreement as rules in the morphology block: which part of speech agrees with which, on which features, and through which paradigm:

```
agreement: [
  { dependent: "adjective", head: "noun", features: ["number",
"case"], paradigm: "adj" }
]
```

Then Inkhaven can inflect a dependent to agree with a given head. Suppose a noun is plural; to get the matching form of the adjective **mira** (“bright”):

```
inkhaven language agree Eldar --word mira --pos adjective \
    --gloss bright --features "number=pl"
```

It finds the agreement rule for adjectives, picks the `adj` paradigm cell that matches `number=pl`, and prints the agreeing form — **mirai**, glossed `bright-PL`. The `--features` you pass are the head’s; only the ones the rule lists as agreement features are copied.

WHAT YOU LEARNED

- **Morphology** is how words change shape; the pieces are **morphemes**, and attached ones are **affixes** (prefix / suffix / **infix** / **circumfix**).
- Beyond gluing affixes, a morpheme can be a **process**: **ablaut** (an internal sound change) or **reduplication** (repeating part of the root).
- **Inflection** changes a word’s form; the full set is a **paradigm**, described as `cells` of features + morphemes.
- `paradigm` builds the forms (allophony applies at every seam); `gloss` shows a sentence interlinear, word by word.
- **Agreement** makes a dependent copy a head’s features; `agree` inflects it to match.

CHAPTER 12

12

Building new words

Inflection (last chapter) changes a word's form without changing what it fundamentally is — **stone** and **stones** are the same word. **Derivation** is different: it makes a genuinely **new** word from an old one. **Build** becomes **builder** (a person who builds); **king** becomes **kingdom** (the realm of a king). This is one of the most productive ways a real lexicon grows, and your language can do it too.

> TERM **Derivation**
>
> Forming a new word (a new **lexeme**, with its own dictionary entry) from an existing one, usually by adding an affix and often changing its part of speech. **teach** (verb) → **teacher** (noun); **happy** (adjective) → **happiness** (noun). Contrast with **inflection**, which only changes a word's form, not its identity.

Declaring a derivation rule

You add derivation rules to the morphology block in the **Grammar** chapter, under a `derivations` field. Each rule names the affix it adds, where it goes, which part of speech it applies to (`from_pos`), what part of speech it produces (`to_pos`), and a template for the new meaning:

```
derivations: [
  { name: "agent", form: "ron", position: "suffix",
    from_pos: "verb", to_pos: "noun", gloss_template: "one who {}
s" }
]
```

This is an **agent noun** rule: take a verb, add **-ron**, and you get a noun meaning "one who does the verb". The `{}` in the template is filled with the root's meaning — so a verb meaning "build" yields a noun glossed "one who builds".

> TERM **Agent noun**
>
> A noun naming the doer of an action, derived from a verb — **teach** → **teacher**, **bake** → **baker**. The "**-er**" of English is an agent suffix. Agent nouns are one of the most common derivations across the world's languages.

Coining derived words

With a rule in place, ask Inkhaven to apply it to a root:

```
inkhaven language derive Eldar --root kata --gloss build --pos
verb
```

Inkhaven runs every derivation rule whose `from_pos` matches "verb", applies the affix (with allophony at the join), and prints the proposed new words — their form, meaning, and part of speech. For **kata** "build" it might propose **kataron**, "one who builds", a noun.

By default this only **proposes**. To actually add the derived words to your dictionary — recording where each came from, as a small etymology — add `--yes`:

```
inkhaven language derive Eldar --root kata --gloss build --pos
verb --yes
```

TERM **Etymology**

The origin and history of a word — what it was derived or descended from. When Inkhaven coins a derived word for you, it records the etymology in the entry ("derived from **kata** via the agent rule"), so your dictionary remembers how each word came to be.

DERIVATION VERSUS GENERATION

Two ways to grow vocabulary, for two purposes. **Generation** (Chapter 10) invents brand-new roots on a theme — good for filling out basic vocabulary. **Derivation** grows words **from words you already have**, the way real languages spin families of related terms from a single root. Use generation for breadth, derivation for depth and internal consistency.

WHAT YOU LEARNED

- **Derivation** makes a new word from an existing one, often changing its part of speech (**build** → **builder**).
- Declare rules under `derivations`: `form`, `position`, `from_pos`, `to_pos`, and a `gloss_template` with `{}` for the root's meaning.
- `derive --root … --gloss … --pos …` proposes derived words; `--yes` adds them with their **etymology** recorded.
- Use generation for new roots, derivation to grow word families from existing roots.

CHAPTER 13

13

Typology: the shape of your grammar

Beyond individual words lies the architecture of a language: does the verb come before or after the object? Does it mark the subject of a sentence specially? Does it have grammatical gender? These big design choices are the subject of **typology**, and making them is how you give your grammar a coherent character. Inkhaven gives you a questionnaire of the major choices, drawn from how real languages actually vary.

> TERM **Typology**
>
> The classification of languages by their structural features — word order, how they mark who-does-what, whether they have case or gender, and so on. **Typology** is, in effect, the menu of high-level design decisions every language makes. Linguists catalogue these in surveys like WALS, the **World Atlas of Language Structures**, which Inkhaven's questionnaire follows.

The questionnaire

Run the grammar command with no options to see the catalogue of twenty-two features, your language's current answers, and how much you have filled in:

```
inkhaven language grammar Eldar
```

The core sixteen are word order, adjective order, genitive order, adpositions (prepositions versus postpositions), alignment, case, gender, number, definiteness, tense, aspect, mood, evidentiality, negation, question formation, and relative clauses. Six more round out the picture: numeral and demonstrative order, morphological type (isolating, agglutinative, fusional, polysynthetic), head- versus dependent-marking, voice, and how comparison is expressed. A few of these may be unfamiliar: **aspect** is how an action unfolds in time (ongoing versus completed), **mood** is the speaker's stance (fact, command, wish), **definiteness** is the "a" versus "the" distinction, and **evidentiality** is marking how you know something (witnessed, reported, inferred) — each has a glossary entry. You do not have to answer them all; answer the ones that matter to your language and leave the rest. Once you have, `universals` (Chapter 29) tells you whether the answers you gave hang together the way real languages' do.

Three choices worth understanding

Three of the features shape a grammar more than any others. Let us define them, since they are the ones a newcomer most often meets.

TERM **Word order**

The default order of the **subject** (S, the doer), **verb** (V, the action), and **object** (O, the thing acted on) in a basic sentence. English is **SVO**: "the cat (S) sees (V) the bird (O)". Japanese and Latin are **SOV**: "the cat the bird sees". The six possible orders (SOV, SVO, VSO, …) are unevenly common across the world; SOV and SVO are by far the most frequent.

TERM **Alignment**

How a language marks the **subject** of a sentence. In a **nominative–accusative** language (like English), the doer is treated the same whether or not there is an object ("**she** runs", "**she** sees it"), and the object is marked differently. In an **ergative–absolutive** language, the lone subject of "she runs" is instead grouped with the **object** of "she sees it". This sounds exotic but is common — Basque and many others work this way. It is one of the most character-defining choices you can make.

TERM **Grammatical case**

Marking a noun's **role** in the sentence by changing its form (usually with an ending), rather than relying on word order. Latin **puella** "girl" becomes **puellam** as a direct object, **puellae** as a possessor. Common cases include **nominative** (subject), **accusative** (object), **dative** (recipient, "to/for"), and **genitive** (possessor, "of"). You built a dative suffix back in Chapter 11; declaring `case: yes` here records that your language has a case system.

Answering a feature

You set an answer with `--set feature=value`. The answer is checked against the catalogue, so you cannot set an invalid one:

```
inkhaven language grammar Eldar --set word_order=sov
inkhaven language grammar Eldar --set
alignment=nominative_accusative
inkhaven language grammar Eldar --set case=yes
```

Each answer is stored in the **Grammar** chapter. They are not just notes: Inkhaven's grammar book and study guide (Part VIII) read these answers and explain them, and the AI translator uses them to put words in the right order.

LET YOUR CHOICES REINFORCE EACH OTHER

Typological features tend to cluster in real languages — SOV languages, for instance, usually put adjectives and possessors **before** their nouns and use **postpositions** ("the house behind" rather than "behind the house"). The questionnaire shows the typical consequences of each answer, nudging you toward a grammar that hangs together naturally. You are free to break the patterns — but knowing them helps you break them on purpose.

Putting words into a sentence

Word order, case, and agreement only come alive when you string words together. Inkhaven can do exactly that. Give it a subject, a verb, and an object — each a word from your lexicon, written `root` or `root:gloss` — and it builds the clause for you:

```
inkhaven language sentence Eldar --subject kira:bird --verb
nami:see \
    --object pata:stone --object-adj mira:bright
```

Behind that one command the engine does four things in sequence. It **orders** the three constituents by your `word_order` (so an SOV language prints subject, then object, then verb). It **assigns case** by your `alignment` (nominative–accusative makes the subject nominative and the object accusative). It **inflects** each noun through your `noun` paradigm to reach that case. And it **runs agreement**, so the adjective copies its noun's case and the verb agrees with its subject. The result is printed three ways — the surface clause, an interlinear gloss, and a literal back-translation:

TERM **Interlinear gloss**

A word-by-word translation lined up **under** the original, the standard way linguists display a sentence in an unfamiliar language. Each native word sits above its meaning and grammatical tags (`stone-ACC`), so a reader can see exactly how the grammar assembles meaning.

This is the same machinery the grammar book uses to print a worked example sentence from your own vocabulary — proof that your phonology, lexicon, paradigms, and typology have come together into something you can actually **say**.

Saying no, and asking: negation and questions

A language needs more than plain statements. Two clause-level operations come straight from typology answers you already gave through the questionnaire above. To **negate** a clause, add `--negate` and, if your language has a negative word, name it with `--negator`:

```
inkhaven language sentence Eldar --subject kira:bird --verb
nami:see \
    --object pata:stone --negate --negator na:not
```

How the negation appears follows your `negation` feature: a **particle** or **auxiliary** puts the negator as its own word before the verb; an **affix** fuses it onto the verb form. If you have not coined a negator yet, Inkhaven marks only the gloss — it never invents a word for you.

To make a **polar** (yes/no) question, add `--question` (and `--q-particle` if your language uses one):

> **TERM Polar question**
>
> A yes/no question — "does the bird see the stone?" — as opposed to a **content** question that asks who, what, or where. Languages mark them by a particle, by inverting the word order, by special verb morphology, or by intonation alone.

```
inkhaven language sentence Eldar --subject kira:bird --verb
nami:see \
    --object pata:stone --question --q-particle ka:Q
```

Your `question` feature decides the realization: a **particle** is appended at the clause edge (glossed `Q`, so the example above yields `kira patan nami ka?`), **word_order** fronts the verb (English-style inversion), **morphology** tags the verb, and every strategy adds a surface "?".

Building bigger sentences: relative clauses and coordination

Real sentences nest and join. A **relative clause** lets one clause modify a noun — "the bird *that sees the stone*":

> **TERM Relative clause**
>
> A clause that modifies a noun, the way "that sees the stone" narrows down "the bird". The modified noun (the **head**) plays a role inside the embedded clause — here it is the one doing the seeing (the subject); the empty slot it leaves behind is called the **gap**.

```
inkhaven language relative Eldar --head kira:bird --role subject
\
    --verb nami:see --with pata:stone --relativizer ya:that
```

You tell Inkhaven which role the head plays inside the clause — `subject` ("the bird that sees…") or `object` ("the stone that … sees") — and supply the other argument with `--with`. The embedded clause runs through the very same engine, so it still case-marks and agrees (the object stays accusative). Whether the clause sits **before** or **after** the head follows your `relative_clause` feature (prenominal, as in Japanese, versus postnominal, as in English).

Coordination joins two things of the same kind with a conjunction — two nouns, or two whole clauses:

```
inkhaven language coordinate Eldar --np kira:bird --np pata:stone
--conjunction na:and
inkhaven language coordinate Eldar --conjunction na:and \
    --clause "kira:bird nami:see pata:stone" --clause "muru:river
tasa:fall"
```

Give two or more `--np` nouns, or two or more `--clause` clauses (each written as space-separated `root:gloss` words), and Inkhaven threads your conjunction between them — assembling each clause in full, so "bird sees stone and river falls" keeps its case marking throughout.

Finally, a **complement clause** lets one whole clause become the object of another — "I know *that the bird sees the stone*":

> **TERM** **Complement clause**
>
> A subordinate clause that serves as an argument — usually the object — of a main (*matrix*) verb of speech or thought, such as **know**, **say**, or **think**. It is often introduced by a **complementizer** like "that". The matrix clause ("I know…") and the embedded clause ("the bird sees the stone") each have their own subject and verb.

```
inkhaven language complement Eldar --subject mi:I --verb
tira:know \
    --complementizer ya:that \
    --comp-subject kira:bird --comp-verb nami:see --comp-object
pata:stone
```

The matrix subject and verb wrap the embedded clause (`--comp-subject` / `--comp-verb` / `--comp-object`), introduced by an optional complementizer (glossed `COMP`). Because the complement fills the matrix **object** slot, your word order places it correctly on its own — an SVO language prints it after the matrix verb, a verb-final language before it — and the embedded clause still case-marks its own object.

> **GRAMMAR YOU CAN HEAR**
>
> Negation, questions, relative clauses, and coordination all read from the same typology answers and the same paradigms as a plain sentence. Once your phonology, lexicon, and grammar are in place, these richer sentences cost you nothing extra — they fall out of choices you already made.

WHAT YOU LEARNED

- **Typology** is the set of high-level structural choices a grammar makes; `grammar` lists twenty-two, WALS-aligned; `universals` checks your answers against the cross-linguistic tendencies.
- **Word order** (SOV, SVO, …) sets the default subject–verb–object sequence.
- **Alignment** (nominative–accusative vs ergative–absolutive) decides how the subject is marked.
- **Case** marks a noun's role by its form (nominative, accusative, dative, genitive).
- `grammar --set feature=value` records an answer, validated against the catalogue.
- `sentence` assembles a clause — ordering, case, and agreement together — with an interlinear gloss.
- `--negate` and `--question` add negation and yes/no questions, realized by your `negation` and `question` features.
- `relative` builds a noun modified by a **relative clause** (gap + relativizer, pre- or postnominal); `coordinate` joins nouns or clauses with a conjunction.
- `complement` makes one clause the object of another ("I know **that** …"), placed by word order around the matrix verb.

CHAPTER 14

14

Idioms and metaphors

A language is more than literal meanings. “It’s raining cats and dogs” has nothing to do with animals; “time is money” treats an abstraction as a substance. These **idioms** and **conceptual metaphors** are part of what makes a language feel lived-in and human. Recording a few gives your conlang texture — and helps Inkhaven’s translator render meaning rather than word-for-word nonsense.

Idioms

An **idiom** is a fixed phrase whose meaning is not the sum of its words. You record one with its literal word-by-word reading **and** its real meaning, so both are preserved:

```
inkhaven language idiom-add Eldar --form "kala men" \
    --literal "cold heart" --meaning "unforgiving" --register
formal
```

Here the Eldar phrase **kala men** literally says “cold heart” but means “unforgiving”. The optional `--register` marks it as formal speech.

> **TERM Idiom**
>
> A fixed expression whose meaning cannot be worked out from its individual words — English “kick the bucket” means “to die”, not anything about buckets. Recording the literal reading and the real meaning separately lets a translator (human or AI) avoid translating it word for word into nonsense.

Conceptual metaphors

A **conceptual metaphor** is a deeper pattern: a whole way of thinking that maps one idea onto another, surfacing in many expressions. English speakers routinely talk about **time** as if it were **money** — you **spend** time, **save** time, find it **wasted**. Declaring such a mapping tells the translator how your speakers think:

```
inkhaven language metaphor-add Eldar --source JOURNEY --target
LIFE \
    --example "she reached a crossroads"
```

This records that your speakers understand **life** (the target) in terms of a **journey** (the source), with an example expression.

TERM **Conceptual metaphor**

A systematic way one domain of experience is understood in terms of another, underlying many everyday expressions — LIFE IS A JOURNEY, ARGUMENT IS WAR ("he **attacked** my point", "I **defended** my claim"). It is written with the **source** (the concrete domain, JOURNEY) mapped onto the **target** (the abstract one, LIFE). Choosing your language's metaphors shapes how its speakers see the world.

Reviewing what you have

List the idioms and metaphors you have recorded with:

```
inkhaven language idioms Eldar
```

Both are stored in the **Grammar** chapter alongside your typology answers. When you translate a paragraph into the language, Inkhaven consults them so the result is idiomatic — reaching for **kala men** where the meaning is "unforgiving", rather than translating "unforgiving" literally.

A LITTLE GOES A LONG WAY

You do not need many. A handful of idioms and one or two governing metaphors give a language a recognisable way of speaking — its own turns of phrase, its own way of carving up experience. They are also a delight to invent: each one is a tiny window into how your speakers think.

WHAT YOU LEARNED

- An **idiom** is a fixed phrase whose meaning is not its literal words; record both with `idiom-add`.
- A **conceptual metaphor** maps a concrete **source** domain onto an abstract **target** (LIFE IS A JOURNEY); record it with `metaphor-add`.
- `idioms` lists what you have; both are stored in the Grammar chapter.
- The translator uses them to render meaning idiomatically rather than literally.

PART V

A History for Your Language

CHAPTER 15

15

Sound change and proto-languages

Real languages have a past. Latin slowly became French, Spanish, and Italian; the single sound /k/ in Latin **centum** became the /s/ of French **cent** and stayed a /k/ in others. Languages change, and the most regular kind of change is in their sounds. Giving your language a history — descending it from an older **proto-language** by a chain of sound changes — is what turns a flat invention into something with depth. This part is optional, but it is where conlanging becomes most rewarding.

> **TERM Proto-language**
>
> An older, ancestral language that later languages descend from. Latin is the proto-language of the **Romance** family (French, Spanish, Italian, …). Proto-languages are sometimes real and recorded (like Latin) and sometimes **reconstructed** — worked out backward from their descendants, like Proto-Indo-European, which no text records.

> **TERM Sound change**
>
> A regular shift in pronunciation that spreads through a language over time, affecting every word that meets its conditions — for example, every /p/ at the end of a word becoming /f/. Because sound changes are **regular**, they are predictable, and they are the engine that turns one language into several.

Sound change is just allophony over time

Here is a piece of good news: a historical sound change is written **exactly** like an allophony rule (Chapter 7). The same `target > result / left _ right` notation — **SPE notation** — applies, sequences included (`a i > e / _ #` is the classic monophthongisation). The only difference is in the telling — allophony is variation happening **now**, within one language; a sound change is a shift that happened **over generations**, turning a parent language into a child.

Declaring a descent

You give a language a parent by adding a `diachronics` block to its **Phonology** chapter. It names the `proto` (the parent language) and an ordered list of the sound-change `rules` that turned the parent into this language:

```
{ diachronics: {
    proto: "ProtoEldarin"
    rules: [
      { rule: "p > f / _ #" }
      { rule: "k > h / V _ V" }
```

```
    ]
} }
```

This says Eldar descends from a language called **ProtoEldarin**, and two changes happened on the way: every /p/ at the end of a word became /f/, and every /k/ between two vowels became /h/. (You would, of course, first create **ProtoEldarin** as its own language with `language init`, and give it a vocabulary.)

> THE RULES ARE DEFINED ON THE PARENT'S SOUNDS
>
> The changes describe what happened to the **proto-language's** sounds, so Inkhaven uses the proto's inventory to read the words before applying the chain. Build the proto first — its phonology and at least some words — then describe how the child diverged from it.

Evolving a word

Watch a single proto-word travel down the chain to its modern form:

```
inkhaven language sound-change Eldar --form tap
```

Inkhaven applies the rules in order and prints the result — `tap > taf` (the final /p/ became /f/). Try it with several words to feel how the chain reshapes the language.

Evolving the whole vocabulary

The real payoff: take the proto-language's **entire** dictionary and evolve it, producing the daughter's vocabulary in one step — each word transformed by the sound changes, with its meaning carried forward and its origin recorded:

```
inkhaven language derive-lexicon Eldar --yes
```

Without `--yes` it only proposes; with it, the words are added to Eldar's dictionary, each remembering the proto-word it came from. In moments you have a vocabulary that is **systematically related** to its ancestor — the hallmark of a real language family, which the next chapter explores.

WHAT YOU LEARNED

- A **proto-language** is an ancestor; **sound changes** are the regular shifts that turn it into descendants.
- A sound change uses the same SPE notation as allophony — change over generations rather than variation now.
- Declare descent with a `diachronics` block: a `proto` and an ordered list of `rules`.
- `sound-change --form` evolves one word; `derive-lexicon` evolves the proto's whole dictionary into the daughter's.

CHAPTER 16

16

Language families

Give one proto-language **two** sets of sound changes and you have two sister languages — a **family**. French and Spanish are sisters, both children of Latin, recognisably related yet distinct. This chapter shows how to grow and explore a family: how related words line up across the daughters, how to draw the family tree, and how Inkhaven can even reconstruct a lost ancestor or judge whether your history is believable.

TERM **Language family**

A group of languages descended from a common proto-language. Members are **related**; a word that survives from the ancestor into several daughters appears in each as a **cognate**. Examples: the Romance family (from Latin), the Germanic family (English, German, Dutch, …).

TERM **Daughter language**

A language that descends from a given proto-language. French and Spanish are daughters of Latin. You make a daughter by creating a language whose `diachronics` block names the proto and gives the sound changes that produced it (Chapter 15).

Cognates: the same word, evolved

When a proto-word survives into several daughters, its descendants are **cognates** — the "same word", reshaped differently by each daughter's sound changes. Latin **centum** gives French **cent**, Spanish **ciento**, Italian **cento**: cognates all. Inkhaven traces a proto-form's reflex in every daughter at once:

```
inkhaven language cognates ProtoEldarin --form takap
```

For the proto-form **takap** it applies each daughter's chain and prints the result in each — perhaps `Eldar takaf` versus `Sindarin tahaf`. Lining up cognates this way is exactly how historical linguists study real families, and it is deeply satisfying to watch your own languages diverge from a shared root.

TERM **Cognate**

A word in one language that descends from the same ancestral word as a word in a related language. English **father**, German **Vater**, and Latin **pater** are cognates — all from one Proto-Indo-European word. Cognates are how relatedness is proven, and how the shape of the proto is recovered.

The family tree

To see the whole family laid out — every language under its declared ancestor — draw the tree:

```
inkhaven language family-tree
```

It prints a genealogical diagram of your languages, each nested under its `proto`, like this:

```
ProtoEldarin
├─ Eldar
│  └─ LowEldar
└─ Sindarin
```

Here **Eldar** and **Sindarin** both descend from **ProtoEldarin**, and **LowEldar** is a daughter of **Eldar** — a granddaughter of the proto. As you add daughters and granddaughters, the tree grows to show the full shape of your invented family.

Looking backward: reconstruction

Historical linguists often work in the other direction: given several cognates, they **reconstruct** the ancestral form that must have produced them. Inkhaven can do this with AI — propose the proto-form from a set of daughter forms:

```
inkhaven language reconstruct --forms "tava taba" --gloss water
```

Given the cognate forms **tava** and **taba** (both meaning "water"), it proposes the most plausible ancestor they could both come from, and explains the reasoning. This is AI-assisted and advisory — a creative aid for designing a deep history.

> TERM **Reconstruction**
>
> Working out the form of an unrecorded ancestral word from its surviving descendants, by reversing the sound changes that produced them. The starred forms in linguistics books (*__pater__) are reconstructions. Inkhaven's `reconstruct` proposes one for you from cognate forms.

Is your history believable?

Finally, you can ask the AI whether the chain of sound changes you invented is **typologically plausible** — whether each change is a natural kind that real languages actually undergo, and whether the order makes sense:

```
inkhaven language realism-check Eldar
```

It assesses your sound-change chain and flags anything unnatural, so you can make a history that feels real. Like reconstruction, it is advisory: a knowledgeable second opinion, not a verdict.

Keeping a register of hypotheses

Serious historical work is an accumulation of **claims**: "*k became tʃ before front vowels in the Northern branch"; "these two daughter forms are cognate"; "this word is a loan". Each has evidence, and each is either holding up or falling apart as you find more of it. Inkhaven can keep that reasoning rather than leaving it in your head.

```
inkhaven language hypothesize <lang> --kind sound-change --claim
"k > tʃ / _
i" --evidence "kina → tʃina"
```

records a hypothesis; it starts **proposed**, and as the evidence comes in you move it along with `hypothesis-status ... --status supported` (or `refuted`). `inkhaven language hypotheses <lang>` shows the register at a glance, each line marked with its status, and `/hypotheses` lists it in the companion.

> **TERM Hypothesis (in historical linguistics)**
>
> A proposed regular sound change, cognacy relation, or borrowing, held provisionally and tested against the data. The comparative method is precisely the business of proposing such hypotheses and keeping the ones that survive every relevant form — which is why recording them, with their evidence and their fate, is how the work is actually done.

A sound-change hypothesis can be **tested** on the spot: `inkhaven language hypothesis-check <lang> --id front-k` runs its claim through the Consequence Tracer of Chapter 15 and shows exactly which words it predicts will

shift and which merge into homophones — the claim made falsifiable, not merely asserted. If the prediction matches the forms you expect, move it to **supported**; if it collapses distinctions you meant to keep, that is your evidence to **refute** it. In the companion the same check is `/hcheck <id>`.

The register turns the one-shot tools of this chapter into a **method**: propose a change, trace its consequences, check the cognates it predicts, and write down whether it held. A refuted hypothesis kept on the books is as valuable as a supported one — it stops you proposing it again.

WHERE THE SUITE SHINES

Diachronics is where Inkhaven's design pays off most. Because sound change reuses the allophony engine, because daughters reuse the lexicon, and because cognates reuse the change-chains, a few rules give you a whole evolving family for almost no extra effort. A proto plus two daughters is a weekend's work and an enormous boost to realism.

WHAT YOU LEARNED

- A **language family** is a set of daughters from one proto; related words are **cognates**.
- `cognates <proto> --form` shows a proto-word's reflex in every daughter; `family-tree` draws the whole family.
- `reconstruct --forms` (AI) proposes a lost ancestor from cognates; `realism-check` (AI) judges whether your sound history is plausible.
- Two daughters of one proto is a small effort for a large gain in depth.

PART VI

A Language in a World

CHAPTER 17

17

Dialects and registers

No real language is spoken just one way. A farmer in the hills and a courtier in the capital share a language, yet a listener can place each of them in a sentence or two — by a vowel here, a different word there. The same speaker, too, talks differently to a magistrate than to a child. These systematic ways one language varies — across regions, across classes, across formality — are what make a tongue feel inhabited rather than printed. This chapter shows how to give your language varieties, and the single idea that makes them almost free to build.

TERM **Variety**

A consistent, recognisable **way of speaking** one language — a dialect, a register, a class accent. A variety is not a separate language; it is the base language with a set of differences. Standard English, Scots English, and legalese are three varieties of one language.

TERM **Dialect**

A variety tied to a **place** or community: the speech of a region, distinguished by its sounds, words, and sometimes grammar. Mutually intelligible with the rest of the language — that is what separates a dialect from a sister language.

TERM **Register**

A variety tied to the **situation** rather than the speaker: formal versus casual, ceremonial versus intimate. The same person commands several registers and switches between them by setting.

The one idea: a variety is a delta

A variety is stored not as a whole second language but as a **delta** — a short list of the differences from the base. And here is the idea that makes the whole pillar cheap:

A DIALECT IS SOUND CHANGE, HAPPENING NOW

The sound differences that mark a dialect are **exactly** an ordered list of sound changes — the same `target > result / left _ right` notation, the same engine, as the historical changes of Chapter 15. The only difference is in the telling. A **daughter language** applies its changes **diachronically**, across generations, and becomes a new language. A **dialect** applies its changes **synchronously**, here and now, and stays the same language. One engine, two framings.

So everything you learned about sound change you already know about dialects. A lowland dialect in which /t/ softens to /d/ between vowels is one rule: `t > d / V _ V`. That is the whole dialect, as far as its sounds go.

Declaring varieties

Varieties live in a `varieties` block in the language's **Grammar** chapter. Each has an `id`, a `kind` (`dialect`, `register`, `sociolect`, or `idiolect`), an `axis` saying what it varies along, an optional `prestige`, its `sound_changes`, and optional word `lexicon` overrides:

```
{ varieties: [
  { id: "lowland", kind: "dialect", axis: "region", prestige:
"low",
    sound_changes: [ { rule: "t > d / V _ V" } ],   // SPE, as in
diachronics
    lexicon: { "water": "móru" } }                  // a
suppletive override
  { id: "high", kind: "register", axis: "formality", prestige:
"high",
    sound_changes: [ { rule: "k > q / # _" } ] }
] }
```

The first is a low-prestige regional dialect with one sound change and one word swapped outright; the second is a high register that hardens word-initial /k/ to /q/. The two `kind`s you have not met yet name finer cuts:

> TERM **Sociolect**
>
> A variety tied to a **social group** — class, trade, generation — rather than a region. The clipped speech of an aristocracy and the slang of an apprentices' guild are sociolects.

> TERM **Idiolect**
>
> The way **one individual** speaks: their personal blend of dialect, register, and habit. Every character in your world has one; Chapter 19 builds idiolects from a character's background.

Two ways a variety differs

A variety changes a word in one of two ways. Most differences are **regular** — produced by the sound changes, applying to every word that meets their conditions. But a variety may also simply **use a different word** for a meaning, unrelated to any sound rule. That is a `lexicon` override:

> **TERM Suppletion (override)**
>
> A form that does not follow from the regular rules but is supplied outright — as English uses **went** for the past of **go**, an unrelated word. A variety's `lexicon` overrides are suppletive: the lowland word for "water" is **móru** not because a sound change produced it, but because the lowlanders simply say **móru**.

Rendering speech in a variety

With varieties declared, you can render any word or line of text **as a given variety would say it**:

```
inkhaven language varieties Eldar                       # list
them
inkhaven language lect Eldar lowland --word kata        # → kada
(t > d / V _ V)
inkhaven language lect Eldar lowland --text "kata tira" # word by
word
```

`varieties` lists each variety with its axis, prestige, and the size of its delta. `lect` (from **dialect**) renders a form or a whole line in a chosen variety: the base word **kata** comes out **kada** in the lowland dialect, because the /t/ sits between two vowels. `lect` also reports the base-to-variety difference, so you can see exactly what changed.

The dialectology table

The classic way linguists display variation is a table: each headword down the side, each variety across the top, so the eye can run along a row and watch a word shift from dialect to dialect. Inkhaven prints exactly that:

```
inkhaven language dialects Eldar [--count N]
```

Each row is a base word and its form in every variety; a trailing `*` marks a word that was **overridden** outright rather than derived by sound change. This is the single most satisfying view of the pillar — your one language, fanned out into a living spread of ways to say the same thing.

Letting the AI propose a dialect

Inventing a coherent set of sound changes for a dialect — ones that hang together and suit a culture — is creative work the AI can seed. Describe the flavour you want and let it propose:

```
inkhaven language propose-dialect Eldar --describe "a harsh
mountain dialect" [--yes]
```

The model suggests a set of sound changes and a few lexical swaps that fit the description. Crucially, **the AI only proposes**: each rule it offers is validated against your actual phoneme inventory and previewed through the variety engine above, so nothing illegal can slip in. Without `--yes` it only shows you the preview; with `--yes` it writes the variety into your Grammar chapter, ready for `lect` and `dialects`. This is the advisory pattern you saw with `reconstruct` and `realism-check`: the AI brings ideas, the deterministic engine keeps them legal, and nothing is written without your word.

WHAT YOU LEARNED

- A **variety** is a consistent way of speaking one language — a **dialect** (region), **register** (situation), **sociolect** (group), or **idiolect** (person) — stored as a small **delta** on the base.
- The headline: a dialect's sound differences are an ordered list of sound changes (Chapter 15's engine) applied **synchronously** rather than across generations.
- Declare varieties in a `varieties` block; a variety differs by regular `sound_changes` and by suppletive `lexicon` overrides.
- `lect` renders a form or text in a variety; `dialects` prints the comparison table (a `*` marks an override).
- `propose-dialect` (AI) suggests a coherent dialect; each rule is validated and previewed, and nothing is saved without `--yes`.

CHAPTER 18

18

Languages in contact

Languages do not live alone. Where two peoples trade, fight, or share a border, their languages reach into one another — words cross over, and over centuries even grammar drifts toward a common shape. English took **beef** from French and **ski** from Norwegian; the languages of the Balkans, though unrelated, came to build their futures and articles alike from living side by side. This chapter gives your languages neighbours: how they borrow words from one another, and how contact pulls a whole region toward a shared type.

Borrowing: a word crosses over

When a language takes a word from another, it rarely keeps it intact. The borrowed word is **heard** through the ears of the receiving language and **bent** to fit its rules. Japanese borrowed English **strike** and made it **sutoraiku** — because Japanese has no **str-** cluster and no word-final **k**, so vowels were inserted to break the sounds into syllables it allows.

> TERM **Loanword**
>
> A word taken from one language (the **donor**) into another (the **recipient**), and adapted to the recipient's sound system. **Loanword** is itself a loan- translation of German **Lehnwort**.

> A LOANWORD IS A PHONOTACTIC REPAIR
>
> The key idea mirrors the last chapter's. A loanword is not a free invention: it is the donor word **repaired** to obey the recipient's phonotactics (Chapter 6). The recipient perceives the foreign sounds as the nearest ones it has, then fixes any sequence its templates forbid. Borrowing reuses the very constraints you wrote for word-building.

Declaring how a language borrows

How a language nativises foreign words is declared in a `loan_phonology` block in its **Phonology** chapter:

```
{ loan_phonology: {
  repair: "epenthesis",          // epenthesis (insert a vowel) |
deletion
  epenthetic_vowel: "u",         // empty → the first declared
vowel
  substitutions: { "θ": "t", "r": "l" }   // a donor sound we
lack → nearest native
} }
```

This says Eldar repairs illegal clusters by **inserting** a vowel (an /u/), and that it hears the foreign sounds /θ/ and /r/ — which it does not have — as its own /t/ and /l/.

> TERM **Epenthesis**
>
> Inserting a sound (usually a vowel) to break up a cluster the language forbids, turning **str** into **sutu…**. The opposite repair is **deletion** — simply dropping the offending consonant. A language picks one strategy as its habit.

> TERM **Substitution**
>
> Mapping a donor sound the recipient lacks onto the nearest sound it does have. A language with no **th** sound will hear it as **t**, **s**, or **f** depending on which is closest in its own inventory.

Borrowing a word

Give the donor form **phonemically** — one symbol per sound — and the recipient and donor languages, and watch the adaptation:

```
inkhaven language borrow Eldar --form tras --from Drake
# tras → tulasu
inkhaven language borrow Eldar --form θuk --from Drake --gloss
demon --yes
```

The adaptation runs in two steps you can read in the output. First **perceive**: apply the substitutions, keep every sound the recipient already has, and map the rest to the nearest native phoneme. Then **repair**: any consonant run longer than the recipient's templates allow gets the epenthetic vowel (or loses a consonant, if the language deletes). So **tras** — with an illegal **tr-** onset and final **-s** cluster — becomes **tulasu**. With `--yes` and a `--gloss`, the adapted word joins the recipient's dictionary, the donor recorded in its etymology, so your borrowings stay historically coherent.

When a whole region converges

Borrowing moves words. **Contact** over long enough also moves **structure**: unrelated languages sharing a region drift toward a common grammatical type — the same word order, the same way of marking the subject — until a stranger could mistake their blueprints for kin.

> TERM **Sprachbund (linguistic area)**
>
> A group of languages that have grown structurally alike through long contact rather than common descent — a **language area**. The German term ("language league") is standard. The Balkans are the classic example: Greek, Albanian, and the local Slavic and Romance tongues share features none inherited.

You declare a language's membership in such an area with a `contact` block in its **Grammar** chapter:

```
{ contact: {
  region: "the Inner Sea"
  with: [ "Sindar", "Khuz" ]                                    //
neighbours
  areal_features: { word_order: "sov", alignment:
"ergative_absolutive" }
} }
```

> TERM **Areal feature**
>
> A grammatical trait shared across a **Sprachbund** because of contact, not inheritance — the thing that spread. The shared subject–object–verb order of a language area is an areal feature.

Reading the convergence

`areal` holds your declared neighbourhood up against a language's own grammar:

```
inkhaven language areal Eldar      # one language's convergence
overlay
```

```
inkhaven language areal            # the whole-region Sprachbund
view
```

For each shared feature, the overlay reports where the language stands: **converged** (it already has the feature, marked `✓`), **shift** (it answers differently and would have to change, `→`), or **adopt** (it has no answer yet and would gain one, `+`). This is strictly an **advisory overlay** — it shows you what joining the area would mean and never rewrites your grammar. With no language named, `areal` prints the regional view: every contact area, its members, and each member's status on every shared feature. Contact also surfaces in the family tree from Chapter 16, as **horizontal** edges (`Eldar ⇄ Sindar`) crossing the vertical lines of descent — kinship and neighbourhood on one diagram.

AI help for contact

Two advisories round out the pillar, both keyed to your project's working language and both write-nothing-without-`--yes`:

```
inkhaven language propose-loans Eldar --from Drake --topic
seafaring --count 6
inkhaven language areal-check Eldar
```

`propose-loans` asks the AI what concepts a recipient would plausibly borrow from a donor in a given domain — a seafaring people lending nautical words — and offers donor forms for each; the **deterministic** borrowing engine then nativises every one (so a donor **stɔrm** comes back as **sitarimi**), and you add the ones you like with `borrow … --yes`. `areal-check` is the contact cousin of `realism-check`: it judges whether a Sprachbund you have declared is typologically believable — the kind of convergence real language areas actually show.

WHAT YOU LEARNED

- Languages in contact **borrow words** and, over long enough, **converge in structure**.
- A loanword is a **phonotactic repair**: declare a `loan_phonology` block (`repair`, `epenthetic_vowel`, `substitutions`); `borrow` perceives then repairs the donor form (**tras → tulasu**).
- A **Sprachbund** is a region grown alike by contact; declare it with a `contact` block and read convergence with `areal` (✓ converged, → shift, + adopt).
- Contact shows as horizontal `⇄` edges in `family-tree`, beside the vertical lines of descent.
- `propose-loans` (AI) suggests borrowings the engine then nativises; `areal-check` (AI) judges a Sprachbund's plausibility. Both advisory, `--yes` -gated.

CHAPTER 19

19

Speech communities and ecology

A language is spoken by **someone**, **somewhere**. The varieties of Chapter 17 and the contact of Chapter 18 only matter because a particular town speaks the lowland dialect and a particular character grew up there. This last chapter of the pillar ties your languages to your world — to its places and its people — and then renders a character's speech in their own voice.

Linking languages to your world

Inkhaven records who speaks what in a small sidecar file, separate from your prose — your manuscript is never touched. You attach a language to a place, and record a character's command of it:

```
inkhaven language link-place Tirion Eldar
inkhaven language link-character Erendil Eldar native
inkhaven language speakers Eldar
```

The first says the city of Tirion speaks Eldar; the second records that the character Erendil is a **native** speaker (other levels mark fluent or learner command). `speakers` then lists everyone and everywhere tied to a language. A place may have a secondary language (`--secondary`), and a character may command several — the picture of a multilingual world.

> **TERM Speech community**
>
> The group of people who share a language or variety — the speakers of a town, a class, a guild. A **place** and a **character** are how Inkhaven anchors speech communities: a community is the set of speakers a language reaches.

Which variety, exactly

A town does not speak "the language" in the abstract — it speaks a **variety** of it. And a person's speech is built on a **home** variety, the one they grew up in. The links carry both:

```
inkhaven language link-place Tirion Eldar --variety lowland
inkhaven language link-character Tost Eldar native --native-
variety lowland
```

Now Tirion does not merely speak Eldar; it speaks Eldar's **lowland** dialect. And Tost's speech is grounded in that same dialect — the base of their idiolect.

> TERM **Native variety**
>
> The variety a speaker acquired first and speaks by default — the dialect or register at the base of their personal **idiolect** (Chapter 17). A marsh-born character's native variety is the marsh dialect, however far they later travel.

The ecology view

With places and people tied to languages and varieties, Inkhaven can draw the whole picture — its **language ecology**:

```
inkhaven language ecology                  # who speaks what, and
which variety, where
inkhaven language ecology -o atlas.svg     # a node-link atlas of
the world's tongues
```

> TERM **Language ecology**
>
> The full pattern of languages and varieties in a world — which are spoken where, by whom, in contact with what. The term (borrowed from biology) frames languages as living in an environment of speakers and neighbours, not in isolation.

The text report lays out every place with its language and variety, every character with the languages they command and their native variety, and the contact areas from Chapter 18. With `-o atlas.svg` it writes a labelled **atlas** — a node-link diagram of the world's languages and the places and people joined to them — a map you can drop straight into your worldbuilding notes.

Speaking as a character

The payoff of all this wiring: render a line **as a particular character would say it**, automatically in their native variety.

```
inkhaven language idiolect Tost --word kata          # → kada
inkhaven language idiolect Tost --text "kata tira"   # a whole
line, in Tost's dialect
```

`idiolect` looks up the character's primary language and native variety, then runs the text through the variety engine of Chapter 17. Because Tost's native variety is the lowland dialect — where /t/ softens between vowels — **kata** comes out of Tost's mouth as **kada**, with no further instruction from you. A marsh-dweller's speech simply **arrives** in the marsh dialect. This is dialogue that knows who is talking.

IT REACHES THE GRAMMAR BOOK TOO

When a language declares varieties or contact, its reference grammar (Chapter 23) grows two new sections automatically: a **Variation** section listing the dialects and registers with the dialectology comparison table, and a **Contact** section describing its linguistic area, shared areal features, and how it adapts loanwords. The sociolinguistics you build here becomes part of the printed, published description of your language.

WHAT YOU LEARNED

- Inkhaven ties languages to your world in a sidecar (your prose is untouched): `link-place` and `link-character`, listed by `speakers`.
- Places carry a `--variety` and characters a `--native-variety`, so a **speech community** speaks a specific dialect, not the language in the abstract.
- `ecology` reports the whole **language ecology** — who speaks what variety where, plus contact areas; `-o FILE.svg` writes a node-link atlas.
- `idiolect <character>` renders a form or line in that character's native variety automatically — dialogue in the speaker's own voice.
- Declared varieties and contact also appear as **Variation** and **Contact** sections in the reference grammar (Chapter 23).

PART VII

A Writing System

CHAPTER 20

20

Designing glyphs

So far your language has been written in ordinary letters. Now you can give it its own **script** — an alphabet of shapes you design, compiled into a real, usable **font** that renders your words in their native form. This is one of Inkhaven's most striking features, and it does it entirely on its own, with no other software. This part is optional and the most advanced; the language is complete without it.

TERM **Script and glyph**

A **script** is a system of written symbols for a language (the Latin script, the Cyrillic script, the Korean Hangul script). A **glyph** is one such symbol — one drawn shape. Designing a script means designing its glyphs, usually one per sound or per syllable.

TERM **Font**

A file containing the drawn glyphs of a script in a form computers can display and print. When you "install a font" you are giving your system a set of glyph shapes tied to characters. Inkhaven compiles your glyph drawings into a standard **TrueType** font that any program can use.

What a glyph file looks like

Each glyph is a small **SVG** file — a vector drawing, the kind made of shapes and curves rather than pixels, so it scales to any size cleanly. A glyph for the sound /a/ might be a simple drawing saved as `a.svg`. You can draw glyphs in any vector editor (Inkscape, Illustrator, and many free tools), or — as we will see — have the AI draft them from a description.

TERM **SVG (vector drawing)**

Scalable Vector Graphics — a way of describing a drawing as shapes and curves (a filled outline) rather than a grid of pixels. Because it is defined mathematically, it stays crisp at any size, which is exactly what a font needs. Each glyph is one SVG file.

What makes a good font glyph

Not every drawing works as a font glyph. A font glyph must be a **filled shape** — a solid black outline — not a thin line, not a photo, not a coloured gradient. A font is monochrome: it has one ink colour, so a glyph is defined purely by its outline. Before you commit a glyph, check it with the **lint** command:

```
inkhaven language glyph-lint --svg ./a.svg
```

This reports whether the SVG is suitable: it requires filled paths, and flags common mistakes — a stroke-only line (which has no fillable shape), an embedded photo, a gradient, or a near-white fill where you probably meant to cut a hole.

> THE WHITE-FILL TRAP
>
> A frequent beginner mistake is to draw the hole in a letter like "O" by painting a white circle over a black one. In a monochrome font this does **not** make a hole — both shapes become solid ink. The correct way to cut a hole (a **counter**) is to draw the inner outline wound in the opposite direction to the outer one. The lint command warns you when it sees a likely white-fill mistake.

> TERM **Counter**
>
> An enclosed empty space inside a glyph — the hole in "O", "D", or "e". In a font, a counter is made by drawing the inner outline in the **reverse** direction to the outer outline, not by painting it white. The opposing directions tell the font where the ink is and where the hole is.

Drawing a glyph with AI

If you would rather describe a glyph than draw it, the AI can draft one for you. You give a short description; it produces an SVG, runs it through the same lint check, and shows you the result:

```
inkhaven language glyph-draft Eldar --describe "a vertical stroke
with a hook" \
    --phoneme p -o p.svg
```

By default it only drafts and previews. The drawing is advisory — review it, re-run with a better description if you like, and only when you are happy do you keep it. (Adding `--yes`, and only if the draft passes the lint check, binds it straight into your font, which the next chapter explains.) The AI is told to draw filled, monochrome shapes and to cut counters the correct way, so its output is usually ready to use.

HAND-DRAWN AND AI GLYPHS MIX FREELY

You can draw some glyphs yourself and have the AI draft others; both go through the same lint check and into the same font. A common workflow is to let the AI rough out a whole alphabet from descriptions, lint them all, and then redraw by hand the few you want to perfect.

WHAT YOU LEARNED

- A **script** is made of **glyphs**; each is a filled **SVG** vector drawing, one per sound.
- A font glyph must be a **filled, monochrome shape**; `glyph-lint` checks suitability.
- Cut holes (**counters**) with reverse-wound outlines, never white fill — lint warns you.
- `glyph-draft --describe …` has the AI draft a glyph from words; advisory, lint-checked, kept only on your approval.

CHAPTER 21

21

Building the font

You have glyphs. Now you **bind** each one to a sound and a character slot, and compile them into a finished font file. Inkhaven does the whole compilation itself — no font-editing software required — and the result is an ordinary TrueType font you can install, embed in documents, and share.

Binding a glyph

To make a glyph part of your language's writing system, you **import** it: Inkhaven preflights it (the lint check from last chapter), copies it into the language's glyph store, and records which sound it represents and which character slot it occupies.

```
inkhaven language font-import-glyph Eldar --svg ./a.svg --phoneme
a --codepoint a
```

This binds the drawing `a.svg` to the phoneme /a/ and to the character "a". For a sound that has an ordinary letter, you can use that letter as the codepoint, as here. For invented symbols with no letter, you use a number from a special range (explained next).

> TERM **Codepoint**
>
> The unique number a computer uses to identify a character. Every character — "A", "ß", "あ", an emoji — has a codepoint in the **Unicode** standard, written like **U+0041**. A font maps codepoints to glyphs: when text contains a codepoint, the font supplies the matching drawn shape.

> TERM **The Private Use Area**
>
> A block of Unicode codepoints (starting at **U+E000**) deliberately left unassigned, reserved for anyone to use for their own characters. This is exactly where your invented symbols belong — they are real, usable characters, but they do not collide with any standard ones. Bind a glyph with no natural letter to a Private Use codepoint like `U+E000`.

For an invented symbol, bind it to a Private Use codepoint:

```
inkhaven language font-import-glyph Eldar --svg ./sun.svg --
phoneme o --codepoint U+E000 --name sun
```

The font lives in your language

These bindings are stored as a `font` block in the language — a record of the family name, the design grid size, and every glyph with its codepoint and phoneme. You do not have to write this by hand; `font-import-glyph` builds it for you. To review what you have bound, with the status of each glyph's artwork:

```
inkhaven language font-config Eldar
```

This lists every glyph, its codepoint, the phoneme it stands for, and whether its drawing is present and usable.

Compiling the font

When your glyphs are bound, compile them into a font file straight from the language:

```
inkhaven language font-build --language Eldar --format ttf -o
Eldar
```

This produces `Eldar.ttf`, a standard TrueType font (`-o` gives the file's **stem**; the extension follows `--format`). Leave `-o` off and the font lands in your project's `exports/eldar/fonts/` folder, and the command prints the path it chose. Inkhaven does the entire compilation in-process: it converts each glyph's outline into font curves, lays them out on the design grid, and assembles a complete font file — with no external program. You can also ask for `ufo` (an editable font **source** you can open in professional font editors) or `both`.

> TERM **TrueType and UFO**
>
> **TrueType** (a `.ttf` file) is the everyday font format your computer already uses — ready to install and use immediately. **UFO** (Unified Font Object) is an editable font **source**, the working format of professional font editors like FontForge or Glyphs. `font-build` can produce either; use TTF to **use** the font, UFO to **keep editing** it.

> USING YOUR FONT
>
> The `.ttf` file is a real font. Install it like any other to type your script in any program, or — as Part VIII shows — let Inkhaven embed it directly in the PDF books it produces, so your dictionary shows each headword in its own native script. To preview the font in a document, compile a Typst file that uses it with `typst compile --font-path <folder> document.typ`.

WHAT YOU LEARNED

- Bind each glyph to a **phoneme** and a **codepoint** with `font-import-glyph`.
- Ordinary sounds can use their letter; invented symbols use the **Private Use Area** (`U+E000` and up).
- Bindings live in a `font` block; `font-config` lists them with artwork status.
- `font-build --format ttf` compiles a real **TrueType** font in-process; `ufo` gives an editable source.

CHAPTER 22

22

Complex scripts and typing

Not every writing system is a simple row of letters. Korean stacks letters into square syllable blocks; Egyptian hieroglyphs pack signs into rectangular groups. And once you have a font, you will want to **type** in it — to turn romanized text into your script automatically. This chapter covers both: composing complex units, and typing the script.

Composed blocks

Some scripts build one written unit out of several component glyphs arranged in two dimensions — a Korean syllable square is a **lead** consonant, a **vowel**, and a **tail**

consonant placed in a grid. Inkhaven supports this with **spatial templates**: named layouts that say where each component goes.

> TERM **Spatial template**
>
> A named layout that divides a square into cells — **left/right**, **top/bottom**, a **2×2 quadrat**, **three stacked rows** — into which component glyphs are placed to build a composite character. Used for syllable-block scripts (like Hangul) and packed scripts (like Egyptian quadrats).

List the available templates, then compose a block by dropping a glyph into each cell:

```
inkhaven language font-templates Eldar
inkhaven language font-compose Eldar --template lr \
    --name ka --codepoint U+AC00 --phoneme ka \
    --slot left=lead --slot right=vowel --yes
```

The built-in templates are `lr` (left/right), `tb` (top/bottom), `quad` (a 2×2 grid), and `stack3` (three rows). This **bakes** the two components into a single new glyph — a precomposed block — that lives in the font alongside its parts.

(The example binds the block to `U+AC00`, the real Hangul syllable 가, because it is modelling a real script. For an invented script, give it a **Private Use Area** codepoint instead, exactly as in the previous chapter.)

Layout-time arrangement

Baking every possible block into the font works for a script with a fixed set of syllables, but not for one where signs combine freely and endlessly — like hieroglyphs. For those, Inkhaven can arrange the components at **layout time** instead, emitting a small Typst snippet that places each component in its cell when the document is typeset:

```
inkhaven language spatial-typst Eldar --template tb \
    --name quadrat_sunbar --slot top=sun --slot bottom=bar
```

You drop the snippet into a Typst document, and the quadrat renders with the glyphs stacked — no precomposed glyph required. This is the path for scripts where the number of possible combinations is too large to bake.

Typing the script

Finally, you want to write **in** your script without hunting for codepoints. The **transliterate** command turns romanized text into your script's characters, using the phoneme bindings you made when building the font:

```
inkhaven language transliterate Eldar --text "katha"
```

It reads the romanized input left to right, matching the **longest** glyph key at each step — so a two-letter key like `th` or `ka` wins over the single letters `t` and `h` — and outputs the corresponding string of your script's codepoints. Anything it cannot match passes through unchanged and is flagged, so you know which sounds still need a glyph.

> TERM **Transliteration (input method)**
>
> Converting text from one script into another — here, from the Latin letters you type into your invented script's characters. The rule "longest key wins" means a digraph (two letters standing for one sound, like `th`) is matched as a unit before its individual letters. This is the engine behind typing a constructed script.

> THIS IS ENOUGH FOR A REAL SCRIPT
>
> With glyphs, a compiled font, optional composed blocks, and transliteration, your language has a complete, usable writing system. The dictionary and grammar books of the next part will show each word in this native script, rendered with the very font you built.

WHAT YOU LEARNED

- **Spatial templates** arrange component glyphs into composite units (Hangul squares, quadrats).
- `font-compose` **bakes** a composed block into the font; `spatial-typst` arranges components at **layout time** for open-ended scripts.
- `transliterate` types the script: romanized text → script codepoints, longest key first.
- Together these give a complete, typeable writing system rendered by your own font.

PART VIII

Producing the Books

CHAPTER 23

23

Producing the books

Everything you have built — sounds, words, grammar, history, a script — can now be turned into finished, printable books. Inkhaven produces four kinds of document from your language's own data: a **dictionary**, a **reference grammar**, a **study guide**, and a learner's **tutorial**. Each comes in two flavours: plain Markdown, or a beautifully typeset book (the same B5 design as the book in your hands), ready to compile into a PDF that embeds your conscript font.

A quick word about output formats

Every output command takes `--format md` or `--format typ`. **Markdown** (`md`) is plain text you can read anywhere or paste into other tools. **Typst** (`typ`) produces a typeset book; you turn it into a PDF with the free Typst program:

```
typst compile --font-path <folder-with-your-ttf> book.typ
book.pdf
```

The `--font-path` points Typst at the folder holding your `Eldar.ttf`, so the PDF can render your script. When a command takes `--font Eldar`, it embeds that font and shows your words in their native form.

The dictionary

```
inkhaven language dictionary Eldar --format typ --font Eldar -o
dict.typ
```

This renders your lexicon as a real dictionary: a title page, a table of contents, an overview of the language, and a two-column list of every word with its pronunciation, part of speech, and meaning, in **your** alphabet's order — the `alphabet` you declared in the Meta block (Appendix B) sets the sequence of the letter sections and of the words inside them; letters you did not declare follow, by code point. The Dictionary chapter in the editor keeps the same order: `add-word` files each entry in place, and `inkhaven language sort-dictionary Eldar` (preview with `--dry-run`) puts an older dictionary back in order. When you pass `--font`, each headword also appears in your native script beside its romanized form. Compile it with Typst and you have a printable dictionary of your language.

The reference grammar

```
inkhaven language grammar-book Eldar --format typ --font Eldar -o
grammar.typ
```

The companion volume: a reference grammar drawn entirely from your language's data — the phonology (inventory, syllable structure, phonotactics, allophony, stress,

tone), the morphology (affixes and derivations), the typology answers, the idioms and metaphors, and the sample texts. It is a faithful, deterministic description of exactly what you built.

In the morphology section, affixes are **grouped by category** — all the case endings together, all the number endings together — using the `category` and `value` tags you gave each morpheme (Chapter 11), with each one's kind (prefix, suffix, infix, circumfix, ablaut, reduplication) shown. Any agreement rules get their own short section. The more carefully you tag your morphemes, the clearer this grammar reads.

If your language declares the sociolinguistics of Part VI, the grammar grows two more sections automatically. A **Variation** section lists every dialect and register with the dialectology comparison table (Chapter 17), so the printed grammar records how the language is spoken across its speakers. A **Contact** section describes the linguistic area, the areal features it shares, and how it adapts loanwords (Chapter 18). Declare varieties and contact, and your published grammar describes a living, situated language — not one frozen in a single voice.

The study guide (AI)

A bare reference grammar can be daunting to a newcomer who does not know what "allophony" or "nominative–accusative alignment" mean. Add `--study` and Inkhaven prepends an AI-written **study guide** that defines and explains every linguistic term the grammar uses, grounded in your language's own examples:

```
inkhaven language grammar-book Eldar --format typ --font Eldar \
    --study --provider deepseek -o grammar.typ
```

The reference itself stays exactly as before — only the study guide is AI-written, and it needs an AI provider. It turns a technical reference into something a beginner can actually learn from.

The learner's tutorial (AI)

Finally, the gentlest on-ramp of all — a complete beginner's **textbook** for your language, written by the AI from your data:

```
inkhaven language tutorial Eldar --format typ --font Eldar \
    --provider deepseek -o learn.typ
```

The model authors a graded course: a warm introduction, a pronunciation guide, lessons that **explain** the grammar with worked examples drawn from your words, a reading passage, and a practice exercise per lesson. It is constrained to your language's actual sounds, words, and rules — it never invents vocabulary or grammar. This is the book you would hand someone who wants to **learn** your language, rather than look things up in it.

> **DETERMINISTIC WHERE IT COUNTS, AI WHERE IT HELPS**
>
> Notice the division of labour. The dictionary and the grammar reference are generated **deterministically** — they are an exact, trustworthy description of your data. The study guide and the tutorial, where the value is in **teaching prose**, are AI-written. You always get an accurate reference; you optionally get a friendly teacher on top. Both kinds of book embed your font and look the same on the shelf.

Beyond the books

The books are not the only thing your language data can become. It can also leave Inkhaven entirely — as a flashcard deck, a LaTeX paper, or a translation-memory file for other tools — and other people's lexicons can come **in**. That whole subject, and the formats linguists and conlangers use, has its own chapter next (*Sharing your language*). What follows here is the one remaining thing the language data can produce on its own: creative text.

Creative text: names, verse, and ritual

For the fun of hearing the language speak, `compose` generates text grounded in what you have built:

```
inkhaven language compose Eldar --kind names --count 6
inkhaven language compose Eldar --kind prose --count 3
inkhaven language compose Eldar --kind poem  --meter 5,7,5
```

`names` draws phonotactic, capitalised names; `prose` assembles real grammatical sentences through the syntax engine (with glosses); `poem` writes metered verse that actually scans against your syllable counts. The themed kinds — `blessing`, `curse`, `incantation` — are AI-composed but **constrained to your existing lexicon**, so the model arranges real words rather than inventing them, and prints the native text with a gloss and a translation. Everything `compose` makes is printed for you to keep or discard; nothing is written into your books.

WHAT YOU LEARNED

- Four documents come from your language: **dictionary**, **grammar reference**, **study guide**, and **tutorial**.
- `--format md` is plain text; `--format typ` is a typeset book you compile with `typst compile --font-path …` .
- `--font Eldar` embeds your script; dictionary and grammar are **deterministic**.
- `grammar-book --study` and `tutorial` are **AI-written** teaching materials (need a provider).
- `compose` generates names, prose, and verse — grounded, deterministic, with AI-composed blessings/curses constrained to your lexicon.
- Moving the lexicon **out of** and **into** Inkhaven (export / import) has its own chapter next.

PART IX

Putting the Language to Work

CHAPTER 24

24

Translating with your language

By now your language has sounds, words, and a grammar. It can do something with all three: **translate**. Inkhaven ships a rule-based translation engine that runs entirely on what you built — it looks up your words, inflects them with your paradigms, and orders them by your typology — plus a growing **memory** of the sentences you approve, so the more you use it, the better it fits your language.

This is not a large language model guessing at Elvish. It is a deterministic machine reading your dictionary and your grammar. Where it has the words and the rules, it is right by construction; where it does not, it tells you what it is missing rather than inventing.

> **TERM Rule-based translation**
>
> Translation done by explicit rules and a lexicon rather than by a statistical model — the engine looks each word up, applies your inflection and agreement rules, and arranges the result by your declared word order. Because it uses **your** language's rules, its output is a direct consequence of the language you built, not an outside guess. Its limits are exactly your lexicon's limits: a word you have not coined cannot be translated, and the engine says so.

Translating into, and out of

The everyday command turns an English sentence into your language, glossed so you can check it:

```
inkhaven language translate Eldar --text "The king drew his
sword."
```

You get back the surface sentence in your language, an interlinear gloss, and a literal back-rendering — three views of the same translation, so you can see not just **what** it said but **how** it said it, and catch a wrong case or a missed agreement at a glance. To go the other way, from your language back into English:

```
inkhaven language reverse Eldar --text "aran makil mapta"
```

And if you have built more than one language, you can translate directly between two of your own, without English in the middle:

```
inkhaven language cross Eldar Khuz --text "aran makil mapta"
```

A memory that learns your voice

The engine is only as good as your rules — and no set of rules captures every turn of a real language. So the engine keeps a **translation memory**: when a translation comes out right, or you fix it into shape, you teach it back:

```
inkhaven language remember Eldar --source "The king drew his
sword." --target "aran makil mapta"
```

From then on the engine reuses that approved pair, and pairs like it, instead of rebuilding the sentence from scratch. Over a project's life this is where a language's **idiom** accumulates — the phrasings that are correct but no rule would have produced. See what it holds at any time:

```
inkhaven language memory Eldar
```

> TERM **Translation memory**
>
> A store of approved source–target sentence pairs the engine reuses. It is how the translator learns the phrasings your rules do not capture on their own — the fixed expressions, the idiomatic word choices, the sentences you corrected once and want kept. Each pair you `remember` makes the next similar translation truer to the language as you actually speak it.

Measuring the engine against your own texts

Your Sample-texts chapter — the passages you translated by hand as you built the language — is itself a **parallel corpus**: sentences in two languages, side by side. The engine can read it:

```
inkhaven language corpus Eldar
```

And, more usefully, it can **grade itself** against it — translating each source sentence and comparing to your hand-done target:

```
inkhaven language eval Eldar
```

The score tells you how much of your own corpus the engine reproduces, and, where it falls short, exactly which sentences it gets wrong. That is a to-do list in disguise: each failure points at a missing word, an unwritten rule, or a phrasing worth adding to the memory. As your language matures, watch the score climb.

Taking the memory with you

The translation memory is portable. Export it as a self-contained `.itm` pack — for a backup, for a collaborator, or to move it between projects:

```
inkhaven language export-translation Eldar -o eldar.itm
```

Everything the engine learned travels in that one file, so a language you translate in is never locked to a single machine.

WHAT YOU LEARNED

- Inkhaven translates with a **rule-based engine** over your phonology, lexicon, and grammar — `translate` (into your language), `reverse` (back to English), and `cross` (between two of your own). Every result comes glossed.
- The engine keeps a **translation memory**: `remember` an approved sentence pair and it is reused thereafter — this is where the language's idiom accumulates beyond what any rule captures. `memory` shows what it holds.
- Your Sample-texts are a **parallel corpus**: `corpus` reads it, `eval` grades the engine against it — and each failure is a pointer at a missing word or rule.
- `export-translation` writes an `.itm` pack, so the memory travels with the language.

PART X

Sharing Your Language

CHAPTER 25

25

Sharing your language

Your language does not have to live only inside Inkhaven. You may want to typeset a paper about it, study it on your phone, hand the lexicon to a collaborator, publish a dictionary website, or bring in a language you started building in another program. This chapter is about moving a lexicon **out of** and **into** Inkhaven, and about the file formats and tools the wider community of linguists and conlangers actually uses — so that whichever one you meet, you know what it is and how to bridge it.

There are two directions, and one command each. **Export** writes your lexicon to a file in some other format; **import** reads a foreign file back into your Dictionary. Everything here works on the lexicon you have already built with `add-word` and `generate-lexicon`; nothing new about the language itself is required.

The landscape: who uses what

Before the commands, it helps to know the formats by name. Each of the following is a real tool or standard you may run into when you look for conlang resources or linguistic software. The term boxes define them once; later sections show how Inkhaven reads or writes each.

> TERM **Standard Format (SFM / MDF)**
>
> A plain-text dictionary format where every field is tagged with a backslash marker — `\lx` for the headword (*lexeme*), `\ps` for part of speech, `\ge` for the English gloss, `\xv` for an example, and so on. **MDF** (Multi-Dictionary Formatter) is the standard set of those markers. It is decades old, plain text, and the common tongue of descriptive lexicography: many tools read and write it.

> TERM **SIL Toolbox / FieldWorks**
>
> Field-linguistics software from SIL International used to record and analyse real languages in the field. Both read and write **Standard Format**, so a lexicon exported from Toolbox is an SFM file — exactly what Inkhaven's Toolbox importer expects.

> TERM **Lexique Pro**
>
> A free program that turns a **Standard Format** dictionary into a browsable, hyperlinked dictionary application and website. Because its source format **is** SFM, a Lexique Pro database imports into Inkhaven the same way a Toolbox one does.

> TERM **PolyGlot**
>
> A popular free desktop application built specifically for constructed languages, with a lexicon, a grammar, and a conjugation engine. It saves to a `.pgd` file, which is really a *ZIP* archive containing an XML dictionary — Inkhaven unzips and reads it for you.

TERM **ConWorkShop (CWS)**

A large online community and toolkit for conlangers, with shared dictionaries and sound-change tools. Its lexicon export is a spreadsheet (CSV), which comes into Inkhaven through the CSV import path.

TERM **CAT tool / translation memory**

Computer-assisted-translation software (OmegaT, memoQ, Trados, the web tool Weblate) that helps translate text by remembering past translations. The remembered pairs live in a *translation memory* the standard interchange file for one is **XLIFF**.

TERM **XLIFF**

XML Localisation Interchange File Format — the OASIS standard a **CAT tool** reads. Each entry is a *translation unit* pairing a source-language phrase with its target-language equivalent. Exporting your lexicon as XLIFF turns it into a translation memory between your working language and your conlang.

TERM **Anki**

A widely used spaced-repetition flashcard program. It imports decks from a simple comma-separated file, so an Anki export of your lexicon becomes a vocabulary deck you can drill on your phone.

TERM **linguex**

A LaTeX package linguists use to typeset numbered, glossed example sentences in papers and grammars. An export in this format is a ready-to-compile LaTeX document of your lexicon, headwords and examples included.

Exporting your lexicon

The single command is `export`, with a `--format` choosing what to write. By default it prints to your screen; add `-o FILE` (the long form is `--output`; the older `--out` still works) or redirect with `>` to save:

```
inkhaven language export Eldar --format xliff   -o eldar.xlf
inkhaven language export Eldar --format anki     > eldar-deck.csv
inkhaven language export Eldar --format ipa-chart
```

The formats, and when to reach for each:

`json` A complete structured dump — dictionary, grammar, phonology, and sample texts — for your own scripts or an archival backup. (It is not one of the formats `import` reads back; for a round trip through Inkhaven, use `csv`.)

`csv` A spreadsheet of your entries, twelve columns, and the **only** format that comes back in through Inkhaven's import path. Use it to edit your lexicon in a spreadsheet and bring it back (see the round trip below), and as the bridge for tools like ConWorkShop.

`anki` A flashcard deck (word, translation, type, example) importable into **Anki** and similar spaced-repetition apps.

`xliff` A **translation memory** (each entry a working-language → conlang pair) loadable into a **CAT tool** such as OmegaT, memoQ, or Weblate — useful if you actually translate text into your language.

`linguex` A LaTeX document using the **linguex** package — bold headwords with glosses and numbered examples — to paste into an academic paper or a hand-written grammar.

`ipa-chart` A Markdown inventory of your phonemes, consonants and vowels grouped, each with its romanization — a quick appendix or forum post.

`dictionary-twocol` / **`grammar`** / **`phrasebook`** Typeset **Typst** documents (a printable two-column dictionary, a grammar reference, a phrasebook). These need `-o FILE.typ`, then you compile them just like the books in the previous chapter.

To make the two least-familiar formats concrete: a single XLIFF entry pairs the meaning with the word, and a linguex headword carries its gloss and a numbered example —

```
<trans-unit id="1"><source>bird</source><target>kira</target></
trans-unit>
```

```
\textbf{kira} \textit{noun} `bird'\\
\ex. kira nami
```

WHICH EXPORT SHOULD I USE?

If you want to **read it back into Inkhaven later**, use `csv` (the one round-trippable format). If you want a **backup** or to feed your own scripts, use `json`. To **study** the words, use `anki`. To **typeset** them, use `linguex` (a paper) or the Typst formats (a finished book). To carry your lexicon into an **external CAT tool** (OmegaT, memoQ, Weblate), use `xliff`. The IPA chart is for a quick reference.

TERM Translating without leaving Inkhaven

XLIFF is for handing your words to **another** program. You do not need to leave Inkhaven to translate at all: the previous chapter's built-in engine (`language translate` / `reverse` / `cross`) turns sentences into and out of your language using its own lexicon and grammar, and keeps its own **translation memory** — exported as an `.itm` pack, not XLIFF. Reach for XLIFF only when the translating happens in a tool other than Inkhaven.

Importing a lexicon

Going the other way, `import` reads a foreign file and adds its entries to your Dictionary. It has one safety habit worth knowing: it **previews by default** and writes nothing until you add `--yes`. So you always see what it found first:

```
inkhaven language import Eldar --file lexicon.sfm --format
toolbox        # preview
inkhaven language import Eldar --file lexicon.sfm --format
toolbox --yes  # write
inkhaven language import Eldar --file MyLang.pgd  --format
polyglot --yes
```

Two foreign formats are read directly:

`toolbox` A **Standard Format** (SFM / MDF) database — the `\lx … \ps … \ge …` marker file written by **SIL Toolbox**, **FieldWorks**, and **Lexique Pro**. Inkhaven maps the standard markers onto its own fields (headword, part of speech, gloss, example, pronunciation, etymology, notes) and folds multi-line fields together.

`polyglot` A **PolyGlot** dictionary. Pass either the native `.pgd` archive — Inkhaven unzips the dictionary XML inside it automatically — or a raw exported `.xml`. The part-of-speech table is resolved so each word arrives with its word class.

A lexicon exported from **ConWorkShop** (or any tool that produces a spreadsheet) comes in through the CSV path instead, which lives on the `add-word` command:

```
inkhaven language add-word Eldar --import cws-export.csv
```

> **IMPORTED WORDS STILL FACE THE RULES**
>
> Importing does not bypass your language. The CSV path runs the same phonotactic pre-flight as a hand-typed word, so a row whose spelling uses a letter or sound your language has not declared is flagged before anything is written (use `--force` only when you mean to). And every importer refuses to overwrite a word you already defined — duplicates are skipped with a warning, so a re-import never clobbers your edits.

A round-trip workflow

The CSV format is built to make a loop: export your lexicon, edit it somewhere comfortable, and bring it back. A common pattern is bulk-editing in a spreadsheet:

1. `inkhaven language export Eldar --format csv -o eldar.csv`
2. Open `eldar.csv` in a spreadsheet; fix translations, add examples, tidy registers across many rows at once.
3. `inkhaven language add-word Eldar --import eldar.csv` — new rows are added, rows you already have are skipped, so your edits merge cleanly.

The same loop lets two people collaborate (one exports, the other imports), or lets you move a language between machines without copying the whole project.

WHAT TRAVELS, AND WHAT DOES NOT

Interchange moves the **lexicon** — words, glosses, parts of speech, examples, and the fields a row carries. It does not move your phonology rules, paradigms, sound-change chains, or font: those are Inkhaven-specific structure with no equivalent in a Toolbox or PolyGlot file. To move a whole **project**, copy the project folder; to move just the **words**, use interchange.

WHAT YOU LEARNED

- **Export** writes your lexicon out (`export --format …`); **import** reads a foreign lexicon in (`import --format … [--yes]`).
- **Standard Format** (SFM/MDF) is the shared text format of **Toolbox**, **FieldWorks**, and **Lexique Pro**; import it with `--format toolbox`.
- **PolyGlot** `.pgd` files (zip + XML) import with `--format polyglot`; **ConWorkShop** and other spreadsheet exports come in via `add-word --import`.
- **XLIFF** feeds **CAT tools**, **anki** feeds flashcards, **linguex** feeds LaTeX, and the Typst formats produce finished books.
- Import **previews until** `--yes`, re-runs the phonotactic check, and never overwrites an existing word; `csv` makes a clean export-edit-import **round trip**.
- Interchange moves **words**, not phonology / paradigms / fonts — copy the project folder to move everything.

PART XI

A Complete Walkthrough

CHAPTER 26

26

A complete walkthrough

This chapter builds a whole small language — **Avesha** — from nothing to finished books, in one unbroken sequence. Follow along command by command and you will practise every pillar of the book at once. Everything here uses what the earlier chapters explained; refer back whenever a step needs more detail.

> **THERE IS A SCRIPT FOR THIS**
>
> Inkhaven ships a runnable script that performs this entire walkthrough automatically — `examples/conlang/build-sample-language.sh`, with a checked-in sample of its output in `examples/conlang/sample-output/`. Read this chapter to understand each step; run the script to watch it happen.

1. Project and language

```
inkhaven init ~/avesha-project
cd ~/avesha-project
inkhaven language init Avesha
```

2. Phonology

Create a file in Avesha's `04-phonology` folder with the sound system — eleven sounds, syllable shapes, a couple of constraints, two allophony rules, and penultimate stress:

```
{
  phonemes: [
    { ipa: "p", kind: "consonant" } { ipa: "t", kind:
"consonant" }
    { ipa: "k", kind: "consonant" } { ipa: "s", kind:
"consonant" }
    { ipa: "m", kind: "consonant" } { ipa: "n", kind:
"consonant" }
    { ipa: "l", kind: "consonant" } { ipa: "r", kind:
"consonant" }
    { ipa: "a", kind: "vowel" } { ipa: "i", kind: "vowel" }
{ ipa: "u", kind: "vowel" }
  ]
  classes: { C: ["p","t","k","s","m","n","l","r"], V:
["a","i","u"] }
  templates: { root: [ { pattern: "C V" }, { pattern: "C V
C" } ] }
  constraints: [ { kind: "no_geminate" }, { kind:
"max_cluster_size", value: 2 } ]
```

```
  allophony: [ { rule: "t > s / _ i" }, { rule: "n > m / _ p" } ]
  stress: "penultimate"
}
```

Then adopt it and check:

```
inkhaven reindex --adopt
inkhaven language stats Avesha
```

3. A vocabulary

```
inkhaven language add-word Avesha pata --type noun --translation
stone
inkhaven language add-word Avesha kira --type noun --translation
bird
inkhaven language add-word Avesha suna --type noun --translation
sun
inkhaven language add-word Avesha nami --type verb --translation
see
inkhaven language add-word Avesha palu --type verb --translation
run
inkhaven language add-word Avesha mira --type adjective --
translation bright
inkhaven language audit Avesha
```

4. Grammar

Add a morphology block to the `03-grammar` folder — a dative and a plural suffix, and an agent-noun derivation:

```
{
  morphemes: [
    { id: "dat", gloss: "DAT", form: "ti", position: "suffix" }
    { id: "pl",  gloss: "PL",  form: "u",  position: "suffix" }
  ]
  derivations: [
    { name: "agent", form: "ar", position: "suffix",
      from_pos: "verb", to_pos: "noun", gloss_template: "one who
{}s" }
```

```
  ]
}
```

Record the typological choices and reindex:

```
inkhaven reindex --adopt
inkhaven language grammar Avesha --set word_order=sov
inkhaven language grammar Avesha --set
alignment=nominative_accusative
inkhaven language grammar Avesha --set case=yes
```

Watch the allophony fire at an affix boundary — **pata** plus the dative **-ti**:

```
inkhaven language paradigm Avesha --root pata --gloss stone
```

The dative form comes out **patasi**: the /t/ of the root softened to /s/ before the /i/ of the suffix, by the rule you wrote in step 2.

5. A writing system

Draw or AI-draft one glyph per phoneme, bind each to its sound and a Private Use codepoint, then compile the font. With AI drafting:

```
inkhaven language glyph-draft Avesha --describe "a bold vertical
post" \
    --phoneme p --codepoint U+E000 --name p --provider deepseek
--yes
# … one per phoneme …
inkhaven language font-config Avesha
inkhaven language font-build --language Avesha --format ttf -o
Avesha
```

Type a word in the new script:

```
inkhaven language transliterate Avesha --text kira
```

6. The books

```
inkhaven language dictionary   Avesha --format typ --font Avesha
-o dict.typ
inkhaven language grammar-book Avesha --format typ --font Avesha
--study \
    --provider deepseek -o grammar.typ
inkhaven language tutorial     Avesha --format typ --font Avesha
\
    --provider deepseek -o learn.typ
```

Compile any of them to a PDF that embeds your font:

```
typst compile --font-path . dict.typ dict.pdf
```

You have built a language

Look back over what you did. You chose a sound system and tuned it by ear. You coined words and kept them consistent. You built grammar that inflects words and coins new ones, with sound changes firing automatically at the seams. You set the language's typological character. You designed an alphabet and compiled it into a real font. And you produced a dictionary, a grammar, and a textbook — printable books in your language's own script.

That is a complete constructed language, made from nothing. From here you can go as deep as you like: hundreds more words, a proto-language and a whole family (Part V), idioms and metaphors, a richer script. The tools are the same ones you have now used; only the scale changes.

> **WHAT YOU LEARNED**
>
> - The whole journey is six steps: project, phonology, vocabulary, grammar, writing system, books.
> - Each step uses commands from earlier chapters; the allophony, dedup gate, and font compiler all work together automatically.
> - The shipped `build-sample-language.sh` runs this end to end.
> - You now have everything you need to build your own language from zero.

PART XII

Scripting Your Language

CHAPTER 27

27

Building your conlang with Bund

Everything you have built so far, you built by **describing** it. You wrote blocks of HJSON — a list of phonemes, a set of templates, a table of affixes — and Inkhaven read them. That is the **declarative** way: you state what is true, and the program works out the rest. It is clear, it is durable, and for most languages it is all you will ever need.

But there is a second way in, and it is worth knowing about even if you never reach for it. Inkhaven carries a small programming language of its own, called **Bund**, and through it you can **build** a language step by step — invent a hundred words in a loop, inspect what you have and act on it, run the whole pipeline from empty project to finished dictionary in one recipe. This chapter is a gentle look at

that programmatic way: what it is, why it is sometimes more powerful than a stack of HJSON files, and enough of the shape of it to read a script without fear.

> **YOU DO NOT NEED TO BE A PROGRAMMER**
>
> If the word "script" makes you uneasy, relax: nothing in this book requires it, and your language is no less real for being hand-written. Think of this chapter as a tour of a workshop's power tools. You can admire them, try one, and still do most of your work by hand.

Two ways to say the same thing

Declarative and programmatic are not rivals; they are two doors into the same room. When a script defines a phoneme block, it writes **exactly** the HJSON a hand-author would have typed — byte for byte the same paragraph in the same chapter. A language built by script opens, inspects, and prints identically to one built by hand, and you can mix the two freely: hand-write the phonology, script the thousand-word lexicon. Nothing is lost either way.

> TERM **Declarative**
>
> Describing **what** you want as data and letting the program act on it — the HJSON blocks of this book. You say "these are my phonemes"; you do not say how to read them.

> TERM **Programmatic**
>
> Giving an ordered list of **instructions** — do this, then this, then this. A program can repeat, decide, and compute, which inert data cannot.

What Bund is

> TERM **Bund**
>
> A small **stack-based** programming language built into Inkhaven. Its conlang vocabulary — the words beginning `ink.lang.` (or the short `lang.`) — reaches every part of the ConLang Suite: the same generators, validators, and renderers the `inkhaven language` commands use.

Bund reads a little unusually at first, because the value comes **before** the action. Where English says "add one and two", Bund says `1 2 add` — push the one, push the two, then run `add`, which takes the two numbers and leaves their sum. Everything works this way: you lay values down, and a **word** picks them up.

> TERM **Stack**
>
> A pile of values, like a stack of plates: the last one put down is the first taken up. Bund words take their inputs from the top of the stack and leave their results there. Reading a script is just following what is on the pile.

> TERM **Word (in Bund)**
>
> One instruction — a named operation. `lang.add_word` is a word; so is `lang.ipa`. Each word's inputs and outputs are written in a **stack-effect** comment like `( lang word pos translation -- )`: the names left of the `--` are taken from the stack, those on the right are left behind.

So when you read, in the reference of Appendix C, that `lang.ipa` has the effect `( lang word -- ipa-surface-string )`, it means: give it a language name and a word, and it hands back that word's pronunciation. You would write `"Eldar" "tap" lang.ipa` — the two values, then the word that consumes them.

Why a script can do more than a file

If the two are equivalent, why ever script? Because a program can do three things a static file cannot.

It can repeat
A file lists each word by hand. A script writes a **loop**: generate fifty word-shapes, derive every cell of a paradigm, evolve a whole proto-dictionary into a daughter — all from one short instruction, however many items there are. The tedium that makes a large language daunting to hand-author simply vanishes.

It can decide
A file cannot look at itself. A script can: it can run the `audit`, read the `gaps` report, check the `stats`, and **act on what it finds** — add a word only if it would not collide, coin only the basic concepts still missing, accept a generated form only if it passes the phonotactics. The language becomes self-checking.

It is a recipe, not just a result
A hand-built language is a **result**; a script is the **recipe** that produces it. Keep the recipe and you can re-run it to rebuild the language from nothing, change one sound rule at the top and regenerate everything that flows from it, or hand the whole recipe to someone else who can reproduce your language exactly. This is the deepest difference: declarative data captures **what your language is**; a program captures **how it came to be**.

A whole language in one breath

Here is a complete small language — sounds, a grammar setting, three words — and then a sentence built from them, start to finish in a single script:

```
"Avesha" lang.init
"Avesha" "Phonology" "{ phonemes:[{ipa:\"k\",kind:\"consonant\"}
{ipa:\"a\",kind:\"vowel\"}]
   classes:{C:[\"k\"] V:[\"a\"]} templates:{root:[{pattern:\"C V
C V\"}]} }" lang.define
"Avesha" "Grammar" "{ grammar:{word_order:\"sov\",alignment:
\"nominative_accusative\"} }" lang.define
"Avesha" "kira" "noun" "bird"  lang.add_word
"Avesha" "nami" "verb" "see"   lang.add_word
"Avesha" "pata" "noun" "stone" lang.add_word
"Avesha" "kira:bird" "nami:see" "pata:stone" lang.sentence
println
```

`lang.init` makes the language and its chapters; each `lang.define` writes one HJSON block into a chapter (the `\"` marks are just quotes carried safely inside a quoted string); `lang.add_word` adds dictionary entries; and `lang.sentence` assembles a subject–verb–object clause and `println` prints it. Open **Avesha**

afterwards with `inkhaven language …` and it is indistinguishable from one you typed by hand — because it **is** the same data.

Three kinds of word, and a safety gate

The conlang vocabulary comes in three flavours, and Inkhaven guards the last two so a script can never surprise you:

> TERM **Inspectors, mutators, AI words**
>
> **Inspectors** only read — `lang.ipa`, `lang.stats`, `lang.sentence` — and are always allowed. **Mutators** change your project — `lang.init`, `lang.add_word` — and **AI words** call a language model — `lang.compose`, `lang.generate_lexicon`. These last two are switched **off** until you opt in.

You enable the guarded categories in `inkhaven.hjson`, naming exactly what a script may do:

```
scripting: { enabled_categories: ["store_write", "ai_write"] }
```

> ADVISORY, LIKE EVERYTHING AI IN INKHAVEN
>
> The AI words follow the same rule as the rest of the suite: they **return** suggestions and never write your book themselves. `lang.generate_lexicon` hands back a list of proposed words; your script decides which to keep and commits them with `lang.add_word`. Nothing reaches the page without an instruction you wrote.

Building artefacts the native way

Writing HJSON inside quoted strings (with all those `\"` marks) is the simplest path for whole blocks, but Bund can also build the data structures directly. Because Bund's curly braces mean something else, a small constructor turns a flat list into a dictionary, and it answers to friendly names so a script reads like what it makes:

```
[ "ipa" "k" "kind" "consonant" ] phoneme
```

`phoneme` here is the dictionary constructor (it is the same word as `rule`, `block`, and `word` — aliases you can read as labels). Hand the result to `lang.define` and it is serialised to the same HJSON as before. Most authors reach for this only when generating blocks in a loop; for a fixed block, the string form is plainer.

Running a script

Two ways to run Bund. From the command line, pass the script and point at your project:

```
inkhaven bund "\"Eldar\" \"tap\" lang.ipa println" --project .
```

Or, inside the editor, write the script into a **Script** book and run it there — handy for a refresh routine you re-run as the language grows. Appendix C lists every conlang word, its stack effect, and its safety category, so you can assemble recipes of your own.

WHAT YOU LEARNED

- Inkhaven offers two equivalent ways to build a language: **declarative** HJSON (what you used all book) and **programmatic Bund** scripts. A script writes the same HJSON, so the two mix freely.
- Bund is **stack-based**: values come before the action (`1 2 add`), and each **word** takes inputs from the stack and leaves outputs — written `( in -- out )`.
- A script beats a file when there is **repetition** (loops over words or paradigms), **decision** (inspect, then act), or a need for a **reproducible recipe** rather than a fixed result.
- Words come in three kinds — **inspectors** (always on), **mutators** and **AI words** (opt in via `scripting.enabled_categories`); AI words stay advisory.
- Run a script with `inkhaven bund "…" --project .` or from a Script book; the full word list is Appendix C.

PART XIII

Analysing Your Language

CHAPTER 28

28

The Inner Linguist

Everything so far has been a command you run and read the output of. That is a fine way to work, but a language is a large thing to hold in your head one command at a time. Inkhaven gives you a second way in: a full-screen workspace that opens over all of your languages at once, lets you walk their books, ask questions about them, and read the analyses of the following chapter side by side. It is called the **Linguistic companion**, and the assistant inside it is the **Inner Linguist**.

> **TERM The Linguistic companion**
>
> A full-screen terminal application, sibling to the Research companion, that opens over your project's `Language` book. The tree of your languages sits on the left; a set of views — a preview, a grounded chat, and the phonology, typology and minimal-pair analyses — sit on the right. It reads your languages; it does not edit them.

Opening it

From a project directory:

```
inkhaven linguistic
```

The screen splits in two. On the left is the tree of your `Language` book: every language you have created, and — when you expand one — its five chapters and their contents. Move with the arrow keys or `j` / `k`; open a branch with `l` or `→`, close it with `h` or `←`. To open focused on one language, name it:

```
inkhaven linguistic --language Eldar
```

The views

The right pane shows one **view** at a time. Press `Tab` to cycle through them, or jump straight to one with its letter:

Chat a conversation with the Inner Linguist (press `i` to ask).
Preview the content of whatever node you have selected in the tree.
Phonology (`m`) the quantitative metrics and the inventory-naturalness report of the next chapter.
Universals (`u`) how your grammar sits against the typological baseline.
Minimal pairs (`p`) the contrasts your lexicon draws, and their load.

Whichever language your selection sits inside is the one every view describes; move the cursor to another language and the analyses recompute for it. The title bar always tells you which language you are looking at.

The grounded chat

Press `i` and a prompt opens at the bottom of the Chat view. Ask anything about the selected language — "is this vowel system typologically ordinary?", "suggest a plausible plural suffix", "what would strengthen the consonant inventory?" — and the Inner Linguist answers.

What makes the answer useful is that it is **grounded**. Before it replies, the companion retrieves the most relevant passages from the very language sub-book you are in — your phonology, your dictionary, your grammar — and hands them to the model as the material to reason from. So it is talking about **your** language, not languages in general, and it is told not to invent forms or rules your book does not support. It answers in your project's language.

> THE SAME RETRIEVAL AS RESEARCH
>
> The grounding is the identical book-aware retrieval the Research companion uses over the Facts book — here pointed at the language sub-book your cursor is in. If an answer seems thin, it usually means the relevant chapter is thin: define more of the language and the assistant has more to stand on.

Sessions

Your conversation is a **session**, and it persists. Every completed exchange is saved under the project's `.inkhaven/` directory and replayed the next time you open the companion, so you can put a line of enquiry down and pick it up days later. Keep separate lines of work in separate named sessions:

```
inkhaven linguistic --language Eldar --session phonology-review
```

Without `--session` you get the `default` session. A session is just the chat transcript; nothing about your language is changed by talking about it.

Slash-commands in the chat

The chat input does more than ask questions. A line that begins with a slash is run **locally** — the companion answers it from your own language book instead

of sending it to the assistant, and prints the result inline. None of them change anything; each is a look, not a leap. There are four.

`/trace` followed by a sound-change rule previews that change across your whole lexicon — which words would shift, and whether any would collide into new homophones. It is the Consequence Tracer of the next chapter, at your fingertips while you think:

```
/trace s > ʃ / _ i
```

`/parse <word>` runs the morphological parser **backwards**: it strips your language's known affixes from a surface form and reports every analysis that bottoms out at a dictionary root — including the non-concatenative processes: reduplication, both full (`kata-kata`) and partial (`ka~kata`), and ablaut, the internal vowel change of **sing/sang** (recovered by running each root forward through the ablaut rules and matching). Where the gloss path builds a word from a root and features, `/parse` takes a word you have written and tells you how it could be built:

```
/parse katakatai
```

`/check <word>` is the Oracle: it judges a candidate word for well-formedness, level by level — first its phonotactics (are the segments and clusters legal?), then its morphology (does it parse to a root, or is it listed?) — and reports either a clean bill of health or the specific findings against it.

`/tree <verb> <subject> [object] [indirect]` builds the X-bar phrase-structure tree of a clause, placing heads and complements according to your language's declared word order, and prints both the indented tree and its bracketed notation:

```
/tree sees she bird
```

`/clause <verb> <subject> [object] [indirect]` runs the Oracle over a whole clause rather than a single word, checking its argument structure against the verb's valence — an intransitive verb given two arguments is flagged, a transitive one given two is not. (Checking that the verb **agrees** with its subject needs the subject's grammatical features, so that half lives on the command line, as `language check-clause`.)

`/igt <sentence>` glosses a whole sentence of your language as **interlinear glossed text** — the morpheme-segmented sentence (`kata-i`), a line glossing each

morpheme (`stone-PL`), and a literal translation, aligned in columns. An affix reshaped at its boundary is still split correctly (a dative **kata + d** surfacing as **katat** is segmented `kata-t`):

```
/igt katai nilo
```

On the command line the same gloss can be **kept**: `inkhaven language igt <lang>` stores the interlinear in a `Texts` chapter `--text "…" --save --name <n>` of the language book, and `inkhaven language texts <lang>` lists what you have gathered — the beginning of a documented corpus of your language. Add `--format latex` to export those texts as a `linguex` document, ready to paste into a grammar sketch or a paper.

Back in the chat, `/texts` lists your stored interlinears, and `/settrans <name> = <translation>` replaces one's free translation — the auto-generated literal line is only a scaffold, and this is where you write the sentence as it should read. The gloss underneath is left exactly as it was.

We meet the tracer, the parser, the Oracle and the syntax engine properly — and the analyses the other views show — in the next chapter.

WHAT YOU LEARNED

- The **Linguistic companion** (`inkhaven linguistic`) is a full-screen workspace over your `Language` book — the tree on the left, a set of views on the right.
- `Tab` cycles the views; `i` asks the Inner Linguist, `m` opens Phonology, `u` Universals, `p` Minimal pairs. Every view describes the language your cursor sits inside.
- The chat is **grounded**: answers are retrieved from your own language book, so the Inner Linguist reasons about your language and answers in your project's language.
- Conversations are saved **sessions** (`--session <name>`), replayed on re-open; talking about a language never changes it.
- Slash-commands run **locally** over your book and print inline: `/trace <rule>` previews a sound change, `/parse <word>` analyses a surface form into root + affixes, `/check <word>` is the word Oracle's verdict, `/tree <verb> <subject>` builds the clause's X-bar tree, `/clause <verb> <subject> …` is the clause Oracle's argument-structure check, `/igt <sentence>` glosses a sentence, and `/texts` lists your stored interlinears. All are read-only except `/settrans <name> = <translation>`, which curates a stored text's free translation.

CHAPTER 29

29

Measuring, testing, and tracing

You have built a language. How do you know whether it is any good — whether its sounds hang together the way real languages' do, whether its grammar is internally consistent, whether a change you are considering will do what you hope? Inkhaven answers these with a set of read-only **analyses**. None of them touches your language; each holds it up against how the world's languages actually behave and tells you where yours is ordinary and where it is unusual. A flag is never an error — real languages break every tendency — it is simply a place to look.

The shape of the sound system

`stats` (Chapter 10) counts your phonemes and syllables. `metrics` measures them.

```
inkhaven language metrics Eldar
```

> TERM **Phoneme entropy**
>
> A number, in **bits**, for how evenly your phonemes are used across the lexicon. A language that leans hard on a handful of sounds has low entropy; one that spreads its inventory evenly has high entropy. Alongside it, **evenness** rescales this to 0–100%, and **perplexity** reports the effective inventory size — how many equally-frequent phonemes would give the same spread.

The report also fits your phoneme frequencies to **Zipf's law** — the tendency, seen across natural languages, for the ^{n}th most common item to appear about `1/n` as often as the most common; a slope near `−1` is Zipfian. **Phonotactic saturation** asks how much of the syllable space your phonotactics allow you actually use (a low number means many possible syllables go unfilled), and **mora weight** reports the average heaviness of your syllables. All of it is computed from the dictionary words that parse cleanly as your own phonemes; loans and names are skipped.

Is the inventory natural?

`naturalness` judges the phoneme inventory itself against cross-linguistic tendencies, using a table of distinctive features — the place, manner and voicing that distinguish one sound from another.

```
inkhaven language naturalness Eldar
```

It checks four things. **Voicing symmetry**: does every voiceless obstruent have a voiced partner, or is there a `/k/`-without-`/g/` gap? **Place coverage**: are the major places — labial, coronal, dorsal — all represented? **Near-universals**: does the inventory include the segments almost every language has (`/m n p t k s a i u/` …)? And **size**: is the inventory in the ordinary range, or notably small or large?

These fold into a single 0–1 score. A gap here is a design signal, not a mistake — plenty of natural languages lack `/p/` or `/g/` — but if the tool reports you are missing `/a/`, that is worth a second look.

> **TERM Distinctive feature**
>
> A single dimension along which two sounds can differ — voicing, place of articulation, nasality, vowel height, and so on. `/p/` and `/b/` differ in one feature (voicing); `/p/` and `/t/` in place. Describing sounds by their features is how a computer — or a linguist — reasons about whether two of them contrast, and whether a class of them behaves alike.

Which contrasts carry the weight?

Two words that differ in a single sound — **pat** and **bat** — are a **minimal pair**, and they are the classic proof that the two sounds are distinct phonemes in your language rather than variants of one. `pairs` finds every minimal pair in your lexicon and, through the feature table, reports the one feature each turns on:

```
inkhaven language pairs Eldar
```

The result is a picture of **functional load** — which distinctions your language actually leans on. If a hundred pairs turn on voicing and only two on aspiration, you have learned something about where your language's contrasts do their work, and perhaps where a distinction you thought important is barely paying its way.

Does the grammar obey the universals?

Chapter 13 had you answer a typological questionnaire. `universals` checks those answers against how real languages combine them.

```
inkhaven language universals Eldar
```

> **TERM Implicational universal**
>
> A statement of the form "if a language has X, it (almost always) has Y" — Greenberg's classic findings. Verb-final languages tend to have postpositions, not prepositions; prepositional languages tend to put the possessor after the noun. These are strong cross-linguistic **tendencies**, not laws, and a natural-seeming language mostly follows them.

The report does two things. It measures **head-directionality harmony** — whether your adpositions, genitives and relative clauses all "branch" the same way as your verb phrase, which harmonious languages do — and it judges a handful of the classic implicational universals (Greenberg's 2, 3 and 4, and the correlations between object–verb order and genitive and relative-clause order), marking each **satisfied**, **violated**, or **not applicable**. A violation is a flag: your language has made a combination that is rare in the world. That may be exactly the character you want — but now you have chosen it on purpose.

Tracing a change before you make it

The most consequential thing you can do to a language is change its sounds. A single rule — palatalize `/s/` before `/i/`, drop final vowels — ripples through every word at once, and some of those ripples you will not have foreseen. The **Consequence Tracer** lets you look before you leap.

```
inkhaven language trace Eldar --rule "s > ʃ / _ i"
```

> **TERM Consequence Tracer**
>
> A preview of a pending sound change. It applies the rule to a copy of your whole lexicon, re-derives every word exactly as a real sound change would, and reports what would happen — without changing anything. Nothing is written; it is a rehearsal.

It tells you two things. First, **which words the change touches**, old form beside new. Second, and more valuable, whether the change would create **new homophones** — words that were distinct but which the rule collapses onto the same form. A merger like that is sometimes what you want (languages merge sounds all

the time) and sometimes a quiet disaster you would rather catch now than after you have written a chapter of prose in the language. Try a rule, read the consequences, and keep it or discard it with nothing lost.

You can run the same trace inside the Linguistic companion — type `/trace s > ʃ / _ i` into the chat — so the rehearsal is always a keystroke away while you work.

A corpus of your own texts

Once you have gathered a few interlinear texts (Chapter 32) — or simply written prose in the language — they become a small **corpus**, a body of real usage you can measure, not just a dictionary of isolated words. By default the corpus is **both** your stored interlinears and the conlang words Inkhaven finds in your manuscript, so it reflects the language as you actually use it; `--source texts` or `--source prose` narrows it to one. `inkhaven language frequency <lang>` counts how often each word occurs and reports the statistics that only usage can give: how many word **tokens** and distinct **types** you have written, their ratio, and how closely the frequencies follow the **Zipf** law that natural languages obey. Add `--lemma` to count a root's inflected forms together.

> TERM **Type–token ratio**
>
> The number of distinct words (**types**) divided by the total number of words (**tokens**) in a text. A high ratio means little repetition (a dense, varied vocabulary); a low one means the same words recur. It is the standard first measure of lexical variety.

And `inkhaven language concordance <lang> --word W` shows every place a word appears across your texts, each occurrence lined up with its context on either side — a **KWIC** (keyword-in-context) concordance, the tool a lexicographer reaches for to see how a word is actually used before writing its definition. `--lemma` finds every inflected form of a root at once.

`inkhaven language collocations <lang> --word W` goes one step further and asks which words a word keeps **company** with — its **collocates**. It counts the neighbours that fall within a few words of the target across your texts and

ranks them not just by raw count but by how **distinctive** the pairing is, so a word that genuinely belongs with the target rises above one that is merely common everywhere.

> **TERM Collocation**
>
> A habitual pairing of words — **strong tea**, not **powerful tea**; **make a decision**, not **do a decision**. Collocations are much of what makes a language idiomatic, and a corpus is the only way to discover them, since they live in usage rather than in any single definition.

All three are in the companion too, as `/frequency`, `/kwic <word>` and `/coll <word>`.

> **WHAT YOU LEARNED**
>
> - The analyses are **read-only**: they describe your language against the cross-linguistic baseline; a flag is a place to look, never an error.
> - `metrics` measures the sound system — phoneme **entropy** (and evenness, perplexity), the **Zipf** fit, **phonotactic saturation**, and **mora weight**.
> - `naturalness` judges the inventory — voicing symmetry, place coverage, near-universal segments, and size — into a 0–1 score.
> - `pairs` finds **minimal pairs** and, via the distinctive-feature table, reports the **functional load** of each contrast.
> - `universals` checks your grammar's **head-directionality harmony** and the classic **implicational universals** (Greenberg/Dryer), flagging marked combinations.
> - `trace` (the **Consequence Tracer**) previews a sound change across the whole lexicon — which words shift, and which new **homophones** it would create — without committing it; `/trace <rule>` does the same inside the companion.

CHAPTER 30

30

Syntax: parsing and structure

Everything up to now has run one way: you declared sounds, morphemes and rules, and Inkhaven **generated** forms from them — a root plus a paradigm became an inflected word, a proto-form became a daughter. This chapter runs the other way. Given a word or a sentence your language could produce, it takes the thing **apart**: peels a surface word back to its root, works out who did what to whom, and draws the tree the sentence hangs on. These are the tools of a working syntactician, pointed at the language you built.

None of them change your language. They read it and report, so you can check that the grammar you declared actually produces the structures you intended.

Reading a word backwards: the parser

Paradigm generation goes root → surface. The **parser** goes surface → root: given a word, it strips the affixes your morphology declares until what remains is an entry in the lexicon.

```
inkhaven language parse Eldar --word katai
```

It reports every analysis it finds, simplest first — here `kata` 'stone' plus the plural suffix. The parser is not a lookup; it genuinely reverses the generator, so it undoes stacked affixes (`nakatas` → `DEF` + `kata` + `DAT`) and the non-concatenative processes too: full reduplication (`katakata` → `REDUP` + `kata`), partial reduplication where only the first syllable copies (`kakata` → `ka~` + `kata`), and ablaut, the internal vowel change of **sing/sang** (a past tense that rewrites `a → i` turns `kat` into `kit`, which the parser reads back as `kat` + `PST`).

> **TERM Lemma**
>
> The dictionary form of a word — its root headword, the form you would look up. Parsing an inflected surface form recovers its lemma; the corpus tools of the next chapters use the lemma to gather every inflected form of a word together.

Ablaut can't be reversed by running the rule backwards — a rule that turns every `a` into `i` can't be told which `i`s were once `a`s. So the parser tests forward instead: it runs each root through the ablaut rules and remembers the results, and a surface word that matches one of them is analysed as that root plus the ablaut morpheme. This is why the parser needs no special cleverness for irregularity — it only ever asks "could the generator have produced this?"

Who did what to whom: argument linking

A verb has a **valence** — the number of participants it takes — and each participant plays a role. `link` works out those roles for a clause from the verb's valence:

```
inkhaven language link Eldar --verb see --args "she, bird"
```

For each argument it reports three things at once: its **thematic role** (is it the agent, the patient, the theme, the recipient?), its **macrorole** in the Role-and-Reference-

Grammar sense (the actor or the undergoer), and its **grammatical relation** (subject, object, indirect object). A transitive clause draws the default linking — the higher-ranking argument is the actor and subject, the lower the undergoer and object — and where the argument count doesn't match the declared valence, it says so.

> TERM **Thematic role**
>
> The semantic part a participant plays in an event: **agent** (the doer), **patient** (the affected), **theme** (the thing moved or located), **recipient**, and so on. Distinct from the grammatical relation (subject/object), which is about form — languages map the two differently, which is much of what makes their syntax vary.

The shape of a sentence: the X-bar tree

Sentences are not flat strings of words; they are nested. `tree` draws that nesting as an **X-bar** phrase-structure tree — the `CP → TP → VP` scaffold generative syntax assumes — from a clause's verb and arguments:

```
inkhaven language tree Eldar --verb sees --args "she, bird"
```

```
CP
├─ C ∅
└─ TP
   ├─ NP
   │  └─ N she
   └─ T'
      ├─ T ∅
      └─ VP
         └─ V'
            ├─ V sees
            └─ NP
               └─ N bird
```

Specifiers sit on the left — the subject is the specifier of `TP` — and the one parameter that varies is the order of a head and its complement, which Inkhaven reads from your language's declared word order. A head-final (SOV) language reverses the tree correctly: the object precedes the verb inside `VP`, `VP` precedes `T`, and `TP` precedes

`C`. Ask for the tree of a clause in a language whose word order you have set, and you can see at a glance whether the grammar you declared builds the constituents you meant.

Moving a piece: fronting and traces

Questions and topicalisation **move** a constituent to the front of the clause. `movement` performs that operation over the tree, leaving a coindexed **trace** where the moved piece came from:

```
inkhaven language movement Eldar --verb sees --args "she, bird"
--move object
```

The object lifts to the specifier of `CP` as $\texttt{NP}_1$, and a trace $\texttt{t}_1$ marks the gap it left in the verb phrase — the derivation made visible. This is the machinery behind **"the bird, she sees ⟨t⟩"** and behind wh-questions.

> **TERM Trace**
>
> The silent placeholder a moved constituent leaves behind, coindexed with it ($\texttt{NP}_1 \ldots \texttt{t}_1$). Traces are how generative syntax keeps track of where a fronted or questioned phrase is interpreted, even though it is pronounced somewhere else.

Who can refer to whom: binding

When can two noun phrases in a clause refer to the same person? **"She sees herself"** works; **"herself sees her"** does not. `binding` decides such questions from the tree and the three binding principles:

```
inkhaven language binding Eldar --verb sees --args "she, herself"
--type reflexive
```

The engine is **c-command** — a node c-commands its sibling and everything the sibling contains — plus the principles: an anaphor (a reflexive) must be bound by a c-commanding antecedent in its clause (Principle A); a pronoun must **not** be (Principle B); a name must be free everywhere (Principle C). So a reflexive object **may** refer to the subject that c-commands it, a pronoun in the same slot **may not**,

and a name there is a Principle-C violation. The relation is correctly asymmetric — the subject c-commands the object, but not the reverse — which is exactly why **"herself sees her"** fails.

> TERM **C-command**
>
> A structural relation: a node c-commands its sibling and everything beneath that sibling. It is the backbone of binding, of how far a fronted phrase can move, and of much else in syntax — a surprising amount of grammar turns out to depend not on linear order but on this "is it my sibling's descendant?" question.

At your fingertips

Three of these live in the Linguistic companion's chat as well, for when you want to check a structure while you think: `/parse <word>`, `/tree <verb> <subject> [object]` and `/clause <verb> <subject> …` (the clause check of the next chapter). Each runs over the language your cursor sits in and prints inline.

> WHAT YOU LEARNED
>
> - `parse` reverses generation: it strips affixes — concatenative, reduplication, ablaut — to recover a word's root and its morphemes, the inverse of the paradigm engine.
> - `link` works out a clause's argument structure: each argument's thematic role, its actor/undergoer macrorole, and its grammatical relation, from the verb's valence.
> - `tree` draws the X-bar phrase-structure tree (`CP → TP → VP`), placing heads and complements by your language's word order; `movement` fronts a constituent leaving a trace; `binding` decides coreference by c-command and Principles A/B/C.
> - All are read-only: they analyse the language you declared and never change it.

CHAPTER 31

31

The Oracle: judging well-formedness

The parser of the last chapter tells you how a word is **built**. The Oracle tells you whether it **could exist** — whether a candidate word or clause is well-formed by your language's own rules, and if not, why not. Where `audit` scans the finished lexicon, the Oracle judges arbitrary input: a word you are about to coin, a sentence you just wrote, a form you found in the prose and can't place. It answers in levels, from the sounds up to the syntax.

Is this a possible word?

`check` runs the Oracle over a single candidate word:

```
inkhaven language check Eldar --word ktazil
```

It works upward through two levels. First **phonotactics**: are all the segments in your inventory, and does the word obey your syllable-structure constraints? A stray foreign sound or an illegal cluster is caught here. Then **morphology**: does the word analyse — as a listed root, or as a root plus affixes the parser can strip to reach one? A word that is phonotactically fine but reaches no root is flagged, so you can decide whether to coin it or reject it. A clean word gets a single line saying so.

> TERM **Well-formedness**
>
> Whether a form obeys the rules of its language — not whether it **means** anything, but whether it **could be** a word or sentence of the language at all. Native speakers judge well-formedness instantly and unconsciously; the Oracle makes your invented language's judgements explicit, so you can hold it to its own standards.

Is this a possible clause?

Words are levels one and two; a clause adds two more. `check-clause` judges a whole sentence:

```
inkhaven language check-clause Eldar --verb katai --args "she,
bird" \
  --verb-root kata --subject-features "number=pl"
```

Agreement (level three): given the subject's grammatical features and the verb's root, the Oracle regenerates the verb form your agreement rule **should** produce and compares it to the one you wrote. A plural subject with a singular verb is flagged, and the expected form is named. **Argument structure** (level four): reusing the argument linker, it checks that the number of arguments matches the verb's valence — an intransitive verb handed two arguments is caught, a transitive one handed two is not.

Agreement is not only subject–verb. `check-agreement` generalises the check to **any** head–dependent pair — an adjective with its noun, a determiner with its noun:

```
inkhaven language check-agreement Eldar --dependent adjective \
  --form mira --root mira --head-features "number=pl"
```

It looks up the agreement rule you declared for that dependent, regenerates the form it should take, and flags the one you gave if it differs. A dependent your language declares no agreement rule for is silently passed — there is nothing to enforce.

The Oracle watching your prose

You met the Oracle already, in the healthy-lexicon chapter: when you save a paragraph, the **phonotactic guardian** checks the conlang words in it and flags any that break your language's rules, as an advisory finding in the Output pane. That is the same level-one judgement, run automatically over your manuscript rather than on a word you typed. The whole-book sweep in the review pass (`Ctrl+B J`) runs it across every chapter at once. So the Oracle works three ways: on demand over a word or clause, as you save, and across the finished manuscript — the same standard applied wherever ill-formed conlang words might hide.

It is always advisory. The Oracle reports; it never edits your prose or rejects your coinage. A language is yours to break when you mean to — the Oracle only makes sure you meant to.

At your fingertips

In the companion, `/check <word>` runs the word Oracle inline and `/clause <verb> <subject> …` runs the clause Oracle's argument-structure check (agreement needs the subject's features, so that half stays on the command line).

WHAT YOU LEARNED

- `check <word>` judges a candidate word by level: phonotactics (legal segments and clusters) then morphology (does it analyse to a root?).
- `check-clause` adds two levels over a sentence: subject–verb agreement (does the verb inflect for its subject?) and argument structure (does the argument count fit the valence?); `check-agreement` opens the agreement check to any head–dependent pair.
- The same phonotactic judgement runs automatically on save and across the whole manuscript in the review pass, flagging ill-formed conlang words in your prose.
- Every Oracle finding is advisory — it reports and never changes your writing.

CHAPTER 32

32

Interlinear glossed text

A dictionary lists words in isolation; a grammar states rules in the abstract. What a language **is** lives in between, in sentences — and the way linguists write a sentence down so a reader who doesn't speak the language can follow it is **interlinear glossed text**: the sentence, a gloss under each word, and a free translation, lined up in columns. It is the single most characteristic artifact of linguistic description, and Inkhaven builds it for you.

Glossing a sentence

`igt` takes a sentence of your language and lays it out as a Leipzig interlinear:

```
inkhaven language igt Eldar --text "katat nilo"
```

```
kata-t     nilo
stone-DAT  friend
'stone friend'
```

The top line is the sentence **segmented into morphemes** — split at every affix boundary — and the line under it glosses each morpheme, aligned. This reuses the auto-gloss engine, so every inflected form is recognised, and it is careful about the hard case: an affix whose sound changes at the boundary is still split correctly. A dative **kata + d** that surfaces as **katat** after your final-devoicing rule is segmented `kata-t`, not `kata-d` — Inkhaven tracks each morpheme's span and re-slices the surface **after** allophony has run. The third line is a **literal** translation, the recognised words' lexical meanings, offered as a scaffold you will rewrite. Words it doesn't recognise — a name, a coinage you haven't defined — pass through untouched.

> TERM **Interlinear glossed text (IGT)**
>
> The standard three-part format for presenting an example from any language: the (morpheme-segmented) text, a morpheme-by-morpheme gloss using standard abbreviations (`PL`, `DAT`, `PST`…), and a free translation in quotes. The alignment lets a reader see exactly how the meaning is assembled — which is why every grammar and every linguistics paper is full of it.

Keeping your texts

A glossed sentence is worth keeping. `--save` stores an interlinear in a `Texts` chapter of your language book, next to its phonology and lexicon:

```
inkhaven language igt Eldar --text "katat nilo" --save --name
the-oath-1
```

`inkhaven language texts Eldar` then lists what you have gathered, and `--name` prints one back. A stored text lives in the tree — versioned, navigable, and part of what makes the language **documented**. Over time these texts become a small corpus, and the corpus tools you met while measuring your language — `frequency`, `concordance`, `collocations` — read them (together with the

conlang words in your manuscript prose) to answer questions about how the language is actually used.

Curating the translation

The literal translation is only a starting point. Replace it with the sentence as it should read:

```
inkhaven language texts Eldar --name the-oath-1 \
  --set-translation "The oath was sworn upon the stone."
```

Only the free translation changes; the morpheme segmentation and its gloss are left exactly as they were. In the companion the same edit is `/settrans the-oath-1 = The oath was sworn upon the stone.`, and `/texts` lists what you have stored.

Out to the page

When you want your texts in a grammar sketch or a paper, `--format latex` renders them as a `linguex` document — each text a numbered example with `\gll` (the segmented sentence over its gloss) and `\glt` (the translation), ready to paste:

```
inkhaven language texts Eldar --format latex
```

Every LaTeX special is escaped, and a multi-word gloss is bound so the two lines stay aligned. What you glossed in the terminal comes out as the numbered, typeset examples a linguist expects.

For machine consumption rather than the page, `--format conllu` emits a **CoNLL-U** file — the interchange format of the Universal Dependencies project. Each word becomes a row with its form, lemma, part of speech, and grammatical features (read straight off the gloss: a `-PL` tag becomes `Number=Plur`), ready to load into any UD tool:

```
inkhaven language texts Eldar --format conllu
```

WHAT YOU LEARNED

- `igt <lang> --text "…"` glosses a sentence as an aligned interlinear: the morpheme-segmented sentence, a gloss under each morpheme, and a literal translation.
- `--save --name N` keeps a text in the language's `Texts` chapter; `texts <lang>` lists them and `--name N` prints one — a growing corpus of documented sentences.
- `texts --name N --set-translation "…"` curates a text's free translation without touching its gloss; `--format latex` exports the texts as a `linguex` document, and `--format conllu` as a Universal Dependencies (CoNLL-U) treebank.
- In the companion, `/igt`, `/texts` and `/settrans` do the same inline.

APPENDIX A

A

Command reference

Every conlang command, grouped by what it does. Most follow the shape `inkhaven language <action> <language> [options]`. A few order their arguments differently — `cross <from> <to>`, `link-place <place> <lang>`, `reconstruct --forms` — and the project-wide `list`, `family-tree`, and `ecology` take no language at all. Add `--help` to any command for its full signature.

Two conventions hold across the family (Inkhaven 3.18). A command that writes a file you name takes `-o F` (long form `--output`; the older `--out`, and `--svg` on `ecology`, still work as hidden aliases); leave it off and a printable command prints, while a font lands under the project's `exports/` folder. A command whose

result a script might read takes `--json`: the result is then one JSON document on stdout and nothing else, with any confirmation on stderr. Every `--count` must be at least 1.

Setup

`init <name>` Create a language sub-book with its five chapters.

`scaffold --from "<description>" [-o F]` (AI) Propose a starter phoneme inventory from a short description, as pasteable Phonology-chapter HJSON (preview-only).

`list [--json]` List every defined language with summary counts (`--json`: `[{name, entries, grammar, phonology, samples}]`).

`doctor <lang> [--json]` A health check — phonotactic and lexical problems plus how much of your manuscript's language-words the lexicon already covers.

Phonology

`generate-word <lang> --role root --count N [--json]` Invent word-shapes that obey your templates and constraints (`N` is 1 … 100 000; an empty batch is an error that names the count).

`syllabify <lang> --word W [--json]` Show how a word divides into syllables.

`ipa <lang> --word W [--json]` Show a word's **surface** pronunciation after allophony.

`stress <lang> --word W [--json]` Mark which syllable carries stress.

`romanize <lang> --text T [--reverse] [--scheme S] [--json]` Convert between spelling and sounds.

`tone <lang> --tones "3 3 3" [--json]` Apply tone-sandhi rules to a tone sequence.

`scan <lang> --text "…" [--json]` Scan a line or stanza of verse — each syllable's beat and the metre it fits.

Lexicon

`add-word <lang> <word> --type POS --translation M` Add a dictionary entry (or `--import file.csv`).

`remove-word <lang> <word>` Delete an entry.

`sort-dictionary <lang> [--dry-run]` Put the Dictionary chapter into the declared alphabet's order (buckets and entries); `--dry-run` lists the moves.

`query <lang> [--pos] [--register] [--domain] [--era] [--text] [--json]`
Search the lexicon.

`generate-lexicon <lang> --topic T --count N [--semantic] [--yes]` AI-generate themed words behind the dedup gate.

`audit <lang> [--json]` Report phonotactic violations, homophones, duplicate meanings.

`scan-manuscript <lang> [--json]` Find language-like words in your prose that are not yet defined.

`stats <lang> [--json]` A descriptive profile of the language's sounds and words.

`gaps <lang> [--scope swadesh_100|file.hjson] [--json]` Report which reference concepts the lexicon is still missing.

`compose <lang> --kind names|prose|poem|blessing|curse|incantation|ceremony [--count N] [--meter 5,7,5] [--seed N] [--provider P]`
Generate creative text grounded in your language.

`import <lang> --file F --format toolbox|polyglot [--yes]` Import a lexicon from another tool (previews without `--yes`).

`export <lang> --format json|csv|anki|xliff|linguex|ipa-chart|dictionary-twocol|grammar|phrasebook [-o F]` Export the lexicon to a portable or typeset format.

Grammar

`paradigm <lang> --root R --template T --gloss G [--json]` Generate a word's inflected forms.

`agree <lang> --word W --pos P --features "number=pl,case=nom" [--gloss G]`
Inflect a dependent word to agree with its head's features.

`gloss <lang> --text "…" [--json]` Interlinear (word-by-word) gloss of a sentence.

`igt <lang> --text "…" [--save] [--name N] [--json]` Interlinear glossed text — auto-gloss a sentence and lay it out as an aligned Leipzig block

(the morpheme-segmented sentence, the gloss, a literal translation). `--save` stores it in the language's `Texts` chapter; `--json` emits the structured IGT.

`texts <lang> [--name N] [--set-translation "…"] [--format text|latex] [--json]` The language's stored interlinear texts. With no `--name`, list them; with `--name`, print that one. `--set-translation` replaces the named text's free translation (curating the auto literal one; the gloss is untouched). `--format latex` emits a `linguex` LaTeX document (`\gll` / `\glt`) of the selected text or all — ready to paste into a grammar sketch or paper.

`frequency <lang> [--lemma] [--source texts|prose|all] [--top N] [--json]` A word-frequency list and descriptive statistics (tokens, types, type–token ratio, the Zipf fit) over the corpus. `--lemma` counts a root's inflected forms together.

`concordance <lang> --word W [--lemma] [--source texts|prose|all] [--window N] [--json]` A KWIC concordance — every occurrence of a word across the corpus, with its context. `--lemma` matches a root's inflected forms too.

`collocations <lang> --word W [--lemma] [--source texts|prose|all] [--window N] [--top N] [--json]`

The collocates of a word — the neighbours within its window, ranked by co-occurrence and scored by PMI (a distinctive collocate outranks a merely-frequent word).

The corpus `--source` selects what the three commands read: `texts` (the stored interlinears), `prose` (the conlang words used in your manuscript, detected as `scan-manuscript` does), or `all` (both, the default).

`parse <lang> --word W [--json]` Analyse a surface word into root + affixes (the morphological parser — the inverse of paradigm generation).

`link <lang> --verb V --args "a,b,c" [--valence V] [--json]` Work out a clause's thematic roles, RRG macroroles (actor / undergoer) and grammatical relations from the verb's valence.

`tree <lang> --verb V --args "subj,obj,iobj" [--word-order O] [--json]`

Build the X-bar phrase-structure tree (CP → TP → VP) of a clause, placing heads and complements by word order.

`movement <lang> --verb V --args "…" --move ROLE [--word-order O] [--json]`

Front a constituent (wh-movement / topicalisation) over the tree, leaving a coindexed trace.

`binding <lang> --verb V --args "…" [--antecedent R] [--anaphor R] [--type reflexive|pronoun|name] [--json]`
: Decide whether one argument may refer to another, by c-command and the binding principles.

`derive <lang> --root R --gloss G --pos P [--yes]`
: Coin derived words from a root.

`grammar <lang> [--set feature=value]`
: View or set the typology questionnaire.

`define-rule <lang>`
: Open your `$EDITOR` to hand-author a grammar or phonology rule — the direct authoring path when a rule is easier written than configured.

`sentence <lang> --subject W --verb W --object W [--*-adj W] [--*-number N] [--negate --negator W] [--question --q-particle W] [--json]`
: Assemble a clause — order, case, agreement, optional negation / yes-no question — with an interlinear gloss.

`relative <lang> --head H --role subject|object --verb V [--with O] [--relativizer W]`
: Build a noun phrase modified by a relative clause.

`coordinate <lang> [--np W …] [--clause "subj verb obj" …] --conjunction W`
: Join nouns or clauses with a conjunction.

`complement <lang> --subject S --verb V [--complementizer W] --comp-subject S2 --comp-verb V2 [--comp-object O2]`
: Make a clause the object of a matrix verb ("I know that …").

`idiom-add <lang> --form F --literal L --meaning M [--register R]`
: Record an idiom.

`metaphor-add <lang> --source S --target T [--example E]`
: Record a conceptual metaphor.

`idioms <lang>`
: List recorded idioms and metaphors.

Diachronics

`sound-change <lang> --form W [--json]`
: Evolve one proto-form through the language's sound-change chain.

`derive-lexicon <lang> [--yes]`
: Evolve the proto's whole dictionary into this daughter.

`family-tree [--json]` Draw the genealogical tree of all languages (and the contact edges).

`cognates <proto> --form W [--json]` Trace a proto-form's reflex in every daughter.

`reconstruct --forms "…" [--gloss G]` (AI) Propose a proto-form from cognates.

`realism-check <lang>` (AI) Assess whether the sound-change chain is plausible.

`trace <lang> --rule "s > ʃ / _ i" [--limit N] [--json]` Preview a pending sound change across the lexicon — which words shift, which merge into new homophones — without committing it.

`hypothesize <lang> --kind sound-change|cognacy|borrowing|other --claim "…" [--note "…"] [--evidence "…" …] [--id N] [--json]`
Record a diachronic/comparative hypothesis in the `Hypotheses` chapter, starting at `proposed`.

`hypotheses <lang> [--status proposed|supported|refuted|retired] [--json]` List the hypothesis register, optionally filtered by status.

`hypothesis <lang> --id N [--json]` Show one hypothesis in full (claim, status, note, evidence).

`hypothesis-status <lang> --id N --status S` Move a hypothesis along as the evidence comes in.

`hypothesis-check <lang> --id N [--limit N] [--json]` Run the Consequence Tracer over a sound-change hypothesis's claim — the words it predicts will shift, and any that merge into homophones — turning the claim into a concrete, testable prediction.

Analysis

`metrics <lang> [--json]` Quantitative sound-system metrics — phoneme entropy, the Zipf fit, phonotactic saturation, mora weight.

`naturalness <lang> [--json]` Judge the phoneme inventory against cross-linguistic tendencies (voicing symmetry, place coverage, near-universals, size) into a 0–1 score.

`suggest-phonemes <lang> [--json]` Recommend phonemes that would round out the inventory (voiced counterparts, missing near-universals). Advisory.

`pairs <lang> [--limit N] [--json]` Find minimal pairs and the distinctive feature each turns on — the functional load of your contrasts.

`harmony <lang> [--json]` Detect vowel harmony (backness, rounding) by how consistently a word's vowels agree.

`distribution <lang> [--json]` Where each phoneme appears (onset / nucleus / coda, word edges) and any restricted distributions.

`universals <lang> [--json]` Check the grammar's head-directionality harmony and the classic implicational universals (Greenberg/Dryer).

`grammar-check <lang> [--json]` Validate the typed grammar blocks (`ug_parameters`, `verb_classes`) and their consistency with the WALS feature answers.

`check <lang> --word W [--json]` The Oracle — judge a candidate word for well-formedness by level (phonotactics, morphology).

`sketch <lang> [-o F]` A one-page prose overview of the language, assembling all of the above.

Morphology and syntax

`parse <lang> --word W [--json]` The morphological parser — analyse a surface word into root + affixes by stripping known affixes (concatenative, full and partial reduplication, and ablaut) until what remains is a dictionary root. The inverse of paradigm generation.

`link <lang> --verb V --args "A,B,C" [--valence …] [--json]` Argument linking — a clause's thematic roles, RRG macroroles (actor / undergoer) and grammatical relations from the verb's valence.

`tree <lang> --verb V --args "subj,obj,iobj" [--word-order O] [--json]`

Build the X-bar phrase-structure tree (CP → TP → VP), placing heads and complements by the language's word order.

`movement <lang> --verb V --args "…" --move ROLE [--word-order O] [--json]`

Syntactic movement — front a constituent (wh-movement / topicalisation) over the tree, leaving a coindexed trace.

`binding <lang> --verb V --args "…" [--antecedent R] [--anaphor R] [--type reflexive|pronoun|name] [--json]`

Binding theory — decide whether one argument may refer to another, by c-command and Principles A / B / C.

`check-clause <lang> --verb V --args "…" [--verb-root R] [--subject-features "number=pl,…"] [--valence …] [--json]` The Oracle over a clause (levels 3–4) — subject–verb agreement (does the verb inflect for the subject's features?) and argument structure (does the argument count match the verb's valence?).

`check-agreement <lang> --dependent D --form W --root R --head-features "number=pl,gender=fem" [--json]` The Oracle's agreement check over any head–dependent pair — does a dependent word (adjective, determiner, verb) correctly inflect for its head's features, under the declared agreement rule?

`trace <lang> --rule "X > Y / A _ B" [--limit N] [--json]` The Consequence Tracer — preview a sound change across the lexicon (which words shift, which distinctions merge, which new homophones appear) without committing it.

The Linguistic companion

`inkhaven linguistic [--language L] [--session S]` Open the full-screen Linguistic companion over the `Language` book — the tree, a grounded chat with the Inner Linguist, and the phonology / universals / minimal-pair views. Not a `language` subcommand; run it directly.

Chat slash-commands `/trace <rule>` previews a sound change, `/parse <word>` analyses a surface form, `/check <word>` runs the word Oracle, `/tree <verb> <subject> [object] [indirect]` builds the X-bar tree, `/clause <verb> <subject> …` runs the clause Oracle's argument-structure check, `/igt <sentence>` glosses a sentence as interlinear text, `/texts` lists the stored texts, `/settrans <name> = <translation>` curates a stored text's free translation, `/frequency` reports corpus statistics, `/kwic <word>` is a concordance over the corpus, `/coll <word>` lists a word's collocates, `/hypotheses` lists the hypothesis register, and `/hcheck <id>` traces a sound-change hypothesis's consequences. Most run locally over the current language book and print inline; only `/settrans` writes (the text's translation).

Sociolinguistics and contact

`varieties <lang> [--json]` List the language's dialects, registers, and sociolects with their deltas.

`lect <lang> <variety> --word W | --text "…"` Render a form or text in a chosen variety.

`dialects <lang> [--count N] [--json]` Print the dialectology comparison table (a `*` marks a word override).

`borrow <recipient> --form W --from L [--gloss G] [--yes]` Adapt a donor word to the recipient's sounds (add it with `--yes`).

`areal [lang]` Show one language's convergence overlay, or (no language) the whole-region Sprachbund view.

`propose-dialect <lang> --describe "…" [--id N] [--provider P] [--yes]` (AI) Suggest a coherent dialect; rules are validated and previewed.

`propose-loans <recipient> --from L --topic T --count N [--provider P]` (AI) Propose borrowings, each nativised by the adapter.

`areal-check <lang>` (AI) Judge whether a declared Sprachbund is typologically plausible.

`ecology [-o F.svg]` Report who speaks what variety where (or write a node-link atlas).

`idiolect <character> --word W | --text "…"` Render a form or text in a character's native variety.

Translation

A rule-based translation engine that runs on the phonology, lexicon, and grammar you built — with a growing memory of the sentences you approve.

`translate <lang> --text "…"` Translate English into your language, using its lexicon and grammar.

`reverse <lang> --text "…"` Translate a sentence in your language back into English.

`cross <from> <to> --text "…"` Translate between two of your own languages.

`remember <lang> --source "…" --target "…"` Add an approved sentence pair to the translation memory.

`memory <lang> [--json]` Show the translation memory — the approved pairs the engine reuses.

`corpus <lang> [--json]` Show the parallel corpus gathered from your Sample-texts chapter.

`eval <lang> [--json]` Score the engine against the corpus — how much it gets right, and where it fails.

`export-translation <lang> -o F` Write an `.itm` translation-memory pack for backup or interchange.

Writing systems

`glyph-lint --svg F` Check whether an SVG is suitable as a font glyph.

`glyph-draft <lang> --describe "…" --phoneme P [--codepoint C] [-o F] [--yes]`
(AI) Draft a glyph from a description.

`font-import-glyph <lang> --svg F --phoneme P [--codepoint C] [--name N]`
Bind a glyph into the font.

`font-config <lang> [--json]` List the font's glyph bindings and artwork status.

`font-build --language <lang> --format ufo|ttf|both [-o F] [--upm N]`
Compile the font.

`font-templates <lang>` List spatial templates for composed blocks.

`font-compose <lang> --template T --name N --slot SLOT=GLYPH … [--yes]`
Bake a composed block glyph.

`spatial-typst <lang> --template T --name N --slot SLOT=GLYPH …`
Emit a layout-time Typst quadrat.

`transliterate <lang> --text "…" [--json]` Type romanized text into the script's codepoints.

Output

`dictionary <lang> --format md|typ [-o F] [--font Fam]` Render the dictionary.

`grammar-book <lang> --format md|typ [-o F] [--font Fam] [--study --provider P]`
Render the reference grammar (with optional AI study guide).

`tutorial <lang> --format md|typ [-o F] [--font Fam] [--provider P]` (AI) Render a learner's textbook.

Worldbuilding links

`link-place <place> <lang> [--secondary] [--variety V]` Link a place to a language (and the variety) spoken there.

`link-character <char> <lang> <level> [--native-variety V]` Record a character's fluency, and their native variety.

`speakers <lang>` List who speaks a language.

APPENDIX B

B

Configuration block reference

The HJSON blocks you place into your language's chapters, in one place. Remember the rule: quote short word-like values (`kind: "consonant"`), and run `inkhaven reindex --adopt` after editing files by hand.

Meta chapter — the overview block

The `Meta/overview` paragraph (seeded by `language init`) holds the language's identity and the defaults the rest of the suite reads. Every key is optional:

```
{
  name: "Eldar"
  family: "Elvish"
  language_kind: "constructed"   // "constructed" | "natural"
  iso_code: "qya"                // optional ISO 639-3 code
  alphabet: ["Aa", "Bb", "Cc"]   // canonical order; sorts the
Dictionary, the book, exports
  reading_direction: "ltr"       // "ltr" | "rtl"
  script: "latin"
  word_order: "SVO"
  morphology: "fusional"
  tonal: false
  has_cases: true
  has_gender: false
  stemmer: "suffix"              // how search strips inflection
  example_corpus_ref: ""
  notes: ""
}
```

A few of these do real work: `language_kind` tightens how strictly the AI translator adheres to your rules (`constructed`) versus leaning on its pretraining (`natural`); `alphabet` orders the Dictionary — its letter buckets, the entries inside them, the dictionary book and every export — and decides where `add-word` files a word (`sort-dictionary` repairs an older dictionary); `reading_direction` and `script` shape rendering; `stemmer` governs how `query` strips inflection when it searches. The rest are descriptive.

Phonology chapter

The full phonology block. Every field except `phonemes` is optional.

```
{
  phonemes: [
    { ipa: "k", romanize: "k", kind: "consonant" }   // kind:
consonant | vowel
    { ipa: "a", romanize: "a", kind: "vowel" }
  ]
  classes: { C: ["k"], V: ["a"] }                     // named
phoneme groups
  templates: { root: [ { pattern: "C V (C)", weight: 1.0 } ] }
  constraints: [
```

```
    { kind: "max_cluster_size", value: 2 }
    { kind: "no_geminate" }
    { kind: "forbid_in_coda", classes: ["Stop"] }     // or
forbid_in_onset
    { kind: "sonority_sequencing" }
  ]
  allophony: [ { name: "palatalization", rule: "k > tʃ / _ i" } ]
  stress: "penultimate"        // initial | final | penultimate |
antepenultimate | latin
  romanizations: [
    { name: "default"
      mappings: [ { ipa: "k", roman: "c" } ]
      contextual: [ { roman: "c", ipa: "s", before:
"FrontV" } ] }
  ]
  default_romanization: "default"
  tone: { kind: "contour", tones: ["1","2","3"], sandhi:
[ { rule: "3 > 2 / _ 3" } ] }
}
```

A separate block, also in the Phonology chapter, declares descent from a proto:

```
{ diachronics: {
    proto: "ProtoEldarin"
    rules: [ { rule: "p > f / _ #" }, { rule: "k > h / V _ V" } ]
} }
```

The `font` block (built for you by `font-import-glyph`) also lives here:

```
font: {
  family: "Eldar"
  upm: 1000
  glyphs: [
    { name: "a", codepoint: "a", phoneme: "a" }
    { name: "sun", codepoint: "U+E000", phoneme: "o" }
  ]
}
```

The `loan_phonology` block — how the language nativises borrowings (Chapter 18) — also lives in the Phonology chapter:

```
{ loan_phonology: {
  repair: "epenthesis"                    // epenthesis (insert a
vowel) | deletion
  epenthetic_vowel: "u"                   // empty → the first
declared vowel
  substitutions: { "θ": "t", "r": "l" }   // a donor sound we
lack → nearest native
} }
```

Grammar chapter

The morphology block — affixes, processes, paradigms, derivations, agreement:

```
{
  morphemes: [
    // Concatenative affixes. position: prefix | suffix | infix |
circumfix.
    // Optional precedence: 0 = any (keep declared order), 1 =
next to the root, …
    // Optional category / value tag the affix for the grammar
book.
    { id: "dat", gloss: "DAT", form: "ti", position: "suffix",
      precedence: 1, category: "case",   value: "dative" }
    { id: "pl",  gloss: "PL",  form: "i",  position: "suffix",
      precedence: 2, category: "number", value: "plural" }
    { id: "ag",  gloss: "AG",  form: "um", position: "infix",
anchor: "before_first_vowel" }
    { id: "ptcp", gloss: "PTCP", form: "ge_t", position:
"circumfix" }   // ge_ + stem + _t
    // Non-concatenative processes.
    { id: "pst", gloss: "PST", process: "ablaut", rules:
[ { rule: "i > a" } ] }
    { id: "rdp", gloss: "PL",  process: "reduplication",
reduplicate: "initial_cv" }
    // reduplicate: full | initial_cv | initial_syllable |
final_syllable
  ]
  paradigms: [ { name: "noun", cells: [
    { features: { number: "sg", case: "nom" }, morphemes: [] }
    { features: { number: "pl", case: "dat" }, morphemes:
["dat","pl"] }
```

```
  ] } ]
  derivations: [
    { name: "agent", form: "ron", position: "suffix",
      from_pos: "verb", to_pos: "noun", gloss_template: "one who
{}s" }
  ]
  agreement: [
    { dependent: "adjective", head: "noun", features:
["number","case"], paradigm: "adj" }
  ]
}
```

The typology answers (written for you by `grammar --set`) and idioms / metaphors (written by `idiom-add` / `metaphor-add`) are also stored in this chapter.

The `varieties` block — the language's dialects and registers (Chapter 17) — lives here too. Each variety is a **delta**: an ordered list of `sound_changes` (SPE notation, applied synchronously) plus optional suppletive `lexicon` overrides:

```
{ varieties: [
  { id: "lowland", kind: "dialect", axis: "region", prestige:
"low",
    sound_changes: [ { rule: "t > d / V _ V" } ]
    lexicon: { "water": "móru" } }              // a suppletive
override
  { id: "high", kind: "register", axis: "formality", prestige:
"high",
    sound_changes: [ { rule: "k > q / # _" } ] }
] }                                              // kind: dialect
| register | sociolect | idiolect
```

The `contact` block — the language's membership in a linguistic area / Sprachbund (Chapter 18):

```
{ contact: {
  region: "the Inner Sea"
  with: [ "Sindar", "Khuz" ]                                    //
neighbours
  areal_features: { word_order: "sov", alignment:
"ergative_absolutive" }
} }
```

The `verb_classes` and `ug_parameters` blocks feed the syntax engine (Chapter 30) and the clause Oracle (Chapter 31). A verb class records a verb's **valence** — how many arguments it takes — so `link`, `tree`, `check-clause` and the rest read it rather than guessing from the argument count. The universal-grammar parameters are typological switches that `grammar-check` validates and cross-checks against your typology answers:

```
{ verb_classes: [
    { name: "see",  valence: "transitive",  note: "perception" }
    { name: "give", valence: "ditransitive" }
    { name: "rain", valence: "impersonal" }                    //
intransitive | transitive | ditransitive | impersonal
  ]
  ug_parameters: {
    head_final:  false                                        //
checked against word_order
    pro_drop:    true
    wh_movement: true
  } }
```

Dictionary chapter

Each word is one small block (written for you by `add-word`). The required fields are `word`, `type`, and `translation`; the rest are optional and add depth:

```
{ word: "makil", type: "noun", translation: "sword",
  register: "formal", domain: ["weapon"], era: "third_age",
  pronunciation: "'ma.kil", etymology: "from mak- 'to cut'",
  related: ["makilya", "makta"], inflection: { plural:
"makili" },
  examples: ["The king drew his makil."], notes: "ceremonial, not
a field blade" }
```

`pronunciation`, `etymology`, `related` (cross-references), `inflection` (a small map of forms), `examples`, and `notes` are all preserved and shown in the rendered dictionary and grammar book.

Idioms and conceptual metaphors also live in the Dictionary chapter, written by `idiom-add` / `metaphor-add`, under an `expressions` block:

```
{ expressions: {
  idioms:    [ { form: "cold hands", literal: "hands of ice",
meaning: "a hard bargainer", register: ["colloquial"] } ]
  metaphors: [ { source: "journey", target: "life", examples:
["the long road of his years"], note: "very common" } ]
} }
```

SPE rule notation

Used by `allophony`, `diachronics`, and tone `sandhi`:

```
TARGET > RESULT / LEFT _ RIGHT
```

`_` the position of the target sound.

`#` a word boundary (start or end of a word).

`∅` **or** `0` the empty string — insertion (on the left) or deletion (on the right).

a class name (`V`, `C`, `Stop`) matches any sound in that class.

a bare phoneme (`i`, `k`) matches just that sound.

a space-separated run (`a i`, `t s`) a **sequence** — on the left, several sounds matched in order; on the right, several sounds emitted. `a i > e / _ #` and `t > t s / _ i` are both one rule. A joined token (`ai`) is split against the inventory when the split is unambiguous; `doctor` warns when it is not.

APPENDIX C

C

Bund API reference

Every conlang word reachable from **Bund** (Chapter 26), grouped by what it does. Each is listed by its short `lang.` name; the full form `ink.lang.<name>` works identically. The notation in parentheses is the **stack effect** — the inputs taken from the stack, then `--`, then the outputs left behind:

```
"Eldar" "tap" lang.ipa        // ( lang word -- surface ) → "tav"
```

Where an output is written `{ … }` it is a Bund dictionary whose listed fields a script can read directly; a **list** is a sequence you can loop over. A trailing `provider` argument on the AI words names a non-default model (an empty string uses the configured default).

The safety gate

Words carry a **category** that decides whether a script may run them. Inspectors (`store_read`) are always allowed. The rest are off until you opt in, naming the categories a script may use in `inkhaven.hjson`:

```
scripting: { enabled_categories: ["store_write", "ai_write",
"fs_write"] }
```

The categories are `store_read` (read the project — default on), `store_write` (change the project), `ai_write` (call a language model), and `fs_read` / `fs_write` (read or write files, always inside the project sandbox).

Inspectors — sounds and words

These read the language and return a value; all are `store_read` (always allowed).

`lang.list ( -- rows )` Every defined language, one `{name, entries, grammar, phonology, samples}` dictionary each — the same rows `inkhaven language list --json` prints.

`lang.generate_word ( lang role seed -- word )` Invent a word-shape obeying the templates (a `seed` makes it repeatable).

`lang.generate_words ( lang role count -- list )` The deterministic batch `generate-word --count` prints (1 … 100 000); an error when no candidate passes the constraints.

`lang.syllabify ( lang word -- list )` Split a word into its syllables.

`lang.ipa ( lang word -- surface )` A word's surface pronunciation after allophony.

`lang.stress ( lang word -- marked )` Mark which syllable carries stress.

`lang.tone ( lang tones -- result )` Apply tone sandhi to a tone sequence (`"3 3 3"` → `"2 2 3"`).

`lang.transliterate ( lang text -- script )` Convert romanized text into the script's codepoints.

`lang.gloss ( lang text -- gloss )` Interlinear (word-by-word) gloss of a sentence.

`lang.query ( lang text -- entries )` Search the lexicon; `text` filters by gloss or headword.

Inspectors — grammar and sentences

`lang.paradigm ( lang root template gloss -- rows )` All inflected forms of a word.

`lang.derive ( lang root gloss pos -- forms )` Productive derivations of a word (does not commit them).

`lang.agree ( lang word pos features -- form )` Inflect a word to agree with given features (`"number=pl,case=dat"`).

`lang.sentence ( lang subject verb object -- {surface,gloss,literal} )` Assemble a subject–verb–object clause.

`lang.relative ( lang head role verb with relativizer -- dict )` Build a relative-clause construction.

`lang.complement ( lang subject verb complementizer comp-subject comp-verb comp-object -- dict )` Build a complement-clause sentence.

`lang.coordinate ( lang clause-list conjunction -- dict )` Join clauses (each `"subj verb [obj]"`) with a conjunction.

Inspectors — analysis and history

`lang.stats ( lang -- profile )` A descriptive profile of the language's sounds and words.

`lang.audit ( lang -- report )` Phonotactic violations, homophones, and duplicate meanings.

`lang.gaps ( lang scope -- report )` Reference concepts the lexicon still lacks (`scope` = `"swadesh_100"` or a file path).

`lang.sound_change ( lang form -- evolved )` Evolve a proto-form through the daughter's sound-change chain.

`lang.cognates ( proto form -- reflexes )` A proto-form's reflex in every daughter language.

`lang.family_tree ( -- tree )` The genealogical tree of all languages.

Inspectors — creative text

`lang.names ( lang count seed -- list )` Generate names grounded in the language.

`lang.prose ( lang count seed -- sentences )` Generate deterministic prose clauses.

`lang.poem ( lang meter seed -- lines )` Generate verse to a meter (`"5,7,5"`).

Inspectors — sociolinguistics

`lang.varieties ( lang -- list )` The dialects and registers, each with its delta sizes.

`lang.lect ( lang variety word -- rendered )` Render a form in a chosen variety.

`lang.idiolect ( character word -- rendered )` Render a form in a character's native variety.

`lang.borrow ( recipient donor-form -- {donor,adapted,steps} )` Nativise a loanword (advisory; commit with `add_word`).

`lang.areal ( lang -- {region,with,convergence} )` A language's areal-convergence overlay.

`lang.ecology ( -- {places,characters} )` The whole speech-community picture.

Inspectors — translation

The rule-based translation engine (`store_read`). `remember` and `export_translation` change the project / write a file and appear under their own categories below.

`lang.translate ( lang text -- {surface,gloss,literal} )` Translate English into the language.

`lang.reverse ( lang text -- english )` Translate a sentence in the language back into English.

`lang.cross ( from to text -- {surface,gloss} )` Translate between two of your own languages.

`lang.memory ( lang -- pairs )` The approved translation-memory pairs the engine reuses.

`lang.corpus ( lang -- pairs )` The parallel corpus gathered from the Sample-texts chapter.

`lang.eval ( lang -- report )` Score the engine against the corpus.

Data constructor

Uncategorised (always allowed) — it builds a value, touching nothing.

`lang.dict ( [k v k v …] -- dict )` Turn a flat list into a dictionary. Also answers to `word`, `rule`, `phoneme`, and `block`, so a script reads like the artefact it builds.

Mutators

These change the project; all are `store_write` (enable `"store_write"`).

`lang.init ( name -- )` Create a language and its five chapters.

`lang.define ( lang chapter block -- )` Write an HJSON `block` as a paragraph under a chapter (`Phonology`, `Grammar`, `Sample texts`, `Meta`).

`lang.add_word ( lang word pos translation -- )` Add a dictionary entry.

`lang.remove_word ( lang word -- )` Delete a dictionary entry.

`lang.derive_add ( lang root gloss pos -- count )` Derive **and** commit forms to the lexicon; leaves the number added.

`lang.grammar_set ( lang feature value -- )` Set one typology answer.

`lang.idiom_add ( lang form literal meaning -- )` Add an idiom.

`lang.metaphor_add ( lang source target -- )` Add a conceptual metaphor.

`lang.remember ( lang source target -- )` Add an approved sentence pair to the translation memory.

AI-backed words

These call a language model; all are `ai_write` (enable `"ai_write"`). They are **advisory** — they return data and never write the book, so a script commits what it likes (typically through `add_word`). The trailing `provider` is empty for the default.

`lang.compose ( lang kind provider -- text )` Themed text — `kind` is `blessing`, `curse`, or `incantation` — constrained to the lexicon.

`lang.reconstruct ( forms gloss provider -- text )` Propose a proto-form from cognate forms.

`lang.realism_check ( lang provider -- text )` Assess whether the sound-change chain is plausible.

`lang.generate_lexicon ( lang topic count provider -- words )` Themed words — forms from the deterministic generator, meanings from the AI, behind the dedup gate. Returns survivor `{ word, gloss, pos }` dicts.

File output

These read or write files (always inside the project sandbox).

`lang.glyph_lint ( svg-path -- report )` Check whether an SVG is usable as a glyph. (`fs_read`)

`lang.dictionary ( lang format out font -- out-path )` Render the dictionary (`format` = `md` or `typ`; empty `font` uses the configured one) to a file. (`fs_write`)

`lang.grammar_book ( lang format out font -- out-path )` Render the reference grammar to a file. (`fs_write`)

`lang.font_build ( lang format out -- out-stem )` Compile the font (`format` = `ufo`, `ttf`, or `both`). (`fs_write`)

`lang.glyph_draft ( lang describe phoneme out provider -- out-path )` AI-draft a glyph SVG, preflight it, and write it (advisory). (`fs_write`, calls the AI)

`lang.export_translation ( lang out-path -- out-path )` Write the translation system — memory pairs and lexicon — as a `.itm` pack, as `inkhaven language export-translation` does. `lang.export` is the older name for the same word. (`fs_write`)

APPENDIX D

D

Glossary of terms

Every linguistic and technical term used in this book, defined in one place, in alphabetical order. Italicised words within a definition have their own entry.

Ablaut Marking grammar by changing a sound **inside** the root (**sing** / **sang**), rather than adding an affix.

Affix A morpheme attached to a root — a **prefix** (front), **suffix** (end), **infix** (middle), or **circumfix** (wrapping).

Agent noun A noun naming the doer of an action, derived from a verb (**teach** → **teacher**).

Agreement (concord) The requirement that a dependent word copy grammatical features (number, case, …) from the head it modifies — an adjective with its noun, a verb with its subject.

Allophone One of the several ways a single **phoneme** is actually pronounced, chosen by context.

Allophony The system of context-dependent pronunciation variants within a language.

Alignment How a language marks the subject of a sentence — **nominative–accusative** or **ergative–absolutive**.

Anki A spaced-repetition flashcard program; Inkhaven can export your lexicon as an Anki deck.

API key A secret password that lets your copy of Inkhaven use an **AI provider**.

Areal feature A grammatical trait shared across a **Sprachbund** through contact rather than inheritance — the feature that spread (a shared SOV **word order**, say).

Aspect How an action unfolds in time — whether it is ongoing, completed, or habitual — as opposed to **when** it happens (that is tense).

Borrowing Taking a word from another language and adapting it to your own sounds; the result is a **loanword**.

Cluster Two or more consonants in a row with no vowel between them (the **str** of **string**).

Cognate A word descended from the same ancestral word as a word in a related language (**father** / Latin **pater**).

Codepoint The unique number identifying a character in the Unicode standard (written like **U+0041**).

Circumfix A single affix in two pieces wrapping the root (German **ge-…-t**).

Conceptual metaphor A systematic understanding of one domain in terms of another (LIFE IS A JOURNEY).

CAT tool Computer-assisted-translation software (OmegaT, memoQ, Weblate) that reuses past translations stored in a **translation memory**.

Complement clause A subordinate clause acting as an argument of a matrix verb of speech or thought ("I know **that the bird flies**").

Complementizer The word introducing a **complement clause** ("that"), glossed COMP.

Conjunction A word that joins two elements of the same kind ("and", "or").

Conlang Short for **constructed language** — a language built on purpose.

Consonant A sound made by obstructing the airflow (p, t, k, s, m, …).

Constraint A single **phonotactic** rule that rejects certain words.

Constructed language A language deliberately created, with invented vocabulary and grammar.

Coordination Joining two phrases or clauses of equal status with a **conjunction**.

Contour tone A **tone** that glides during the syllable (rising, falling, dipping), as opposed to a level tone held at one pitch.

Counter The enclosed empty space inside a glyph (the hole in "O"), cut with a reverse-wound outline.

Daughter language A language descended from a given **proto-language**.

Dedup gate The automatic checks that stop word generation from ever creating a duplicate.

Definiteness Whether a noun is marked as specific/known ("the dog") versus general ("a dog").

Derivation Forming a genuinely new word from an existing one (**build → builder**).

Design grid The coordinate square (also called the em-square) every glyph is drawn within; its size is the font's units-per-em.

Diachronics The historical change of a language over time, and how it splits into a family.

Dialect A **variety** of a language tied to a region or community, differing in sound and vocabulary yet mutually intelligible with the rest.

Domain The subject area a word belongs to (weapon, kinship, weather).

Donor / recipient In **borrowing**, the language a word comes from (donor) and the language that takes it in (recipient).

Epenthesis Inserting a sound (usually a vowel) to break up a **cluster** a language forbids (**strike → sutoraiku**); the opposite repair is **deletion**.

Etymology The origin and history of a word — what it descended or was derived from.

Evidentiality Grammatically marking the **source** of information — whether the speaker witnessed it, was told it, or inferred it (as in Quechua and Tariana).

Font A file of drawn glyphs that computers display and print.

Gap (syntactic) The empty slot a noun leaves inside a **relative clause** — the role the head noun plays there but does not itself fill (in "the bird that sees the stone", the bird is the missing subject of "sees").

Geminate A doubled or lengthened consonant (Italian **gatto**).

Gloss (grammatical) A short small-capitals label for a grammatical piece (PL, DAT, PST).

Gloss (meaning) A short statement of a word's meaning in your working language.

Glyph One drawn symbol of a **script**.

Grammar The rules for how words change form and combine into sentences.

Grammatical case Marking a noun's role by changing its form (nominative, accusative, dative, genitive).

Homophone A word that sounds the same as another but means something different.

HJSON A relaxed, human-readable data format used to define a language's blocks.

Idiolect The distinctive way one individual speaks — their personal blend of **dialect**, **register**, and habit, built on a **native variety**.

Idiom A fixed phrase whose meaning is not the sum of its words.

Infix An affix inserted inside the root rather than at an edge (Tagalog **s-um-ulat**).

Inflection Changing a word's form to express grammar, without making a new word (**cat** / **cats**).

Interlinear gloss A word-by-word breakdown lined up under a sentence (Leipzig glossing).

IPA The International Phonetic Alphabet — one symbol per speech sound.

Language ecology The whole pattern of which languages and **varieties** are spoken where, by whom, and in contact with what.

Language family A group of languages descended from a common **proto-language**.

Lexeme A single vocabulary item — one word with its own dictionary entry, independent of the forms it inflects into.

Lexicon The full vocabulary of a language; a single word is a **lexeme**.

linguex A LaTeX package for typesetting numbered, glossed example sentences; an export format for academic writing.

Linguistic area See **Sprachbund**.

Linguistics The scientific study of human language.

Loanword A word taken from a **donor** language into a **recipient** and adapted to its sound system (English **beef**, from French).

Matrix clause The main clause that contains a subordinate one — "I know" in "I know that it rains".

Mood Grammatically marking a speaker's stance toward an event — fact, command, wish, possibility (indicative, imperative, subjunctive…).

Morpheme The smallest unit of a word that carries meaning.

Morphology How words are built from morphemes and change shape for grammar.

Native variety The **variety** a speaker acquired first and speaks by default — the base of their **idiolect**.

Naturalism Making a constructed language feel like it could be a real human language.

Negation Expressing that a clause is **not** the case — by a particle, an affix, or a negative auxiliary.

Nucleus The vowel at the centre of a **syllable**.

Onset The consonant(s) before the vowel in a **syllable**.

Coda The consonant(s) after the vowel in a **syllable**.

Packed script A writing system whose signs are grouped into blocks read as units rather than written in a single line (Egyptian **quadrats**, Korean Hangul syllable blocks).

Paradigm The organised set of all inflected forms of a word.

Part of speech The grammatical category of a word (noun, verb, adjective, adverb).

Phoneme A single distinct sound that can change a word's meaning.

Phoneme class A named group of phonemes that behave alike for some rule (C, V).

Phonology The sound system of a language.

Phonotactics The rules for which sound sequences a language allows.

Polar question A yes/no question, as opposed to a **content** question (who, what, where).

Prestige The social standing attached to a **variety** — whether it is regarded as high (the standard, the court) or low (rural, vernacular).

Private Use Area A range of Unicode codepoints (from **U+E000**) reserved for invented characters.

Proto-language An ancestral language that later languages descend from.

Quadrat A rectangular group of signs arranged and read as a single unit, as in Egyptian hieroglyphic writing.

Reconstruction Working out an unrecorded ancestral word from its surviving descendants.

Reduplication Marking grammar by repeating part or all of the root (**buku** → **buku-buku**).

Register The level of formality a word belongs to (formal, neutral, vulgar, sacred, archaic); also a **variety** of a whole language tied to the situation — ceremonial, plain, intimate — rather than to a region.

Relative clause A clause that modifies a noun, the way "that sees the stone" narrows "the bird"; the modified noun is the **head**.

Relativizer The word that introduces a **relative clause** ("that", "which"), glossed REL.

Romanization Writing a language's sounds with the Latin alphabet.

Sandhi A sound change that happens at a boundary because of neighbouring sounds; **tone sandhi** is when one tone shifts next to another.

Script A system of written symbols for a language.

Sociolect A **variety** tied to a social group — class, trade, or generation — rather than a region.

Sonority How open and resonant a sound is; clusters tend to rise toward the vowel.

Sound change A regular historical shift in pronunciation that turns one language into another.

SPE notation The shorthand `target > result / left _ right` for sound rules.

Speech community The group of people who share a language or **variety**.

Sprachbund (linguistic area) A group of languages grown structurally alike through long contact rather than common descent (the Balkans); the shared traits are **areal features**.

Standard Format (SFM / MDF) A plain-text dictionary format with backslash field markers (`\lx`, `\ps`, `\ge`), shared by Toolbox, FieldWorks, and Lexique Pro.

Stop A consonant made by fully blocking the airflow and releasing it (p, t, k, b, d, g).

Stress The relative emphasis given to one syllable of a word.

Substitution In **borrowing**, mapping a **donor** sound the **recipient** lacks onto the nearest sound it has (**th** heard as **t**).

Suppletion An irregular form supplied outright instead of following the regular rules (**go** / **went**); a **variety**'s word overrides are suppletive.

Surface form How a word is actually pronounced after allophony rules apply.

SVG Scalable Vector Graphics — a drawing made of shapes and curves; each glyph is one.

Swadesh list A standard list of the most basic, universal concepts (here 100 of them), used to gauge how much core vocabulary a language has.

Syllabary A writing system with one symbol per syllable (Japanese kana), rather than one per sound.

Syllable A beat of speech built around a vowel (onset + nucleus + coda).

Syllable template A pattern of class names describing an allowed syllable shape (C V (C)).

Syllable weight Whether a syllable counts as **heavy** (a long vowel or a coda) or **light** (a short vowel, no coda) — some stress rules fall on the heaviest syllable.

Syntax The part of grammar dealing with word order and sentence structure.

Tone The use of pitch to distinguish word meanings.

Translation memory A store of source–target phrase pairs a **CAT tool** reuses; exporting to **XLIFF** makes your lexicon one.

Transliteration Converting text from one script into another (here, Latin ⟶ your script).

TrueType An everyday font format (`.ttf`), ready to install and use.

Typology The classification of languages by their structural features.

UFO Unified Font Object — an editable font source format.

Underlying form The abstract phoneme sequence a word is built from, before allophony.

Variety A consistent, recognisable way of speaking one language — a **dialect**, **register**, **sociolect**, or **idiolect** — not a separate language.

Vowel A sound made with open, unobstructed airflow (a, e, i, o, u).

Word order The default order of subject, verb, and object (SOV, SVO, …).

Word root The core form of a word, carrying its basic meaning, before affixes.

XLIFF An XML interchange file for a **translation memory**, read by **CAT tools**; a lexicon export format.